People's Democratic Republic of Algeria
Ministry of Higher Education and Scientific Research
Higher School of Computer Science
8 May 1945 – Sidi Bel Abbes



ENGINEERING THESIS

To obtain the diploma of **Engineer**
Field: **Computer Science**
Specialty: **Computer Systems Engineering (CIS)**

Theme

Evidence-Based SBOM Recovery from Compiled Android APKs

Design, Implementation, and Evaluation of a Hybrid Static Analysis Pipeline

Presented by: **Fendi Mohamed**

Defense date: **[PLACEHOLDER: Final date in 2026]**

In front of the jury composed of:

[PLACEHOLDER: Jury President]	President
Serai Abdelhak	Supervisor
Manel Baba Ahmed	Co-supervisor
[PLACEHOLDER: Examiner Name]	Examiner

Academic Year: 2025–2026

Dedication

This work is dedicated to my parents,

to my dear sister and brother,

to my entire family,

and to the friends who accompanied and supported me throughout my journey, from elementary school to graduation.

Acknowledgements

First and foremost, I thank God for granting me the strength, patience, and perseverance necessary to complete this work and reach this important stage of my academic journey.

I express my deepest gratitude to my parents and my entire family. Their constant encouragement, sacrifices, trust, and unconditional support have shaped the person I am today. This achievement would not have been possible without the values they taught me and the confidence they placed in me.

I am sincerely grateful to my supervisors, Serai Abdelhak and Manel Baba Ahmed, for their guidance, availability, constructive feedback, and valuable advice throughout this project. Their support helped me address the technical and research challenges encountered during the development of this work.

I also extend my appreciation to the Higher School of Computer Science of Sidi Bel Abbès (ESI-SBA), its teachers, and its staff for the knowledge, academic environment, and opportunities provided throughout my studies. The education I received at the school has been essential to my growth as a computer science engineer.

Finally, I warmly thank all my friends who accompanied me from elementary school through graduation. Their friendship, encouragement, shared efforts, and presence made the difficult moments easier and the successful moments more meaningful. To everyone who contributed, directly or indirectly, to this journey, I offer my heartfelt thanks.

Abstract

Software Bills of Materials support software supply-chain transparency and vulnerability management, but most generation workflows rely on source code, build manifests, or package-manager metadata. In Android audit scenarios, the available artifact may be only a compiled APK, where third-party components are merged into DEX bytecode and transformed by shrinking, optimization, and obfuscation.

This engineering thesis presents a static, evidence-gated APK-to-SBOM pipeline. It extracts archive, manifest, bytecode, resource, and native-file evidence; compares APK features with versioned Maven reference profiles; separates component identity from exact-version confidence; preserves ambiguous alternatives; emits CycloneDX JSON with Package URLs; and performs confidence-aware OSV vulnerability lookup. Source dependency trees are used for evaluation when available, while machine-learning and R8 assistance remain report-only.

The evaluation separates controlled fixtures, clean and partial source-validated APKs, no-Maven-signal controls, full source-denominator coverage, source-known upper-bound recognition, failed experiments, and limitation cases. The system recovers 84/84 expected versions in controlled fixtures. Across the 102 source-validated real APKs that emitted Maven components, 927/1,837 emitted versions are confirmed, which is a 50.5% emitted-version confirmation rate rather than full dependency recall. Four additional source-validated APKs emit no Maven components and are retained as recall controls. Against 12,082 resolved source dependency rows, 927 exact rows are emitted end to end. A separate source-version-constrained lane confirms 6,772/6,772 emitted rows and is reported only as an upper bound.

The resulting MVP demonstrates practical binary-derived SBOM recovery while exposing reference-coverage, matcher-recall, adjacent-version, artifact-family, native-code, hidden-code, scalability, and AI-authority limitations. Its main principle is to prefer auditable partial knowledge and explicit abstention over unsupported exact-version certainty.

Keywords — Android security, APK analysis, software bill of materials, third-party library detection, component attribution, CycloneDX, vulnerability management, R8.

Résumé

Les nomenclatures logicielles soutiennent la transparence de la chaîne d'approvisionnement logicielle et la gestion des vulnérabilités. Cependant, la plupart des méthodes de génération reposent sur le code source, les manifestes de construction ou les métadonnées d'un gestionnaire de paquets. Dans un contexte d'audit Android, le seul artefact disponible peut être une APK compilée, dans laquelle les composants tiers sont fusionnés au bytecode DEX puis transformés par la réduction, l'optimisation et l'obfuscation.

Ce mémoire d'ingénieur présente une chaîne statique, fondée sur des preuves et contrôlée par des seuils, qui transforme une APK en SBOM. Elle extrait les preuves provenant de l'archive, du manifeste, du bytecode, des ressources et des fichiers natifs ; compare les caractéristiques de l'APK à des profils de référence Maven versionnés ; sépare l'identité d'un composant de la confiance dans sa version exacte ; conserve les alternatives ambiguës ; produit un fichier CycloneDX JSON muni d'identifiants Package URL ; et interroge OSV en tenant compte du niveau de confiance. Les arbres de dépendances du code source servent à l'évaluation lorsqu'ils sont disponibles, tandis que l'apprentissage automatique et l'assistance liée à R8 restent limités au rapport.

L'évaluation distingue les applications contrôlées, les APK réelles validées par le code source et classées comme propres ou partielles, les contrôles sans composant Maven émis, la couverture du dénominateur source complet, la reconnaissance majorée avec versions source connues, les expériences rejetées et les cas limites. Le système retrouve 84/84 versions attendues dans les applications contrôlées. Pour les 102 APK réelles validées par le code source qui ont émis des composants Maven, 927/1 837 versions émises sont confirmées, soit un taux de confirmation des versions émises de 50,5%, et non un rappel de toutes les dépendances. Quatre APK supplémentaires validées par le code source n'émettent aucun composant Maven et restent des contrôles de rappel. Par rapport aux 12 082 lignes de dépendances résolues depuis le code source, 927 lignes exactes sont émises de bout en bout. Une voie distincte, contrainte par les versions source, confirme 6 772/6 772 lignes émises et n'est présentée que comme une borne supérieure.

Le MVP obtenu démontre la faisabilité pratique d'une reconstruction de SBOM à partir d'un binaire, tout en exposant ses limites concernant la couverture de la base de référence, le rappel du moteur de correspondance, les versions voisines, les familles d'artefacts, le code natif, le code dissimulé, le passage à l'échelle et l'autorité de l'IA. Son principe central consiste à préférer une connaissance partielle, vérifiable et accompagnée d'une abstention explicite à une certitude non justifiée sur la version exacte.

Mots-clés — sécurité Android, analyse d'APK, nomenclature logicielle, détection de bibliothèques tierces, attribution de composants, CycloneDX, gestion des vulnérabilités, R8.

ملخص

تدعم قوائم مكونات البرمجيات شفافية سلسلة توريد البرمجيات وإدارة الثغرات الأمنية، غير أن معظم مسارات إنشائها تعتمد على الشيفرة المصدرية أو ملفات البناء أو بيانات مدير الحزم. وفي عمليات تدقيق تطبيقات أندرويد، قد لا يتوفر سوى ملف APK مترجم، حيث تندمج المكونات الخارجية داخل شيفرة DEX وتتعرض لتقليص الشيفرة وتحسينها وتمويهها.

تقدم هذه المذكرة الهندسية خط معالجة ساكنا، قائما على الأدلة ومحكوما بضوابط، لاسترجاع قائمة مكونات البرمجيات من ملف APK. يستخرج النظام الأدلة من الأرشيف وملف التصريح والشيفرة والموارد والملفات الأصلية، ثم يقارن خصائص التطبيق بملفات مرجعية لإصدارات مكتبات Maven. كما يفصل بين تحديد هوية المكون والثقة في إصداره الدقيق، ويحافظ على البدائل الملتبسة، ويولد ملف CycloneDX JSON مزودا بمعرفات Package URL، ثم يستعلم من خدمة OSV مع مراعاة درجة الثقة. وتستخدم أشجار تبعيات الشيفرة المصدرية لأغراض التقييم عند توفرها، بينما تظل مساعدة التعلم الآلي وR8 محصورة في التقارير.

يفصل التقييم بين التطبيقات المضبوطة، والتطبيقات الحقيقية المتحقق منها بالمصدر والمصنفة إلى نظيفة وجزئية، وتغطية المقام الكامل لتبعيات المصدر، والتعرف ذي الحد الأعلى المقيد بإصدارات المصدر، والتجارب الفاشلة، وحالات الحدود. استرجع النظام 48/48 إصدارا متوقعا في التطبيقات المضبوطة. ومن بين 201 تطبيق حقيقي متحقق منه بالمصدر أصدر مكونات Maven، تأكدت 738,1/729 نسخة Maven صادرة، أي بنسبة تأكيد 5.05% للإصدارات الصادرة، وليس استدعاء لجميع التبعيات. كما توجد أربعة تطبيقات إضافية متحقق منها بالمصدر لم تصدر أي مكون Maven، وتبقى حالات ضابطة للاستدعاء. وبالمقارنة مع 280,21 سطرا لتبعيات محلولة من المصدر، أصدر النظام 729 سطرا مطابقا من البداية إلى النهاية. كما أكد مسار مستقل مقيد بإصدارات المصدر 277,6/277,6 سطرا صادرا، ولا يقدم هذا الرقم إلا بوصفه حدا أعلى.

يثبت النموذج الأولي قابلية الاسترجاع العملي لقائمة مكونات البرمجيات انطلاقا من الملف الثنائي، مع إظهار حدود تغطية قاعدة المراجع، واستدعاء المطابقة، والتمييز بين الإصدارات المتجاورة وعائلات الملفات، والتعامل مع الشيفرة الأصلية والمخفية، وقابلية التوسع، وحدود سلطة الذكاء الاصطناعي. ويتمثل مبدؤه الأساسي في تفضيل معرفة جزئية قابلة للتدقيق مع امتناع صريح على ادعاء يقين غير مدعوم بشأن الإصدار الدقيق.

الكلمات المفتاحية --- أمن أندرويد، تحليل ملفات APK، قائمة مكونات البرمجيات، كشف المكتبات الخارجية، إسناد المكونات، CycloneDX، إدارة الثغرات، R8.

Contents

Dedication i

Acknowledgements ii

Abstract iii

Résumé iv

ملخص v

Contents vi

List of Figures xiii

List of Tables xv

List of Acronyms xvii

I General Introduction 1

1 General Introduction 2

1.1 Context . . . 2

1.2 Problem Statement . . . 3

1.2.1 The Source-to-Binary Information Gap . . . 3

1.2.2 Loss and Transformation of Component Evidence . . . 3

1.2.3 The Vulnerability-Authority Problem . . . 4

1.3 Motivation . . . 5

1.3.1 Security Auditing without Source Access . . . 5

1.3.2 Transparency rather than Forced Completeness . . . 5

1.3.3 An End-to-End Engineering Gap . . . 6

1.4 Research Questions . . . 6

1.5 Objectives . . . 7

1.5.1 General Objective . . . 7

1.5.2 Specific Objectives . . . 7
1.6 Methodology Overview . . . 7
1.7 Contributions . . . 8
1.8 Scope and Limitations . . . 9
1.8.1 Included Scope . . . 9
1.8.2 Excluded or Bounded Scope . . . 10
1.9 Thesis Organization . . . 10
1.10 Conclusion . . . 11

II Background 12

2 Technical Background 13
2.1 Introduction . . . 13
2.2 Software Supply-Chain Security . . . 13
2.2.1 Software Supply Chains . . . 13
2.2.2 Supply-Chain Attacks . . . 14
2.2.3 Transparency, Traceability, and Provenance . . . 14
2.3 Software Bills of Materials . . . 15
2.3.1 Definition and Purpose . . . 15
2.3.2 SPDX and CycloneDX . . . 15
2.3.3 Package URL . . . 16
2.3.4 Use Cases and Limitations . . . 17
2.4 Vulnerability Identification and Management . . . 17
2.4.1 Identifiers, Weaknesses, Scores, and Catalogs . . . 17
2.4.2 Why Exact Identity and Version Matter . . . 18
2.5 Android Application Ecosystem . . . 18
2.5.1 Application Architecture and Development . . . 18
2.5.2 JAR, AAR, and Maven Dependencies . . . 19
2.6 APK Structure . . . 19
2.6.1 The APK as a Container . . . 19
2.6.2 Manifest, Resources, and Assets . . . 20
2.6.3 DEX and Multidex . . . 20
2.6.4 Signing and Native Libraries . . . 21
2.6.5 Nonstandard and Dynamically Loaded Code . . . 21
2.7 Android Compilation and Optimization . . . 21
2.7.1 From Source Dependencies to APK . . . 21
2.7.2 R8 and ProGuard-Compatible Rules . . . 22
2.8 Reverse Engineering and Static Analysis . . . 23
2.8.1 Static and Dynamic Analysis . . . 23
2.8.2 Recoverable DEX Evidence . . . 23

2.8.3 Reverse-Engineering Challenges . . . 23
2.9 Dependency and Component Identification . . . 24
2.9.1 Library Identity versus Version Identity . . . 24
2.9.2 Reference Profiles . . . 24
2.9.3 Candidate Generation, Similarity, and Confidence . . . 24
2.10 Evidence-Based SBOM Recovery . . . 25
2.10.1 Evidence Concepts . . . 25
2.10.2 Evidence Sources and Failure Modes . . . 26
2.10.3 Fusion, Conflict, and Abstention . . . 27
2.10.4 Relationship to the Proposed System . . . 27
2.11 Conclusion . . . 27

3 Artificial Intelligence for Component Identification 29
3.1 Introduction . . . 29
3.2 Problem Formulation . . . 29
3.2.1 From Version Selection to Risk Triage . . . 29
3.2.2 Triage Labels . . . 30
3.2.3 Allowed and Forbidden Actions . . . 31
3.3 Dataset and Feature Representations . . . 32
3.3.1 Structured Project-Evidence Dataset . . . 32
3.3.2 Feature Families . . . 32
3.3.3 Structured Token Profiles . . . 33
3.3.4 Raw Opcode Sequences . . . 34
3.4 Train, Validation, Test, and Held-Out Strategy . . . 34
3.4.1 Case-Grouped Structured Split . . . 34
3.4.2 Raw-Opcode Case-Held-Out Split . . . 35
3.5 Classical Machine Learning: Random Forest . . . 35
3.5.1 Algorithm and Selection Rationale . . . 35
3.5.2 Baselines . . . 36
3.5.3 Feature Importance and Ablation . . . 36
3.6 Transformer-Based Experiments . . . 37
3.6.1 Transformer Encoder . . . 37
3.6.2 Full Structured-Token Transformer . . . 37
3.6.3 MVP-Safe Structured-Token Transformer . . . 38
3.6.4 Raw-Opcode Transformer . . . 38
3.7 Evaluation Metrics . . . 39
3.7.1 Confusion Terms . . . 39
3.7.2 Macro, Micro, Weighted F1, and Accuracy . . . 39
3.7.3 Baselines and Promotion Criteria . . . 40
3.8 Results and Interpretation . . . 40

3.8.1 Comparative Summary . . . 40
3.8.2 Random Forest Class-Level Behavior . . . 41
3.8.3 Shortcut Risk . . . 42
3.8.4 Negative Evidence as a Result . . . 42
3.9 Production Authority and Safety Boundary . . . 43
3.9.1 Integration Level . . . 43
3.9.2 Report-Only Guardrail Proof . . . 43
3.9.3 Authority Matrix . . . 44
3.10 Threats to Validity and Lessons Learned . . . 45
3.10.1 Dataset Validity . . . 45
3.10.2 Label Validity . . . 45
3.10.3 Metric Validity . . . 46
3.10.4 Reproducibility . . . 46
3.10.5 Engineering Lessons . . . 46
3.11 Conclusion . . . 46

III State of the Art 48

4 State of the Art 49
4.1 Introduction and Review Scope . . . 49
4.2 Evolution of Android Third-Party Library Detection . . . 49
4.2.1 Package, Frequency, and Reference Profiles . . . 50
4.2.2 Class Dependencies and Version-Aware Detection . . . 50
4.3 Optimization-Aware and Structural Matching . . . 51
4.4 Learned Representations . . . 51
4.5 SBOM-Oriented Approaches . . . 52
4.6 Benchmark and Assessment Evidence . . . 52
4.7 Comparative Analysis . . . 53
4.8 Limitations of Existing Approaches . . . 55
4.8.1 Dataset and Metric Fragmentation . . . 55
4.8.2 Package and Reference Dependence . . . 55
4.8.3 Incomplete R8 and Optimization Coverage . . . 55
4.8.4 Weak Standard Output and Provenance . . . 55
4.8.5 Limited Uncertainty and Failure Reporting . . . 55
4.9 Impact on the Proposed MVP . . . 56
4.10 Research Gap and Positioning . . . 57
4.11 Conclusion . . . 57

IV Contribution 58

5 Proposed Solution and System Design 59

5.1 Introduction . . . 59
5.2 Functional and Non-Functional Requirements . . . 59
5.3 General Architecture . . . 60
5.4 APK Extraction and DEX Analysis . . . 60
5.5 Reference Database Design . . . 60
5.6 Evidence Model . . . 61
5.7 Candidate Generation and Version Selection . . . 61
5.8 CycloneDX and Vulnerability Design . . . 61
5.9 Experimental Isolation and Fail-Fast Controls . . . 61
5.10 Use Cases and Behavioral Diagrams . . . 62
5.10.1 Actors and Use Cases . . . 62
5.10.2 APK-to-Report Activity . . . 62
5.10.3 APK-to-Report Interaction Sequence . . . 63
5.10.4 Reference-Database Construction Sequence . . . 64
5.10.5 Component and Deployment View . . . 64
5.10.6 Reference-Database Data Model . . . 65
5.11 Conclusion . . . 66

6 Implementation, Evaluation, and Discussion 67

6.1 Introduction . . . 67
6.2 Development Environment . . . 67
6.3 Implementation by Module . . . 68
6.3.1 Module Map . . . 68
6.3.2 Extraction and DEX Analysis . . . 70
6.3.3 Reference Construction . . . 70
6.3.4 Evidence Scoring and Version Decisions . . . 70
6.3.5 SBOM, Vulnerability, and Reporting Layers . . . 71
6.4 Command-Line Workflow . . . 72
6.4.1 Minimal APK-to-Report Run . . . 72
6.4.2 Diagnostic Commands . . . 72
6.4.3 Reference-Database Workflow . . . 73
6.5 Evaluation Methodology . . . 73
6.5.1 Research Questions . . . 73
6.5.2 Evidence Layers . . . 73
6.5.3 Metrics and Denominators . . . 74
6.5.4 Regression and Promotion Protocol . . . 75
6.6 Controlled Evaluation . . . 75
6.7 Source-Validated Real-APK Evaluation . . . 76

6.7.1 Main Emitted-Version Results . . . 76
6.7.2 Full Source Dependency Denominator . . . 76
6.8 Source-Known Upper-Bound Evaluation . . . 77
6.9 Vulnerability Demonstration . . . 78
6.10 AI and R8 Evaluation . . . 84
6.11 Failed and Non-Promoted Experiments . . . 85
6.12 Comparison with Existing Work . . . 87
6.13 Discussion . . . 88
6.13.1 Answers to the Research Questions . . . 88
6.13.2 Strengths . . . 88
6.13.3 Error Categories . . . 88
6.13.4 Practical Value of Partial Results . . . 89
6.13.5 Runtime and Scalability . . . 89
6.14 Threats to Validity . . . 89
6.14.1 Internal Validity . . . 89
6.14.2 Construct Validity . . . 89
6.14.3 External Validity . . . 90
6.14.4 Reference-Database Bias . . . 90
6.14.5 AI Validity . . . 90
6.15 Conclusion . . . 90

V General Conclusion 91

7 General Conclusion and Perspectives 92
7.1 General Conclusion . . . 92
7.1.1 Evaluation Synthesis . . . 93
7.1.2 Practical Significance . . . 94
7.2 Answers to the Research Questions . . . 94
7.3 Limitations . . . 95
7.3.1 Reference Coverage . . . 95
7.3.2 Matcher Recall and Candidate Generation . . . 96
7.3.3 Adjacent-Version and Artifact-Family Ambiguity . . . 96
7.3.4 Native, Hidden, and Dynamic Code . . . 96
7.3.5 Runtime, Scalability, and Operational Coverage . . . 96
7.3.6 Source Validation . . . 97
7.3.7 Vulnerability Interpretation . . . 97
7.3.8 Artificial-Intelligence Authority . . . 97
7.4 Future Perspectives . . . 97
7.4.1 Safer Candidate Retrieval . . . 98
7.4.2 Calibrated Evidence and Version Abstention . . . 99

7.4.3 Reference and Provenance Knowledge Graph 99
7.4.4 Native and Dynamic Components 99
7.4.5 Larger and More Independent Evaluation 99
7.4.6 AI as a Guarded Research Direction 100
7.5 Closing Remarks 100

A Reproducibility and Evidence Index **102**
A.1 Build and Runtime Environment 102
A.1.1 Recommended Docker Environment 102
A.1.2 Host-Python Alternative 103
A.2 Core APK-to-SBOM Command Sequence 104
A.2.1 Stage-Level Inspection 104
A.2.2 Expected Output Package 104
A.3 Reference-Database Construction 105
A.3.1 Isolated Build Sequence 105
A.3.2 Reset and Promotion Safety 105
A.4 Source Validation and Vulnerability Enrichment 105
A.5 Dataset and Evidence Inventory 107
A.6 Claim-to-Artifact Traceability 108
A.7 Reproduction Constraints 109

Bibliography **110**

List of Figures

1.1 Android software-supply-chain context: dependencies enter before the build, while the auditor may observe only the compiled APK. 3
1.2 Source-based SBOM generation and binary-only SBOM recovery answer related but non-identical composition questions. 5
1.3 Fail-fast, target/control methodology used to promote or reject matcher, reference-database, R8, and AI experiments. 8

2.1 Principal APK regions and the evidence they can expose. 20
2.2 Simplified Android build path from source and resolved dependencies to a signed APK. 21
2.3 How R8 transformations weaken different component-identification signals. 22

3.1 AI evidence pipeline and separation from authoritative matcher and SBOM outputs. 32
3.2 Case grouping prevents evidence rows from the same APK case from crossing structured train, validation, and test partitions. 35
3.3 Learned evidence may enrich reports but is blocked from the authoritative matcher-to-SBOM path. 45

4.1 Evolution from fragile package signatures toward structural, optimization-aware, learned, and SBOM-oriented analysis. 50

5.1 Detailed architecture of the implemented APK-to-SBOM system, including the authoritative analysis and reference-building paths and the non-authoritative ML/DL/R8 assistance layer. 60
5.2 Actors and use cases, with experiment-only actions shown as dashed orange elements. 62
5.3 Activity flow from APK input to the final evidence package. 63
5.4 Sequence of interactions during an APK-to-report execution. 64
5.5 Sequence used to construct a versioned Maven reference database. 64
5.6 Component and deployment view with authoritative and report-only paths separated. 65
5.7 Simplified physical schema of the SQLite reference-feature database; each arrow represents a `component_id` foreign-key relation. 66

6.1 Clean F-Droid report summary screenshot 79

6.2 Clean F-Droid component evidence screenshot 80
6.3 Controlled vulnerable report summary screenshot 81
6.4 Controlled Jackson CVE screenshot 82
6.5 Controlled vulnerable output artifacts screenshot 83

List of Tables

1.1 Principal PFE contributions and their concrete realization. 9

2.1 SBOM-standard dimensions relevant to compiled APK analysis. 16
2.2 Vulnerability concepts used by the APK-to-SBOM workflow. 17
2.3 Example distinction between component and exact-version evidence. . . . 25
2.4 Static evidence sources for APK-derived SBOM recovery. 26

3.1 Structured AI triage labels and their permitted interpretation. 31
3.2 Feature families used by the structured Random Forest experiment. 33
3.3 Case-grouped split for the structured ML/DL dataset. 34
3.4 Random Forest feature ablation on the grouped test partition. 37
3.5 Case-held-out raw-opcode experiment and baselines. 39
3.6 AI model results, input authority, and project decisions. 41
3.7 Random Forest per-class results on 3,868 grouped test rows. 41
3.8 AI authority matrix in the implemented MVP. 44

4.1 Capability-oriented comparison of representative Android TPL and SBOM approaches. 54
4.2 State-of-the-art ideas and their disposition in the proposed MVP. 56

5.1 Evidence families used by the component-attribution pipeline. 61

6.1 Development and execution environment of the implemented MVP. 68
6.2 Principal implementation modules and their observable outputs. 69
6.3 Default matcher weights in the authoritative path. 71
6.4 Core CLI commands and their expected result. 72
6.5 Evaluation layers and permitted claims. 74
6.6 Controlled evaluation by transformation category. 75
6.7 Source-validated emitted Maven version results. 76
6.8 Coverage-first view of the full source dependency denominator. 77
6.9 Source-version constrained v2 upper-bound results. 77
6.10 Representative vulnerability demonstrations and their authority. 78
6.11 Machine-learning and Transformer results with authority decisions. 84

6.12 Representative failed, blocked, or non-promoted experiments. 86
6.13 Dimension-based positioning relative to the reviewed detector families. . . 87

7.1 Answers to the research questions and their evidence boundaries. 95
7.2 Prioritized future work and the evidence required for promotion. 98

A.1 Reproduction environment used by the project. 103
A.2 Principal generated report artifacts. 104
A.3 Evidence classes and their permitted interpretation. 107
A.4 Principal thesis claims and reproducibility artifacts. 108

List of Acronyms

Acronym	Definition
AAB	Android App Bundle
AAR	Android Archive
ABI	Application Binary Interface
AI	Artificial Intelligence
APK	Android Package Kit
API	Application Programming Interface
ART	Android Runtime
CVE	Common Vulnerabilities and Exposures
CVSS	Common Vulnerability Scoring System
CWE	Common Weakness Enumeration
DEX	Dalvik Executable
DL	Deep Learning
EPSS	Exploit Prediction Scoring System
F1	Harmonic mean of precision and recall
JAR	Java Archive
JSON	JavaScript Object Notation
JVM	Java Virtual Machine
KEV	Known Exploited Vulnerabilities
MCC	Matthews Correlation Coefficient
ML	Machine Learning
MVP	Minimum Viable Product
NVD	National Vulnerability Database
OSV	Open Source Vulnerabilities
PURL	Package URL
R8	Android code shrinker and optimizer
SBOM	Software Bill of Materials
SPDX	Software Package Data Exchange
TPL	Third-Party Library

Part I

General Introduction

Chapter 1

General Introduction

1.1 Context

Modern software is rarely developed as an isolated body of source code. Applications combine first-party logic with operating-system services, open-source libraries, commercial components, build plug-ins, package registries, and automated delivery infrastructure. This network of producers, tools, services, and artifacts forms a software supply chain. A weakness or compromise at any point in that chain can affect the delivered application, which is why supply-chain risk management must cover the complete software life cycle [1].

Third-party components are especially important in mobile development. They provide networking, cryptography, database access, user-interface widgets, media handling, dependency injection, analytics, and many other capabilities. Reuse accelerates development, but it also transfers maintenance, licensing, and security risk into every application that embeds the component. When a vulnerability is disclosed, an organization must determine which applications contain the affected package and version before it can prioritize remediation.

A Software Bill of Materials (SBOM) supports this need by representing software components, identifiers, versions, hashes, and relationships in a structured form. The NTIA describes an SBOM as a nested inventory for software and positions it as a foundation for software transparency [2]. Standards such as SPDX and CycloneDX make this inventory machine-readable and interoperable with governance and vulnerability tools [3, 4].

The Android ecosystem creates a difficult setting for this transparency. Applications are commonly written in Kotlin or Java, assembled with Gradle, and distributed as compiled Android packages. External JAR and AAR artifacts are resolved, transformed, merged with application code, converted to DEX, combined with resources and native libraries, and signed. The resulting APK is the artifact installed or audited; it does not contain the original dependency declaration in a form equivalent to the resolved build graph.

Figure 1.1 summarizes the motivating situation. The same dependency that enables application functionality can also become a source of inherited vulnerability or provenance risk. The security analyst may receive only the delivered binary, after the original supply-chain metadata has been transformed or removed.

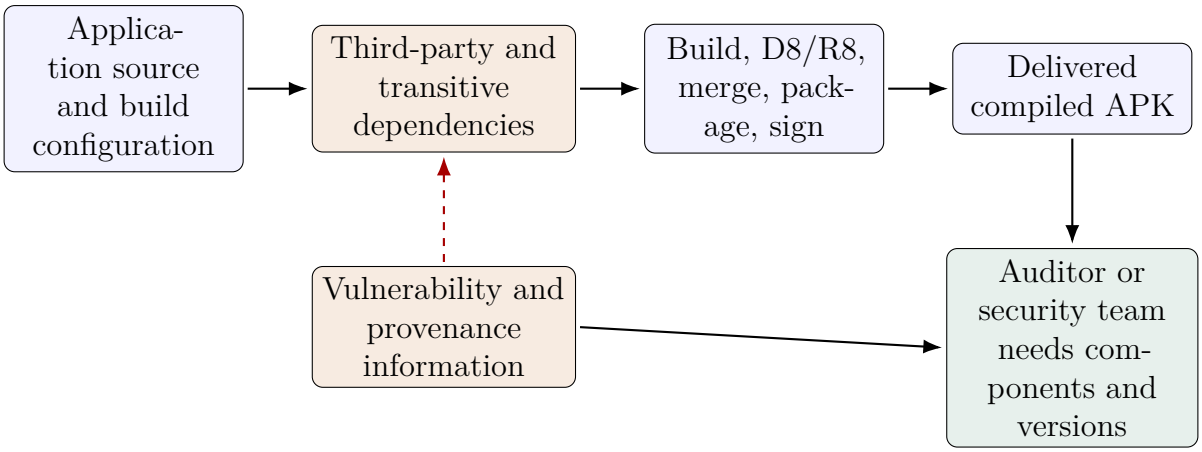


Figure 1.1: Android software-supply-chain context: dependencies enter before the build, while the auditor may observe only the compiled APK.

1.2 Problem Statement

1.2.1 The Source-to-Binary Information Gap

Conventional SBOM generation is most reliable when it runs inside a trusted build environment. Package-manager metadata, lock files, dependency-resolution results, source modules, and compiler information can be collected before they disappear into the final binary. This information makes it possible to name direct and transitive dependencies and to preserve their declared coordinates.

That assumption does not hold in every audit scenario. A security analyst, application store, incident-response team, customer, or third-party assessor may possess only an APK. The source repository may be unavailable, incomplete, private, outdated, or inconsistent with the distributed binary. In this case, the SBOM must be recovered from evidence that survived inside the compiled artifact.

The source and binary views are not equivalent. A source dependency graph may contain build-time, metadata-only, optional, or unused dependencies that contribute no executable code. Conversely, the APK may include shaded, vendored, generated, repackaged, or native code that does not correspond to a simple direct dependency declaration. The objective is therefore not to recreate an imaginary perfect Gradle file. It is to identify which claims can be supported by the delivered binary and to state where the reconstruction is incomplete.

1.2.2 Loss and Transformation of Component Evidence

During Android compilation, classes from the application and its dependencies are converted to DEX and commonly merged into shared files. Original JAR and AAR boundaries disappear. Large applications may use several DEX files, while variant-aware dependency resolution can select Android-, JVM-, Kotlin-, or other target-specific artifacts whose physical names differ from the public dependency coordinate.

R8 further combines shrinking, optimization, obfuscation, and bytecode rewriting [5]. It may:

- rename packages, classes, methods, and fields;
- remove unused component code;
- inline methods into application or library callers;
- merge classes or change package organization;
- simplify instructions and constants; and
- eliminate the few methods or literals that distinguish adjacent library versions.

These transformations affect evidence families differently. Names may vanish while opcode or dependency patterns remain. Structural evidence may survive renaming but change under inlining or class merging. A component identity may be recognizable even when its exact version is not. Android third-party-library research consequently distinguishes library detection from the more difficult version-identification problem [6, 7, 8].

Recent common-benchmark evidence also shows why results from selected libraries or convenient datasets cannot be treated as universal deployment accuracy. Detector behavior changes with partial inclusion, optimization, reference coverage, version similarity, and dataset construction [9, 10].

1.2.3 The Vulnerability-Authority Problem

Vulnerability databases generally associate an advisory with a package identity and an affected version range. The inference chain is:

$$\text{APK evidence} \rightarrow \text{component coordinate} \rightarrow \text{exact version} \rightarrow \text{affected range}.$$

A wrong coordinate can retrieve unrelated advisories. A wrong version can create a false alarm or hide an affected release. Even a correct package/version mapping does not prove runtime exploitability, because configuration, reachability, platform behavior, and mitigations may matter.

The central problem addressed by this thesis can therefore be stated as follows:

> How can a system recover an auditable and standards-based software-component inventory from a compiled Android APK, estimate component identities and exact versions from incomplete or transformed evidence, and support vulnerability analysis without converting ambiguity into false precision?

Figure 1.2 contrasts the two generation conditions. The source-based path can use declared and resolved build metadata. The APK-only path must reconstruct bounded claims from static observations and known references.

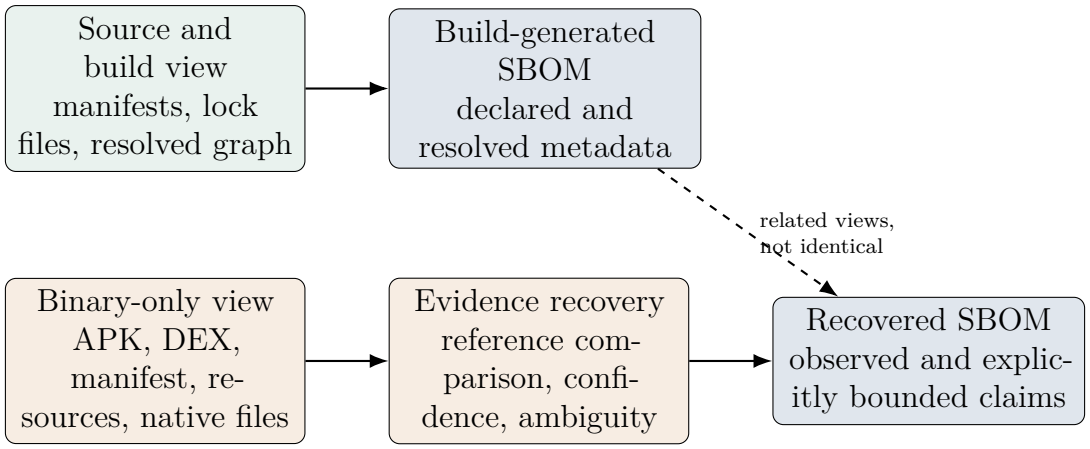


Figure 1.2: Source-based SBOM generation and binary-only SBOM recovery answer related but non-identical composition questions.

1.3 Motivation

1.3.1 Security Auditing without Source Access

Organizations often need to inspect software they did not build. Acquired applications, legacy products, third-party mobile clients, incident artifacts, and app-store packages may lack a reproducible source environment. Static APK analysis offers a way to recover useful evidence without executing the application or trusting vendor-provided metadata.

A practical APK-to-SBOM tool can help an analyst:

- inventory recognizable third-party JVM/Maven components;
- identify strong exact-version evidence and adjacent-version ambiguity;
- obtain a standard document consumable by other tools;
- prioritize vulnerability investigation;
- compare binary observations with source dependency evidence when source is available; and
- recognize native-heavy, hidden-code, or missing-reference boundaries that require another analysis method.

1.3.2 Transparency rather than Forced Completeness

The usefulness of an SBOM depends on the reliability of its identifiers. Emitting a large inventory with unsupported exact versions may be more dangerous than emitting a smaller inventory with clear evidence and limitations. The project is therefore motivated by *evidence quality*, not by maximizing the number of rows at any cost.

This principle has three consequences. First, library identity and exact version are treated as separate decisions. Second, ambiguous alternatives and reference-coverage gaps are preserved instead of hidden. Third, vulnerability findings inherit the confidence and validation status of the underlying component.

1.3.3 An End-to-End Engineering Gap

Existing Android library-identification research contributes important techniques based on package signatures, class dependencies, opcodes, call structure, optimization simulation, and learned representations. However, detector output alone does not complete the operational workflow. A usable system also needs:

- a reproducible versioned reference database;
- an explicit identity/version decision policy;
- standard SBOM serialization;
- vulnerability lookup with uncertainty propagation;
- source-backed validation where possible;
- report formats for both machines and analysts; and
- regression gates that preserve negative experiments and prevent unsafe global promotions.

The project addresses this integration gap with a working MVP whose claims are limited by its observed evidence.

1.4 Research Questions

The thesis evaluates the following four research questions. They are phrased to match the evidence layers and answers reported in Chapter 6.

RQ1: Can the pipeline recover exact Maven versions when ground truth and adequate reference evidence are available?

RQ2: How does emitted-version quality change between clean and difficult real APK families?

RQ3: How much of the full source dependency denominator is represented by the reference database and emitted by the matcher?

RQ4: Can SBOM, vulnerability, and AI assistance preserve uncertainty and authority boundaries?

These questions separate capability from coverage. RQ1 tests the mechanics under favorable evidence. RQ2 examines realistic variation across application families. RQ3 distinguishes reference coverage from matcher recall. RQ4 evaluates whether downstream outputs remain honest when the analysis is uncertain.

1.5 Objectives

1.5.1 General Objective

The general objective is to design, implement, and evaluate a static evidence-based system that recovers a bounded CycloneDX SBOM from a compiled Android APK and supports confidence-aware vulnerability analysis.

1.5.2 Specific Objectives

The specific objectives are:

1. inspect an APK and recover manifest, DEX, resource, native-file, and forensic metadata without installing or executing the application;
2. extract comparable packages, classes, methods, instructions, literals, API references, and dependency structures from all successfully parsed DEX files;
3. acquire selected Maven JAR/AAR releases, convert their bytecode to DEX, and construct a versioned SQLite reference-feature database;
4. combine several static evidence families to rank component and version candidates;
5. distinguish component identity from exact-version confidence and expose ambiguity or abstention when evidence is insufficient;
6. emit accepted components in CycloneDX JSON using Package URL identifiers and evidence properties;
7. query ecosystem-aware vulnerability information while preserving component and version uncertainty;
8. validate emitted Maven rows against source dependency trees where auditable source is available; and
9. evaluate controlled, clean, partial, source-known, report-only, failed, and limitation evidence without merging them into one misleading score.

1.6 Methodology Overview

The project followed an iterative research-and-engineering methodology. It did not begin by selecting a single detector and treating its score as a complete SBOM. The work progressed through the following stages:

1. **literature and standard review**: analyze Android third-party-library detection, obfuscation resilience, realistic benchmarking, SBOM standards, and vulnerability identifiers;
2. **scope and architecture**: define a static JVM/Maven MVP, choose CycloneDX and OSV, and establish native, dynamic-code, and heavy-obfuscation boundaries;

3. **reference construction**: download known Maven releases, convert JAR/AAR bytecode to DEX, extract comparable features, and store coordinate/version profiles;
4. **APK analysis and matching**: inspect the package, aggregate multidex evidence, generate candidates, rank references, and separate identity from version confidence;
5. **reporting and validation**: generate SBOM, JSON, Markdown, and HTML outputs, query vulnerabilities, and compare emitted rows with source dependency trees;
6. **controlled and real evaluation**: test clear, multidex, R8, clean real, partial real, native-heavy, and hidden-code cases in separate evidence lanes; and
7. **guarded improvement**: evaluate reference growth, advanced feature weights, rerankers, and AI assistance with target/control gates; promote only changes that improve the target without regressing controls.

Figure 1.3 shows the evidence-gated loop used during implementation and optimization.

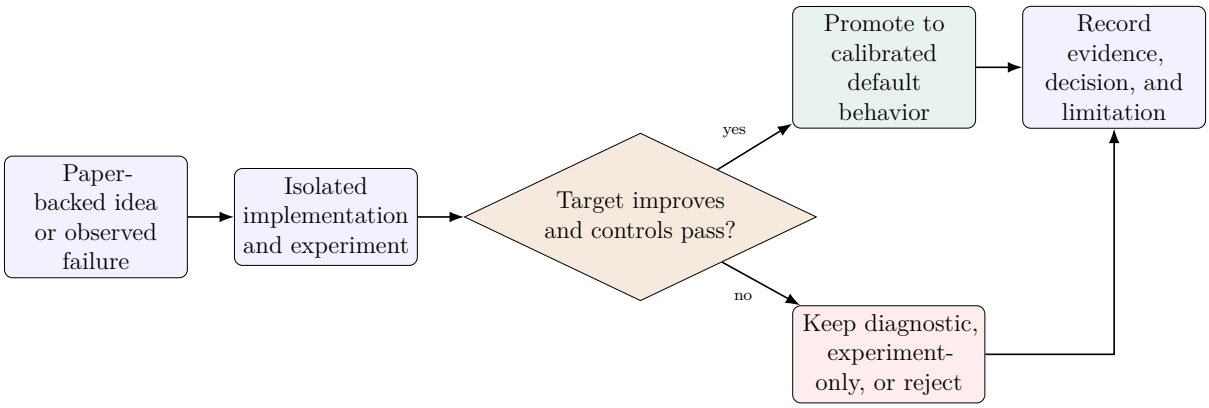


Figure 1.3: Fail-fast, target/control methodology used to promote or reject matcher, reference-database, R8, and AI experiments.

This methodology treats failed experiments as evidence. An improvement that fixes one target but causes false versions or components in controls is not a successful system improvement.

1.7 Contributions

The thesis contributes an implemented system and an evidence-governance methodology rather than a claim of universal Android component recovery. Table 1.1 maps each contribution to its concrete realization.

Table 1.1: Principal PFE contributions and their concrete realization.

Contribution	Realization
End-to-end APK-to-SBOM MVP	A command-line pipeline connects APK inspection, DEX analysis, Maven reference matching, CycloneDX generation, vulnerability lookup, and multi-format reporting.
Common binary evidence representation	Target APKs and reference JAR/AAR releases are analyzed through DEX-oriented feature extraction, reducing the mismatch between source and compiled representations.
Hybrid explainable matching	Package, class, method, opcode, literal, dependency, metadata, and guarded contextual signals are combined while preserving per-feature evidence.
Identity/version separation	Component presence, exact-version status, confidence, score margin, and ambiguous alternatives are represented as distinct fields rather than one forced label.
Guarded reference-database workflow	Maven releases are acquired, converted, stored, and promoted through isolated databases and target/control regression checks; unsafe broad catalogs remain evaluation-only.
Standards-based security output	Accepted Maven components use Package URLs in CycloneDX JSON; native files remain hash-backed file observations; OSV results inherit component certainty.
Source-backed evaluation methodology	Emitted rows are compared with resolved dependency trees, and controlled, clean, partial, source-known, report-only, and limitation evidence are reported separately.
Transparent experiment and AI governance	Failed scoring changes, unsafe rerankers, R8 diagnostics, and ML/DL models are retained as evidence; AI is integrated only as opt-in report assistance.

These contributions are connected. The reference database is useful only if its candidates are interpreted conservatively. CycloneDX output is useful only if component identity and version evidence are trustworthy. Vulnerability lookup is useful only if it preserves that trust boundary. Evaluation is therefore part of the contribution, not a final step added after implementation.

1.8 Scope and Limitations

1.8.1 Included Scope

The MVP focuses on:

- static analysis of standard Android APK files;
- all discoverable standard and nonstandard DEX entries;
- Java and Kotlin ecosystem components whose bytecode can be represented through Maven JAR/AAR references;

- selected known coordinate/version profiles in a local SQLite database;
- evidence-based library identity and exact-version estimation;
- CycloneDX JSON output with Package URLs and evidence properties;
- OSV-oriented vulnerability lookup;
- native shared-library extraction and hashing as file evidence; and
- optional source validation for open-source evaluation cases.

1.8.2 Excluded or Bounded Scope

The following areas remain outside the current MVP or are represented only as limitations:

- complete reconstruction of the original Gradle dependency graph;
- universal coverage of all Maven coordinates and historical versions;
- general package/version identification for native shared libraries;
- code downloaded only at runtime;
- encrypted, packed, or strongly protected payloads that cannot be recovered through the implemented static process;
- perfect recovery under heavy R8 shrinking, inlining, merging, and repackaging;
- runtime reachability or exploitability analysis for vulnerability findings; and
- AI authority over selected components, versions, SBOM rows, or vulnerability labels.

Source dependency trees are used as external evaluation evidence when available. They are not supplied to the normal APK-only production matcher. The source-known v2 lane is consequently treated as a constrained upper-bound experiment rather than unconstrained detector accuracy.

1.9 Thesis Organization

The remainder of the thesis is organized as follows:

- **Chapter 2 – Technical Background** introduces software supply-chain security, SBOM standards, vulnerability terminology, Android packaging and compilation, DEX evidence, reverse engineering, component identification, and evidence-based recovery.
- **Chapter 3 – Artificial Intelligence for Component Identification** presents only the ML/DL methods implemented in the project: the structured Random Forest, full and MVP-safe Transformers, the raw-opcode negative result, evaluation metrics, shortcut risks, and the report-only authority boundary.

- **Chapter 4 – State of the Art** reviews Android third-party-library and version-identification approaches by technique, compares their capabilities and limitations, and derives the research gap addressed by the MVP.
- **Chapter 5 – Proposed Solution and System Design** defines requirements, architecture, actors, use cases, activities, sequences, evidence families, version decisions, components, deployment, and the reference-database model.
- **Chapter 6 – Implementation, Evaluation, and Discussion** describes the implemented modules and commands, defines the evidence layers and metrics, reports controlled and real-APK results, preserves failed experiments, answers the research questions, and discusses validity threats.
- **Chapter 7 – General Conclusion and Perspectives** synthesizes the contributions and limitations and presents the most defensible directions for future work.

1.10 Conclusion

Recovering an SBOM from a compiled Android APK is not a simple dependency listing task. Compilation removes archive boundaries, R8 transforms identity and structure, reference catalogs are incomplete, adjacent versions may be observationally similar, and vulnerability results depend on the correctness of every earlier attribution step.

This thesis addresses the problem through an evidence-gated static pipeline that favors auditable claims over forced completeness. It combines binary feature extraction, versioned Maven references, conservative matching, standard CycloneDX output, source-backed evaluation, and explicit authority boundaries for experiments and AI. The next chapter establishes the technical concepts required to understand this design.

Part II

Background

Chapter 2

Technical Background

2.1 Introduction

Recovering a Software Bill of Materials from a compiled Android application connects several technical domains. The task begins with software supply-chain security, because an application inherits code and risk from external components. It then requires a representation for those components, a method for relating them to vulnerability information, and an understanding of how the Android build process transforms source dependencies into an APK.

This chapter develops that chain in the same order. It first introduces software supply chains, SBOM concepts, package identifiers, and vulnerability terminology. It then explains the Android ecosystem, the internal structure of an APK, the D8 and R8 compilation stages, and the evidence available to static reverse engineering. The final sections distinguish library identity from exact-version identification and define the evidence, provenance, confidence, and uncertainty concepts used throughout this thesis.

The purpose is not to reproduce the implementation described in Chapters 5 and 6. It is to establish why a compiled application cannot be treated as a dependency manifest and why trustworthy recovery must combine several imperfect evidence sources.

2.2 Software Supply-Chain Security

2.2.1 Software Supply Chains

A software supply chain comprises the people, processes, tools, services, and artifacts involved in producing and delivering software. It includes source repositories, build systems, package registries, external libraries, continuous-integration services, signing infrastructure, distribution channels, and the final deployed artifact. NIST treats cybersecurity supply-chain risk management as a life-cycle activity because a weakness or compromise may enter through any of these elements, including products and services acquired from external suppliers [1].

Third-party dependencies are therefore part of the application's security boundary. They reduce development effort and provide mature functionality, but they also introduce inherited code, licenses, maintenance schedules, and vulnerabilities. A single dependency

may itself depend on other components, forming a transitive graph rather than a flat list.

For an open-source project, a build file and resolved dependency graph can describe this supply-chain view before compilation. A released binary offers a different view: it contains the code and resources that survived resolution, merging, shrinking, and packaging. These two views overlap, but they are not identical. A declared dependency may contribute no code to the final artifact, while copied, shaded, vendored, generated, or dynamically obtained code may not appear as a simple direct dependency row.

2.2.2 Supply-Chain Attacks

Supply-chain attacks target trust relationships instead of attacking only the final application. Relevant attack paths include:

- compromising a source repository or developer account;
- publishing a malicious or dependency-confusion package;
- modifying an artifact in a registry or distribution channel;
- compromising a build service, compiler, plug-in, or signing key;
- exploiting a known vulnerability in a reused component; and
- inserting or downloading code outside the expected build graph.

These cases have different causes, but they share an attribution problem: security teams need to know which component is present, which version or variant is involved, where it originated, and which evidence supports the claim. A component name without a version is often insufficient for vulnerability assessment, while a version without provenance may be impossible to audit.

2.2.3 Transparency, Traceability, and Provenance

Transparency makes the composition and production context of software visible. *Traceability* connects a delivered artifact to upstream components and transformations. *Provenance* records where an artifact or claim came from and how it was produced. These properties support incident response, patch prioritization, supplier assessment, and reproducible analysis.

In this thesis, provenance is also applied to analysis evidence. A recovered component claim should preserve at least:

- the analyzed APK and its cryptographic hash;
- the observed file, DEX, class, method, literal, or structural signal;
- the reference artifact and version used for comparison;
- the matching or validation procedure;
- the confidence and competing alternatives; and
- the evidence class, such as controlled, source-validated, partial, or experiment-only.

This is narrower than full build provenance, because a binary-only analyzer cannot recreate every upstream build event. It is nevertheless essential for making the resulting SBOM auditable.

2.3 Software Bills of Materials

2.3.1 Definition and Purpose

An SBOM is a structured inventory of software components and their relationships. The NTIA describes it as a nested inventory for software, analogous to a list of ingredients, and emphasizes its role in software transparency and vulnerability management [2]. An SBOM can be created during a trusted build, supplied by a vendor, generated from source metadata, or reconstructed from a compiled artifact.

A useful component record commonly contains:

- a component type, supplier, name, namespace, and version;
- one or more persistent package identifiers;
- cryptographic hashes;
- licenses and copyright information;
- dependency or containment relationships;
- external references and vulnerability links; and
- evidence or properties that explain how the record was obtained.

Not every field is available in every production context. A source-generated SBOM may know the declared dependency graph and licenses but not which methods survived optimization. A binary-recovered SBOM observes the shipped artifact but may be uncertain about coordinate, version, scope, or transitive relationships. The generation method is therefore part of the meaning of the document.

2.3.2 SPDX and CycloneDX

SPDX and CycloneDX are widely used machine-readable standards for software composition data. SPDX originated around licensing and package information and now supports several software supply-chain profiles. CycloneDX is an OWASP standard designed for bill-of-materials and risk-analysis use cases, including components, services, dependencies, vulnerabilities, compositions, and evidence [3, 4].

Table 2.1 compares the dimensions relevant to this project. The comparison does not imply that one standard is universally superior; the choice depends on the information available and the downstream workflow.

Table 2.1: SBOM-standard dimensions relevant to compiled APK analysis.

Dimension	SPDX	CycloneDX
Primary emphasis	Software-package, file, license, security, build, and provenance information through defined profiles	Component and service inventory, dependency relationships, security analysis, vulnerabilities, and operational risk
Representations	JSON, JSON-LD, RDF, YAML, XML, and tag/value depending on the specification version	JSON, XML, and Protocol Buffers depending on the specification version
Identity	Package metadata, external references, checksums, and package identifiers	Component metadata, Package URL, CPE, hashes, properties, and external references
Relationships	Rich typed relationships among elements	Dependency graph, composition, formulation, and service relationships
Evidence use	Supports provenance and declared or concluded information	Supports component evidence, identity evidence, occurrences, methods, and confidence-oriented properties
Project choice	Suitable alternative or exchange format	Selected output format because it maps directly to component, PURL, evidence, dependency, and vulnerability workflows

The MVP emits CycloneDX JSON. This is an engineering choice, not a claim that SPDX is unsuitable. A later exporter could map sufficiently supported component data to either format.

2.3.3 Package URL

Package URL is a standardized package identifier whose general form is:

```
pkg:type/namespace/name@version?qualifiers#subpath
```

It identifies software according to a package ecosystem rather than by a free-text name [11]. For a Maven component, a PURL can represent the group, artifact, and version, for example:

```
pkg:maven/androidx.fragment/fragment@1.6.1
```

This normalization is important because names such as “fragment” or “core” are not globally unique. The ecosystem and namespace reduce ambiguity and give vulnerability services a stable query key. However, a syntactically valid PURL does not prove that the identified component exists in an APK. Its trustworthiness remains bounded by the evidence used to infer the coordinate and version.

2.3.4 Use Cases and Limitations

SBOMs support component inventory, vulnerability triage, license review, supplier communication, incident response, asset correlation, and policy checking. During a newly disclosed vulnerability, an organization can search its inventories for affected packages instead of manually examining every application.

An SBOM is not, by itself, proof that software is secure. It may be incomplete, outdated, generated with weak identifiers, or disconnected from the deployed artifact. It does not establish exploitability, configuration, reachability, or maliciousness. For binary reconstruction, the limitations are stronger: shrinking can remove dependency evidence; optimization can mix boundaries; native code may lack ecosystem metadata; and runtime-loaded components may be absent from the package. A responsible recovered SBOM must expose these limitations instead of presenting inferred rows as original build facts.

2.4 Vulnerability Identification and Management

2.4.1 Identifiers, Weaknesses, Scores, and Catalogs

Vulnerability management uses several related but non-interchangeable concepts. Table 2.2 summarizes the role of each one.

Table 2.2: Vulnerability concepts used by the APK-to-SBOM workflow.

Term	Role	Interpretation
CVE	Public vulnerability identifier	Names a publicly disclosed vulnerability record so tools and organizations can refer to the same issue [12].
CWE	Weakness taxonomy	Classifies recurring design and implementation weaknesses that may lead to vulnerabilities; it is not a component-version advisory list [13].
NVD	Enriched vulnerability database	Adds analysis and structured data to CVE records, including affected-product and severity information [14].
CVSS	Technical severity framework	Describes intrinsic and environmental severity characteristics; it does not predict whether exploitation will occur [15].
EPSS	Exploitation-probability model	Estimates the probability that a published CVE will be exploited in the wild during the next 30 days [16].
KEV	Known-exploitation catalog	Lists vulnerabilities for which CISA has evidence of exploitation and provides remediation-oriented operational priority [17].
OSV	Ecosystem-aware advisory schema and service	Represents affected package ecosystems and version ranges and supports package/version queries [18, 19].

These sources answer different questions. CVSS concerns severity, EPSS concerns forecast exploitation probability, and KEV records known exploitation. None of them repairs an incorrect package identity. The first requirement is still to determine whether an affected component and version are present.

2.4.2 Why Exact Identity and Version Matter

Most vulnerability advisories describe an affected package and one or more version ranges. Mapping the advisory to an application therefore depends on the chain:

binary evidence → package coordinate → version → affected range → vulnerability context.

An error early in the chain propagates. A wrong coordinate can produce unrelated advisories; a wrong version can create a false positive or hide a real exposure. Adjacent library versions are especially difficult because they may differ by only a few methods, and optimization can remove precisely those differences.

This thesis consequently distinguishes three forms of vulnerability output:

- **confirmed mapping**: the component and exact version have sufficiently strong evidence, and the version falls within an advisory's affected range;
- **potential mapping**: the component is plausible, but version, variant, reachability, or affected-range evidence is incomplete; and
- **report-only enrichment**: external risk information is shown for analyst review but is not allowed to change the authoritative SBOM or source-validation metrics.

Even a confirmed package/version mapping is not a proof of runtime exploitability. Configuration, platform behavior, call reachability, and mitigations still require separate assessment.

2.5 Android Application Ecosystem

2.5.1 Application Architecture and Development

Android applications are commonly written in Kotlin or Java, with optional C or C++ native code, and are executed within the Android application framework and runtime. Application components such as activities, services, broadcast receivers, and content providers are declared through the application manifest and interact with platform services through framework APIs [20, 21].

The standard build system uses Gradle with the Android Gradle Plugin. Application code, generated code, resources, local modules, and external dependencies are processed into an installable artifact [22]. Distribution may use an Android App Bundle, from which an app store generates device-specific APKs, while direct analysis usually receives one universal or split APK.

2.5.2 JAR, AAR, and Maven Dependencies

Java libraries are commonly distributed as JAR files containing compiled JVM classes and metadata. Android libraries are commonly distributed as AAR archives, which may contain a JAR together with Android resources, a manifest, native libraries, consumer rules, and other Android-specific content [23]. Maven coordinates conventionally identify an artifact as `group:artifact:version`.

Dependency resolution is variant-aware. Gradle selects compatible variants according to attributes such as platform, usage, build type, flavor, and target environment [24]. This matters for reverse identification because a public dependency coordinate can resolve to different physical artifacts:

- a base Java or Android artifact;
- a Kotlin extension artifact, often identified by a `-ktx` suffix;
- a JVM-, Android-, or other platform-specific target;
- a metadata or bill-of-materials artifact with no runtime bytecode; or
- a shaded or repackaged artifact whose internal classes no longer preserve the public coordinate name.

The analyzer observes the selected and transformed binary content, not the declarative choice that produced it. Public-coordinate, target-variant, and packaging differences must therefore be considered when building reference profiles.

2.6 APK Structure

2.6.1 The APK as a Container

An APK is a ZIP-format package containing executable code, resources, configuration, signatures, and optional native binaries. Android Studio's APK Analyzer exposes these archive entries and reports DEX, resource, and file composition information [25]. Figure 2.1 presents the main evidence-bearing regions.

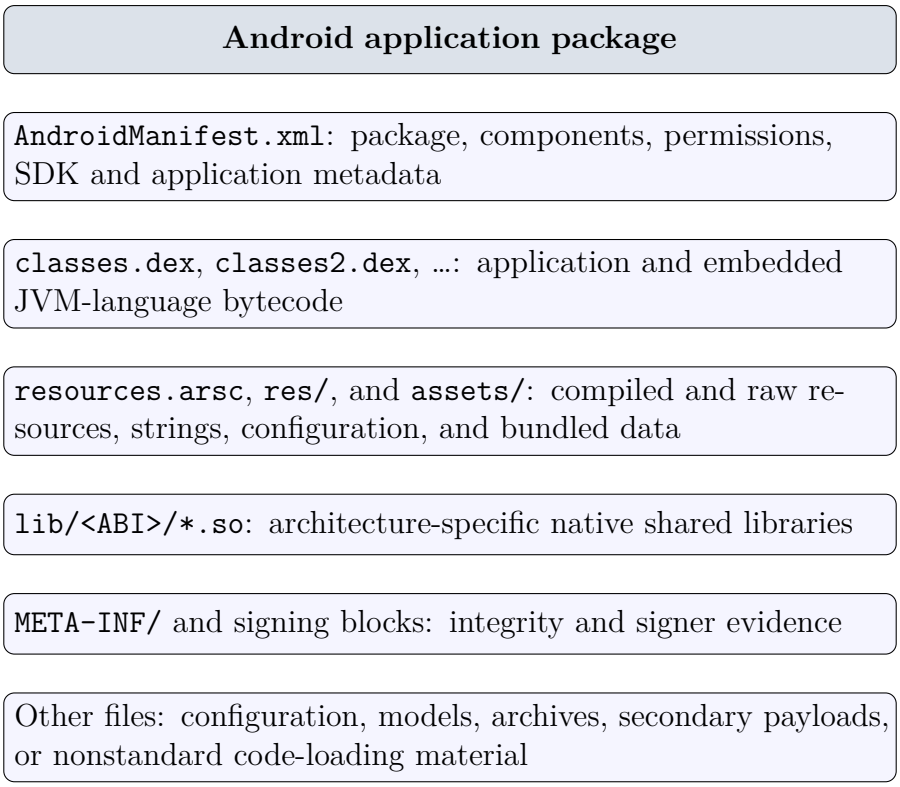


Figure 2.1: Principal APK regions and the evidence they can expose.

2.6.2 Manifest, Resources, and Assets

The manifest declares the application package, components, permissions, features, SDK constraints, intent filters, and other installation or runtime metadata [21]. In a packaged APK it is stored in a compiled binary XML representation. Manifest metadata can support application identification and contextual analysis, but it does not normally enumerate all embedded Maven dependencies.

Compiled resources are indexed through `resources.arsc` and the `res/` hierarchy. The `assets/` directory may contain arbitrary application data. Resource names, strings, notices, and configuration files can occasionally reveal component evidence; however, these signals may be renamed, removed, shared among products, or unrelated to executable code.

2.6.3 DEX and Multidex

DEX is the executable bytecode format used by the Android runtime. A DEX file contains type, string, field, method, class, annotation, and instruction data. Application and dependency classes are typically merged into one or more DEX files, so the original JAR or AAR boundaries are no longer present.

The DEX format imposes reference-index limits. Applications exceeding the single-DEX method-reference limit use multidex packaging, commonly producing `classes.dex`, `classes2.dex`, and additional numbered files [26]. A component detector must analyze all relevant DEX entries. Examining only the primary DEX can silently miss dependencies.

2.6.4 Signing and Native Libraries

Android requires APKs to be digitally signed. The signing identity supports update trust and package integrity; modern signing schemes can protect the whole artifact rather than only individual ZIP entries [27]. A signer does not identify every embedded dependency, but signing information is useful provenance for the application artifact.

Native libraries are stored under ABI-specific directories such as `lib/arm64-v8a/` or `lib/x86_64/`. An ABI defines the binary interface expected by a CPU architecture and platform [28]. Native file names and hashes are observable, but general package and version attribution from stripped shared objects is a separate binary-analysis problem. This thesis therefore distinguishes native file evidence from Maven component identification.

2.6.5 Nonstandard and Dynamically Loaded Code

Not all executable behavior must appear in conventional `classes*.dex` entries. Applications may unpack encrypted payloads, load additional DEX or native code, use custom class loaders, or download code after installation. Android security guidance warns that dynamic code loading increases risk, especially when remote sources or weak integrity controls are used [29].

A static APK analyzer can report suspicious archives, nonstandard DEX paths, native-heavy packages, and other observable indicators. It cannot truthfully inventory code that is absent, encrypted beyond available analysis, or fetched only at runtime. These are scope limitations, not ordinary negative detection results.

2.7 Android Compilation and Optimization

2.7.1 From Source Dependencies to APK

Figure 2.2 summarizes the build path that turns source and external artifacts into packaged evidence.

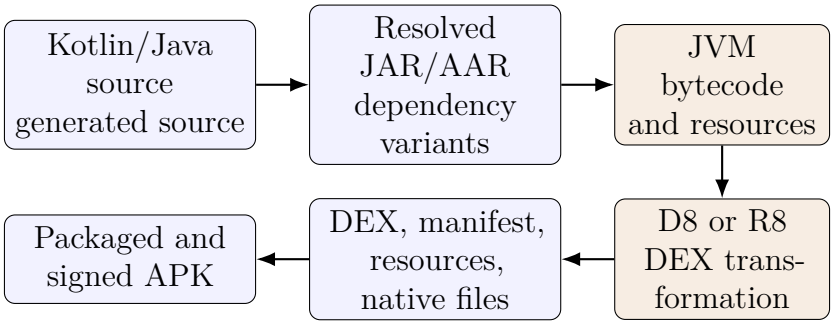


Figure 2.2: Simplified Android build path from source and resolved dependencies to a signed APK.

Kotlin and Java sources are compiled to JVM bytecode. Resources and manifests are processed separately, while external artifacts are resolved according to the build's dependency and variant rules. D8 transforms Java bytecode into DEX bytecode and can

receive compiled classes, JARs, APKs, and ZIP archives as inputs [30]. Packaging then combines code, resources, native libraries, and metadata before signing.

This process destroys a convenient assumption for component recovery: the output is not a collection of untouched dependency archives. Classes from the application and many libraries coexist in merged DEX files, while AAR resources are merged into the application resource table.

2.7.2 R8 and ProGuard-Compatible Rules

R8 combines shrinking, optimization, obfuscation, and bytecode transformation in the Android release build process [5]. It consumes keep rules, including rules written in the ProGuard configuration language, but R8 is more than a simple identifier renamer.

Its transformations include:

- **renaming**: replacing class, method, and field names with short identifiers;
- **shrinking**: removing unreachable classes, methods, fields, and resources;
- **optimization**: rewriting instructions and simplifying control or data flow;
- **inlining**: moving a method body into its callers and removing the original boundary;
- **class merging or repackaging**: changing structural and package organization; and
- **constant and resource propagation**: replacing or eliminating values that would otherwise provide literals or references.

Figure 2.3 relates these transformations to the evidence used by a component detector.

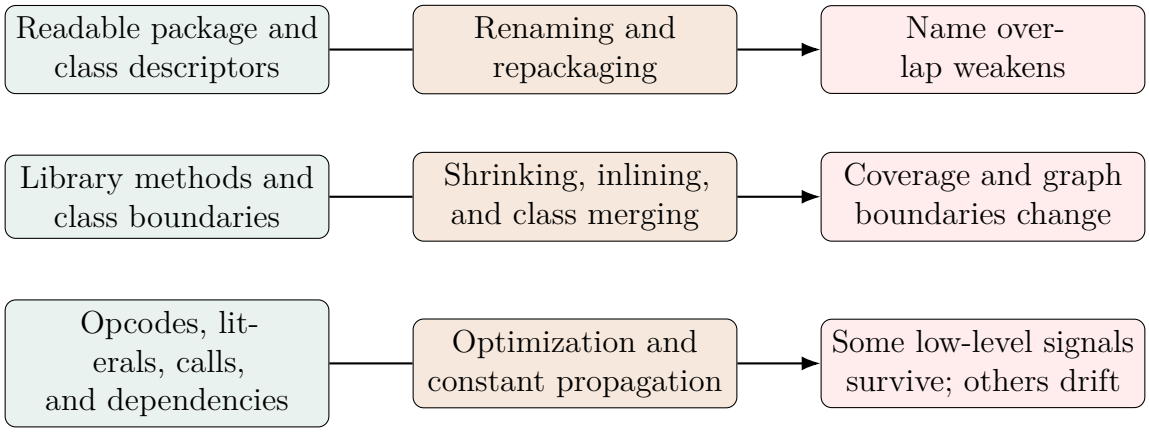


Figure 2.3: How R8 transformations weaken different component-identification signals.

Renaming mainly attacks textual identity, while shrinking and inlining alter the amount and organization of surviving code. Two neighboring versions may become observationally identical if R8 removes their version-distinguishing methods. Conversely, a heavily transformed component may retain distinctive opcode, literal, call, or dependency patterns even after names disappear. This motivates multi-feature matching and explicit abstention.

2.8 Reverse Engineering and Static Analysis

2.8.1 Static and Dynamic Analysis

Static analysis examines an application without executing it. For Android, this may include archive inspection, manifest decoding, resource analysis, DEX disassembly, control-flow construction, call analysis, and native binary inspection. OWASP describes static analysis as examining an application's code and resources to identify properties without runtime execution [31].

Dynamic analysis observes behavior while the application runs, often through instrumentation, debugging, system-call observation, network interception, or runtime hooking. It can reveal decrypted or downloaded code and actual execution paths, but its coverage depends on devices, inputs, permissions, and the explored behavior. Reverse engineering commonly combines both approaches when the objective requires it [32].

The MVP in this thesis is intentionally static. Static analysis is reproducible, suitable for offline batch processing, and directly connected to the shipped APK. The trade-off is that purely runtime or protected code may remain invisible.

2.8.2 Recoverable DEX Evidence

A DEX parser can recover several evidence families:

- package and class descriptors;
- fields and method signatures;
- instruction opcodes and opcode-frequency summaries;
- string literals and constants;
- method invocations and framework API references;
- inheritance, interface, field-type, and call dependencies; and
- class-, method-, or basic-block-level graph structure.

These signals have different robustness. Class names can be highly discriminative in unobfuscated applications but fragile under renaming. Opcode distributions are less dependent on names but may be too coarse for nearby versions. Literals can distinguish releases, yet optimization may remove them. Dependency graphs preserve structural context, but inlining and class merging can change their topology. No feature family should therefore be interpreted independently of its failure modes.

2.8.3 Reverse-Engineering Challenges

Practical analysis must handle multidex applications, malformed entries, unusual archive paths, generated code, duplicate classes, library repackaging, Kotlin compiler artifacts, reflection, resource-only dependencies, native code, and protection systems. It must also distinguish a parse failure from an empty result. If one DEX file cannot be analyzed,

silently treating it as containing no components would convert missing evidence into false negative evidence.

Legal and ethical boundaries also matter. Reverse engineering should be performed on authorized artifacts for research, interoperability, audit, or security purposes consistent with applicable law and policy. This project uses static analysis to identify components and evidence; it does not attempt to bypass access controls or modify third-party applications.

2.9 Dependency and Component Identification

2.9.1 Library Identity versus Version Identity

Third-party library identification asks whether code from a known component is present. Version identification asks which release or release range best explains the observed evidence. The second task is strictly harder. Many versions share most packages, classes, methods, and strings, while a small patch may change only a few instructions.

This distinction is central to SBOM and vulnerability accuracy. A detector may have strong evidence for `androidx.fragment:fragment` while lacking enough surviving evidence to separate versions `1.6.1` and `1.6.2`. Reporting the library identity can be justified while asserting an exact version cannot.

2.9.2 Reference Profiles

Reference-based detection begins with known artifacts. For each coordinate and version, the system extracts a profile containing comparable evidence, such as descriptors, methods, opcodes, literals, API references, and dependency structure. The same extraction process is applied to the target APK. Comparing DEX-like representations is preferable to comparing source code directly with optimized DEX because it reduces representation mismatch.

Reference coverage creates an important boundary. If the correct component or version is absent from the reference database, the matcher cannot select it. A high similarity to the nearest available version is not proof that the available version is correct. Reference absence must be represented as a coverage limitation rather than hidden inside the matcher score.

2.9.3 Candidate Generation, Similarity, and Confidence

A scalable matcher commonly uses two stages. Candidate generation removes obviously unrelated references through inexpensive evidence such as package or class overlap. Detailed scoring then combines several feature similarities for the remaining candidates. This coarse-to-fine pattern reduces cost, but an overly strict shortlist can discard the correct component after obfuscation.

A weighted score can be expressed abstractly as:

$$S(c, v \mid A) = \sum_{i=1}^{n} w_i s_i(A, R_{c,v}),$$

where A is the analyzed application, $R_{c,v}$ is the reference profile for coordinate c and version v, s_i is a feature-specific similarity, and w_i is its configured weight. The score ranks evidence; it is not automatically a calibrated probability that the component is present.

Confidence also depends on separation from alternatives, feature coverage, evidence quality, reference completeness, and known transformations. Table 2.3 illustrates the required decision boundary.

Table 2.3: Example distinction between component and exact-version evidence.

Observed situation	Identity claim	Version claim	Permitted output
Distinctive classes, methods, literals, and structure match one reference	Strong	Strong and separated	Emit coordinate and exact version with supporting evidence
Library family is clear, but two adjacent versions have nearly equal evidence	Strong	Ambiguous	Emit identity only or preserve both version candidates; do not invent a tiebreak
Names are removed, while partial structural and opcode evidence remains	Candidate	Weak	Report as uncertain or abstain from authoritative SBOM emission
No suitable reference exists	Unknown	Unsupported	Record a reference coverage gap, not component absence
Only a native file hash is visible	File observation	Unsupported Maven version	Emit a native file component if useful, without claiming a Maven coordinate

2.10 Evidence-Based SBOM Recovery

2.10.1 Evidence Concepts

Evidence-based recovery treats every emitted component as a claim that must be connected to observable support. Five concepts organize that support:

- **evidence source**: the artifact or observation from which a signal was obtained;
- **provenance**: the chain linking the signal to the analyzed APK, reference artifact, tool, configuration, and procedure;
- **strength**: how specifically and reliably the signal supports an identity or version;

- **conflict**: disagreement among sources or similarly supported candidates; and
- **uncertainty**: the bounded lack of knowledge that remains after analysis.

Evidence strength is contextual. A readable package name may be strong in a clear application and weak after repackaging. A shared literal may be weak alone but useful when combined with version-unique methods and dependency structure. Evidence must therefore be interpreted as a set rather than a collection of universally reliable indicators.

2.10.2 Evidence Sources and Failure Modes

Table 2.4 summarizes the main evidence sources available to a static APK analyzer.

Table 2.4: Static evidence sources for APK-derived SBOM recovery.

Source	Useful evidence	Principal limitation
APK container	File paths, sizes, hashes, DEX inventory, native files, signing and suspicious payload indicators	Archive presence does not provide package identity for every file
Manifest	Application identity, components, permissions, SDK and feature context	Normally does not enumerate embedded Maven dependencies
DEX descriptors	Packages, classes, fields, methods, types	Renaming, repackaging, merging, generated code, and multidex omissions
DEX instructions	Opcodes, calls, constants, strings, API references	Optimization, inlining, dead-code removal, and common compiler patterns
Dependency structure	Class, method, inheritance, field-type, and call relationships	Graph topology changes under shrinking, merging, and inlining
Resources and assets	Resource names, notices, metadata, bundled files and strings	Shared or unrelated data; removal and renaming; no guaranteed version semantics
Native libraries	ABI, filename, size, hash, imports, and binary structure	Stripping, static linking, absent ecosystem identifiers, and separate native version-analysis requirements
Reference artifacts	Known coordinate/version profiles and version-unique evidence	Catalog incompleteness, packaging errors, variant mismatch, and unavailable historical releases
Source validation	Resolved Gradle dependency graph for an open-source app	Represents the source/build view and may differ from code surviving in the distributed APK

2.10.3 Fusion, Conflict, and Abstention

Evidence fusion combines signals that fail differently. Names can establish clear identities, structural evidence can survive some renaming, literals can separate particular versions, and source validation can verify emitted rows for open-source applications. Agreement among independent signals supports a stronger claim than repetition of one signal family.

Conflict should not be hidden by a forced maximum. If one candidate wins by class overlap while another wins by version-unique literals, the report should preserve the disagreement and identify which signal may have been affected by optimization or catalog error. Similarly, a narrow score margin between adjacent versions is evidence of ambiguity, not a reason to select the newer release.

Abstention is therefore a valid system outcome. The analyzer may:

- emit a coordinate and exact version;
- emit a coordinate while withholding exact version authority;
- expose ranked candidates for analyst review;
- emit only a hash-based native file observation;
- record a reference or parse coverage gap; or
- classify the case as outside the supported static-analysis boundary.

This hierarchy prevents uncertainty from being converted into false precision. It also protects vulnerability analysis: a potential candidate may support a warning or investigation path, but it must not silently become a confirmed CVE finding.

2.10.4 Relationship to the Proposed System

The system developed in this thesis operationalizes these concepts through a common DEX evidence representation, a versioned Maven reference database, multi-feature ranking, explicit confidence and ambiguity fields, CycloneDX output, OSV enrichment, and source-backed validation when a dependency graph is available. The authoritative path remains deterministic and evidence-gated. Experimental R8 and artificial-intelligence signals are kept outside SBOM and vulnerability authority unless a separate target/control evaluation justifies promotion.

This separation also guides evaluation. Controlled ground truth, clean source-validated applications, partial cases, source-known upper-bound experiments, report-only assistance, and unsupported boundaries answer different questions. Combining them into one accuracy number would erase the provenance needed to interpret the result.

2.11 Conclusion

Software supply-chain security requires knowledge of the components delivered inside an application, yet the compiled APK does not retain a complete dependency manifest. JAR and AAR boundaries are merged, DEX files combine application and library code, resources

are processed, and R8 may rename, remove, inline, or reorganize the very evidence needed for attribution.

SPDX, CycloneDX, and Package URL provide standard structures for describing components, but a standard representation cannot compensate for weak identity or version evidence. Vulnerability databases likewise depend on correct package/version mapping and must remain subordinate to the certainty of the recovered component claim.

Consequently, no single APK signal is sufficient for trustworthy recovery. Names, methods, opcodes, literals, dependency structure, native hashes, reference profiles, and source validation each contribute useful but bounded evidence. A defensible system must fuse these signals, retain provenance, expose conflicts, and abstain when exact identification is not supported. The next chapter examines whether machine-learning and deep-learning methods can assist this process while remaining outside the authoritative production boundary.

Chapter 3

Artificial Intelligence for Component Identification

3.1 Introduction

Chapter 2 established that component recovery from an APK is an evidence-fusion problem. Clear applications may preserve package and class names, while R8-affected applications may retain only partial opcode, literal, call, and structural evidence. Adjacent versions can also remain difficult to separate even when the library identity is clear.

These conditions motivate a question: can machine learning help recognize patterns across scores, evidence survival, ambiguity, and obfuscation risk? The project investigated this question through one classical machine-learning model and three Transformer-based experiments. The goal was not to replace the deterministic matcher. It was to evaluate whether learned models could assist report wording, triage difficult rows, or identify useful future research directions.

This authority distinction is essential. A model can classify a row according to labels derived from previous project evidence without proving that it can identify an exact Maven version from a previously unseen APK. A high score may also result from contextual shortcuts that would not exist during source-free deployment. The chapter therefore presents the complete evidence, including model limitations, class imbalance, ablation results, shortcut risk, and a raw-opcode experiment that failed to beat a simpler baseline.

All numerical results in this chapter are frozen from the generated artifacts dated 20 June 2026. They belong to the *report-only experimental evidence* class. The production matcher, CycloneDX output, vulnerability labels, and source-validation metrics remain deterministic.

3.2 Problem Formulation

3.2.1 From Version Selection to Risk Triage

The primary APK-to-SBOM task is component attribution: determine whether a known coordinate is present and, when justified, select an exact version. The AI experiments deliberately solve a narrower problem. Given a row of existing matcher and diagnostic

evidence, they predict a triage label that describes how that row should be treated in a report or investigation.

Let a structured evidence row be represented by:

$$x = (s_1, \ldots, s_p, n_1, \ldots, n_q, b_1, \ldots, b_r),$$

where s_i are matcher and confidence scores, n_i are overlap counts, and b_i are binary context indicators. The supervised task is:

$$f_\theta(x) \to y,$$

where y belongs to a six-label triage space. The output is not a Maven coordinate or version.

The raw-opcode experiment uses a separate formulation. Its input is a bounded method-level opcode sequence:

$$o = (o_1, o_2, \ldots, o_T),$$

and its label represents the R8 or real-APK profile of the source case. This experiment tests whether raw instruction sequences carry enough transferable signal to distinguish controlled obfuscation profiles under a case-held-out split. It does not perform component or version identification.

3.2.2 Triage Labels

The structured dataset contains the six labels in Table 3.1. The labels are generated from existing matcher recall, transparency, source-validation, boundary, Phase 5, and R8 diagnostic artifacts. They encode project decisions and evidence states, not universal manually annotated Android ground truth.

Table 3.1: Structured AI triage labels and their permitted interpretation.

Label	Rows	Meaning in this project
`risk_only`	7,811	Preserve the row as risk or investigation evidence without making a positive component/version suggestion.
`confirmed_exact`	6,790	Existing project evidence labels the row as exact. It is a training class, not an authority that the AI model may emit into the production SBOM.
`boundary_do_not_suggest`	3,029	The row belongs to a known limitation, alias, missing-reference, hidden-code, or unsupported boundary and must not become a positive suggestion.
`suggest_report_only`	1,429	The evidence may justify a visible source-context or diagnostic suggestion, but not matcher promotion.
`suppress_uncertain`	870	The current wording or candidate should be withheld or made more cautious.
`mismatch_or_extra`	594	The row represents a version mismatch, unexpected emission, or related discrepancy requiring investigation.

The class distribution is imbalanced. The largest class contains 7,811 rows, while the smallest contains 594. This imbalance motivates balanced class weights during training and macro-averaged evaluation metrics.

3.2.3 Allowed and Forbidden Actions

The trained assistant may support three report actions:

- suggest a report-only investigation path;
- suppress or weaken unsupported wording; and
- mark a row as risk-only for analyst attention.

It may not:

- change the selected coordinate or version;
- add, remove, or modify CycloneDX components;
- turn a potential vulnerability into a confirmed finding;
- modify source-validation classifications or metrics; or
- bypass target/control promotion gates.

Figure 3.1 shows the resulting experimental pipeline. The dashed boundary separates learned assistance from authoritative outputs.

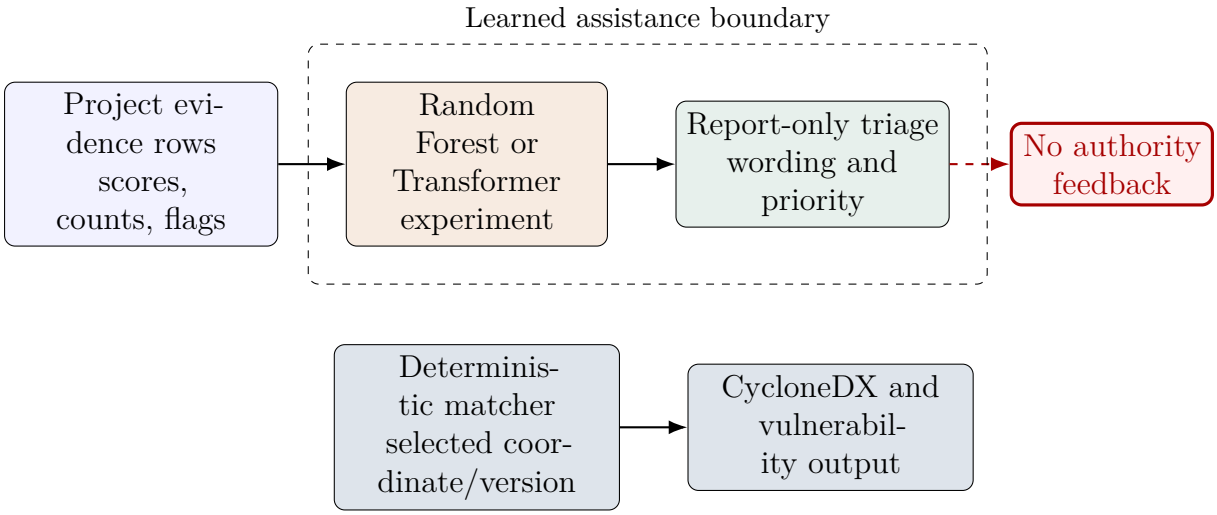


Figure 3.1: AI evidence pipeline and separation from authoritative matcher and SBOM outputs.

3.3 Dataset and Feature Representations

3.3.1 Structured Project-Evidence Dataset

The structured dataset contains 20,523 rows from 149 case identifiers and 1,370 coordinates. It combines four generated evidence sources:

- 11,184 boundary rows;
- 6,772 emitted-result rows;
- 2,281 matcher-recall rows; and
- 286 scoped Phase 5 diagnostic rows.

This composition gives the model examples of success, mismatch, recall, boundary, and R8-oriented diagnostic states. It also imposes a limitation: labels are functions of the project's existing evidence and policies. The dataset is appropriate for learning project triage behavior, but it is not an external benchmark proving generalization to all Android applications.

A larger manifest contains 31,589 rows. Of these, 20,523 are training-eligible and 11,066 belong to project-held-out or report-only lanes. The held-out lanes support drift checks, integration smoke tests, and transparency. The manifest explicitly records that it is not an external corpus.

3.3.2 Feature Families

The Random Forest receives 17 numeric or Boolean features. Table 3.2 groups them by purpose.

Table 3.2: Feature families used by the structured Random Forest experiment.

Family	Features	Interpretation
Overall ranking	`best_score`, `score_gap_to_best`, `source_rank`, `identity_confidence`	Strength and separation of the current candidate within existing matcher or source-context evidence.
Feature-specific scores	`package_score`, `class_score`, `method_score`, `opcode_score`, `literal_score`, `dependency_score`	Relative support from individual static-evidence families.
Overlap counts	`matched_packages`, `matched_classes`, `matched_methods`	Amount of direct overlap observed between the APK and a reference candidate.
Context indicators	`is_r8_or_minified`, `is_boundary_or_limitation`, `is_clean_real`, `is_partial_real`	Known case role and analysis context derived from the project evidence ledger.

These features are inexpensive because the deterministic pipeline already produces them. The model does not parse a new APK independently. It consumes post-analysis evidence and learns a report-triage function over that representation.

3.3.3 Structured Token Profiles

The Transformer experiments convert the same rows into sequences of discrete tokens. Each numeric feature is bucketed into categories such as `zero`, `very-low`, `low`, `medium`, `high`, or `one-or-more`. Coordinates and versions are split into textual parts, and the sequence is padded to 64 tokens.

Two profiles were evaluated:

1. **full structured profile**: includes the raw coordinate, source version, raw candidate version, all 17 features, and diagnostic context such as recall status, next action, gap class, reason label, and row role;
2. **MVP-safe profile**: removes source versions, raw candidate versions, the raw coordinate, source rank, source-validation role flags, boundary flags, and diagnostic-context labels.

The safer profile retains source-free score buckets, overlap counts, tokenized group/artifact words, candidate-version shape, and the R8/minification indicator. Its vocabulary falls from 5,097 to 1,704 tokens. This reduction is not merely a compression step; it is an experimental control for shortcut information.

3.3.4 Raw Opcode Sequences

The raw-opcode corpus is separate from the structured row representation. The extractor reads method instructions from 14 APKs and stores at most 120 method sequences per application. Each sequence is truncated or padded to 128 tokens. The resulting dataset contains:

- 1,680 method-level sequences;
- a vocabulary of 135 tokens;
- six controlled R8 profiles; and
- one real-APK-smoke profile.

The labels are `r8-inline-stress`, `r8-optimize`, `r8-rename`, `r8-repackage`, `r8-shrink`, `r8-shrink-optimize`, and `real-apk-smoke`. Each class contains 240 sequences from two APK cases. The task is therefore obfuscation-profile triage, not library or version classification.

3.4 Train, Validation, Test, and Held-Out Strategy

3.4.1 Case-Grouped Structured Split

Random row splitting would allow rows from the same APK case to appear in both training and test partitions. Because rows from one case share context, coordinates, evidence generation, and role labels, such a split would overestimate generalization. The project instead groups rows by `case_id`.

Case groups are sorted deterministically and assigned in a repeated ten-bucket pattern:

- buckets 0 and 1 to test;
- bucket 2 to validation; and
- buckets 3 through 9 to training.

This produces the split shown in Table 3.3.

Table 3.3: Case-grouped split for the structured ML/DL dataset.

Partition	Rows	Case IDs
Training	14,905	104
Validation	1,750	15
Test	3,868	30
Total	20,523	149

The Random Forest and both structured-token Transformers use the same partition assignments. Figure 3.2 emphasizes that grouping occurs before row assignment.

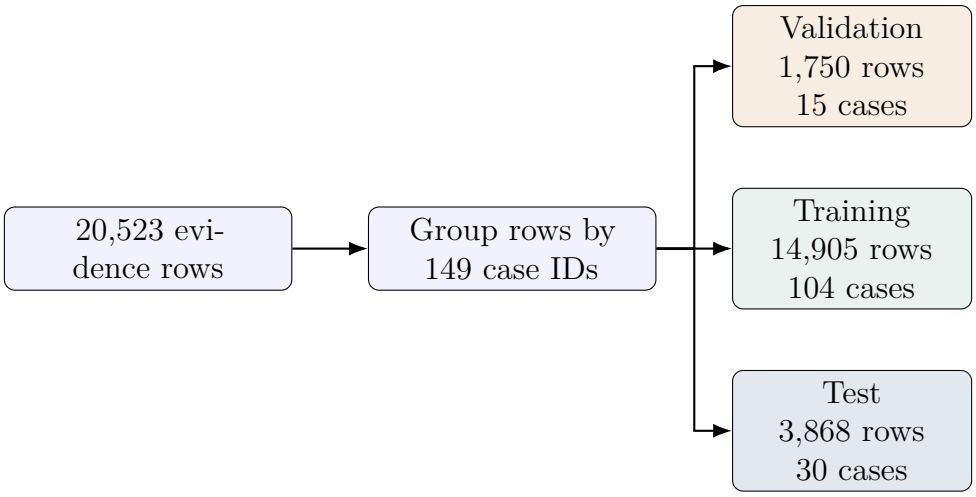


Figure 3.2: Case grouping prevents evidence rows from the same APK case from crossing structured train, validation, and test partitions.

This split reduces direct case leakage, but it does not create an external test set. Cases still originate from the same project, extraction pipeline, diagnostic conventions, and label-generation policy.

3.4.2 Raw-Opcode Case-Held-Out Split

The raw-opcode corpus uses a stricter per-label case holdout. For each of the seven labels, one APK case is reserved for test. The other APK case supplies training and validation rows. Validation is a deterministic sample from that training case because the corpus currently contains only two APK cases per label.

The final split contains:

- 665 training sequences from seven APK cases;
- 175 validation sequences from the same seven training cases; and
- 840 test sequences from seven unseen APK cases.

This design tests transfer between APK cases rather than memorization of methods from one application. Its weakness is sample diversity: one training APK and one test APK per label cannot capture the range of real applications, compilers, libraries, and optimization configurations.

3.5 Classical Machine Learning: Random Forest

3.5.1 Algorithm and Selection Rationale

A Random Forest is an ensemble of decision trees trained with randomized data and feature selection. Individual tree predictions are aggregated to obtain the final class [33]. Random forests are suitable for this project because they:

- handle nonlinear interactions among heterogeneous numeric and Boolean features;

- require limited preprocessing;
- support class weighting;
- produce feature-importance diagnostics; and
- train quickly enough for repeated local ablation experiments.

The implemented model uses 200 trees, a maximum depth of 8, balanced class weights, and random state 42. Constraining depth reduces the tendency of trees to memorize individual row patterns, although it does not eliminate project-context dependence.

3.5.2 Baselines

Two inexpensive baselines define whether training adds value:

- **majority baseline**: predicts `risk_only`, the most frequent training label, for every row;
- **score-threshold rule**: maps a boundary flag directly to the boundary label, maps a best score of at least 0.75 to report-only suggestion, maps a score of at least 0.50 to risk-only, and otherwise suppresses uncertainty.

The generated Phase 6A artifact records accuracy for these baselines: 0.383 for the majority rule and 0.165 for the threshold rule. The Random Forest reaches 0.690 test accuracy and 0.708 macro-F1. Because the baseline artifact does not generate baseline macro-F1, only accuracy is compared directly in this lane.

3.5.3 Feature Importance and Ablation

The highest fitted feature importances are:

- identity confidence: 0.231;
- boundary/limitation flag: 0.217;
- best score: 0.206;
- source rank: 0.086;
- dependency score: 0.068; and
- opcode score: 0.049.

These values describe how the fitted forest divided this dataset. They are not causal estimates, and correlated features may share or displace importance.

Table 3.4 reports the generated ablation study.

Table 3.4: Random Forest feature ablation on the grouped test partition.

Feature configuration	Features	Test macro-F1
No overlap counts	14	0.710
All features	17	0.708
No context flags	13	0.564
Scores only	10	0.558
Overlap counts only	3	0.176
Context flags only	4	0.162

Removing raw overlap counts slightly improves macro-F1 from 0.708 to 0.710. The difference is too small to justify a production feature removal, but it shows that counts add little to this bounded triage task. Removing context flags causes a larger decrease to 0.564, confirming that project-role context contributes substantially. This dependence is acceptable for report triage, but it is another reason not to interpret the model as an APK-only component detector.

3.6 Transformer-Based Experiments

3.6.1 Transformer Encoder

The Transformer architecture uses self-attention to relate tokens within a sequence without relying on recurrent processing [34]. The implemented prototype is intentionally small: an embedding dimension of 64, four attention heads, one encoder layer, a feed-forward width of 128, dropout of 0.1, and mean pooling over non-padding tokens. A linear classifier maps the pooled representation to the output labels.

The purpose is to exercise an end-to-end deep-learning workflow over project evidence, not to reproduce large-scale Smali pretraining. The learned representation can model interactions among tokenized scores, coordinate parts, version shapes, and context that a fixed linear model would not express directly.

3.6.2 Full Structured-Token Transformer

The full structured profile uses 20,523 rows and a 5,097-token vocabulary. It trains for three epochs with AdamW, class-weighted cross-entropy, batch size 128, and learning rate 0.001. It reaches:

- validation macro-F1 0.948;
- test macro-F1 0.977; and
- test accuracy 0.992.

These values are high, but the input representation contains information closely connected to label generation: raw coordinates, source versions, raw candidate versions, source rank, evidence-role flags, and diagnostic context such as next actions and gap labels.

Neural networks often exploit predictive shortcuts that perform well within a dataset while failing to learn the intended causal or semantic problem [35]. In this experiment, the intended deployment question is source-free risk triage, yet the full representation exposes source and workflow context.

The full-profile result is therefore classified as *research-status only*. It proves that the tokenization, training, evaluation, persistence, and reporting pipeline works. It does not demonstrate exact-version recovery or general APK-only intelligence.

3.6.3 MVP-Safe Structured-Token Transformer

The MVP-safe experiment removes the strongest source and workflow shortcuts. It uses the same 20,523 rows and grouped split but reduces the vocabulary to 1,704 tokens. It trains for five epochs and reaches:

- validation macro-F1 0.860;
- test macro-F1 0.882; and
- test accuracy 0.907.

The drop of approximately 0.095 macro-F1 relative to the full profile is informative. It shows that the removed context contributed meaningfully to the stronger result. The remaining performance indicates that source-free matcher scores, overlap features, coordinate-word parts, version shape, and R8-related evidence still contain useful signal for triage.

Nevertheless, the safer profile is not fully independent. Its labels remain project-derived, and its cases come from the same project evidence ecosystem. The integration gate consequently classifies it as a candidate for future opt-in report-only risk triage, with production decision wiring disabled.

3.6.4 Raw-Opcode Transformer

The raw-opcode model removes structured matcher and source features entirely. It receives only bounded method instruction sequences and predicts one of seven APK role or obfuscation-profile labels. The architecture uses the same 64-dimensional, four-head, one-layer Transformer pattern, but trains for eight epochs with batch size 64.

Two baselines accompany it:

- a majority-class predictor; and
- balanced logistic regression over opcode-frequency counts.

Table 3.5 presents the case-held-out test results.

Table 3.5: Case-held-out raw-opcode experiment and baselines.

Model	Test macro-F1	Test accuracy
Majority class	0.036	0.143
Opcode-count logistic regression	0.128	0.146
Raw-opcode Transformer	0.110	0.146

The Transformer is 0.018 macro-F1 below logistic regression. Its training loss decreases, but validation macro-F1 remains near 0.22 and test macro-F1 falls to 0.110 on the unseen APK cases. This is an important negative result: sequence modeling did not overcome the small number of APK cases or the lack of class, package, and application-level context.

The experiment is retained because it tests a more direct bytecode-learning path than the structured models. Its failure prevents a misleading conclusion that any Transformer necessarily improves obfuscation analysis. Future work would require substantially more APK cases, richer context, hierarchical aggregation from methods to classes and applications, and evaluation against component/version tasks rather than only obfuscation profiles.

3.7 Evaluation Metrics

3.7.1 Confusion Terms

For one class k, predictions can be summarized as:

- TP_k: rows of class k predicted as k;
- FP_k: rows from another class predicted as k;
- FN_k: rows of class k predicted as another class; and
- TN_k: all remaining correctly rejected rows.

Precision and recall are:

$$\text{Precision}_k = \frac{TP_k}{TP_k + FP_k}, \qquad \text{Recall}_k = \frac{TP_k}{TP_k + FN_k}.$$

The class F1 score is their harmonic mean:

$$F1_k = 2\frac{\text{Precision}_k \,\text{Recall}_k}{\text{Precision}_k + \text{Recall}_k}.$$

These measures describe different failure costs. High precision means that a predicted label is usually correct, while high recall means that most true rows of that label are found.

3.7.2 Macro, Micro, Weighted F1, and Accuracy

For K classes, macro-F1 is:

$$F1_{\text{macro}} = \frac{1}{K} \sum_{k=1}^{K} F1_k.$$

Each class contributes equally, irrespective of its support. This makes macro-F1 appropriate when minority triage labels matter [36].

Micro-F1 aggregates true positives, false positives, and false negatives before computing the score. In a single-label multiclass problem, it is closely related to overall accuracy and is dominated by frequent classes. Weighted F1 averages class F1 values using class support as weights. It reflects dataset prevalence but can hide weak minority performance.

Accuracy is:

$$\text{Accuracy} = \frac{\text{correct predictions}}{\text{all predictions}}.$$

This chapter reports macro-F1 as the principal multiclass metric and accuracy where generated artifacts or baselines use it. Per-class precision, recall, F1, and confusion matrices remain necessary because one aggregate number cannot reveal which semantic classes are confused.

3.7.3 Baselines and Promotion Criteria

A learned model should be compared with an inexpensive alternative that uses the same available information. The Random Forest is compared with majority and threshold rules. The raw-opcode Transformer is compared with both majority prediction and logistic regression over opcode counts.

Beating a baseline is necessary but not sufficient for production authority. Promotion also requires:

- an input profile available at APK-only inference time;
- case-separated target and control evidence;
- no regression in default matcher outputs;
- semantic SBOM and vulnerability equivalence where assistance is report-only;
- nonzero eligible candidates after strict controls; and
- a documented rollback and authority boundary.

The present models do not satisfy the authority gates for matcher or SBOM changes.

3.8 Results and Interpretation

3.8.1 Comparative Summary

Table 3.6 places the four trained models in their correct evidence context.

Table 3.6: AI model results, input authority, and project decisions.

Model	Input and split	Test result	Permitted interpretation
Random Forest	17 structured features; 20,523 rows; grouped by case	Macro-F1 0.708; accuracy 0.690	Useful project-evidence triage and integrated report-only suggestions; not exact-version authority.
Full structured Transformer	Full project tokens with source and context shortcuts; same grouped split	Macro-F1 0.977; accuracy 0.992	Complete DL workflow proof and research-status evidence only.
MVP-safe Transformer	Source/context shortcuts removed; same grouped split	Macro-F1 0.882; accuracy 0.907	More credible source-free triage signal; candidate for future report-only use, wiring disabled.
Raw-opcode Transformer	1,680 method sequences from 14 APKs; case-held-out by label	Macro-F1 0.110; accuracy 0.146	Negative result; below opcode-count logistic regression and not promotable.

The strongest numerical result is not the strongest deployment evidence. The full structured Transformer has the highest score and the weakest input independence. The MVP-safe Transformer sacrifices performance to remove unavailable source and diagnostic context. The raw-opcode model uses the most direct APK-derived input but performs poorly on unseen cases. The Random Forest occupies the practical middle: its inputs are already present in project reports, its behavior is inspectable through feature importance and ablation, and its authority is limited to conservative triage.

3.8.2 Random Forest Class-Level Behavior

The aggregate Random Forest macro-F1 of 0.708 hides substantial class variation. Table 3.7 reports the generated test metrics.

Table 3.7: Random Forest per-class results on 3,868 grouped test rows.

Label	Precision	Recall	F1	Support
`suggest_report_only`	1.000	1.000	1.000	157
`suppress_uncertain`	0.858	0.973	0.912	112
`risk_only`	0.991	0.214	0.352	1,483
`confirmed_exact`	1.000	1.000	1.000	1,471
`mismatch_or_extra`	0.101	0.782	0.179	133
`boundary_do_not_suggest`	0.671	1.000	0.803	512

The model separates several policy-shaped classes almost perfectly, but it confuses

`risk_only` with `mismatch_or_extra` and boundary states. Of 1,483 true risk-only rows, only 317 are predicted as risk-only; 926 are predicted as mismatch/extra and 222 as boundary. This explains the high risk-only precision but low recall, and the inverse pattern for the mismatch class.

For report triage, this conservative drift can still be useful because it tends to escalate uncertain rows rather than confirm components. For semantic classification, however, it is weak. The model should not be presented as having uniformly strong six-class performance.

3.8.3 Shortcut Risk

Shortcut learning occurs when a model uses a predictive pattern that is easier than the intended task but does not generalize to the deployment environment [35]. Three shortcut categories are relevant here:

- **label-construction shortcuts**: diagnostic context such as "next action" or "reason label" directly reflects the rules used to create the training class;
- **source shortcuts**: source and candidate versions expose information unavailable in a source-free APK-only decision;
- **case/ecosystem shortcuts**: coordinate names and repeated project conventions may identify familiar cases without learning general obfuscation behavior.

The grouped split controls row sharing within a case, but it does not remove all three shortcut types. The full-to-MVP-safe score drop is direct evidence that shortcut-prone fields contributed to performance. The safer score is therefore the more defensible result for discussing future report assistance.

3.8.4 Negative Evidence as a Result

The raw-opcode experiment was designed to move closer to source-free bytecode learning. Its failure is scientifically useful for three reasons:

1. it prevents promotion based only on the popularity of Transformer architectures;
2. it demonstrates the importance of comparing against a simple opcode-count baseline; and
3. it identifies missing context and dataset scale as concrete future requirements.

Had only the 0.977 full-profile result been reported, the AI contribution would appear stronger but less trustworthy. Reporting the safer 0.882 result and the negative 0.110 result gives a more accurate picture: learned triage is promising, while learned APK-only component authority remains unproven.

3.9 Production Authority and Safety Boundary

3.9.1 Integration Level

The current integration level is `production-report-only-ai-assistance`. The normal report command can include a hybrid AI section through:

```
--include-hybrid-ai-assistance
```

The section exposes Random Forest recommendations, strategy routing, R8 structural-gate status, and Transformer research status. It does not feed predictions back into the matcher.

The integration gate evaluated 31,589 project-manifest rows, of which 20,523 are training-eligible and 11,066 belong to held-out or report-only partitions. It concluded:

- Random Forest: keep integrated as report-only;
- MVP-safe Transformer: candidate for report-only risk triage;
- full structured Transformer: retain as research-status only;
- raw-opcode Transformer: retain as research-only future work; and
- matcher/SBOM authority: blocked.

The strict APK-only promotion matrix tested five guarded profiles. Although 219 rows passed preliminary conditions, zero candidates remained eligible after representative and same-coordinate controls. The default matcher therefore remains unchanged.

3.9.2 Report-Only Guardrail Proof

A two-target smoke test compares default reports with opt-in hybrid/AI reports for F-Droid and Aegis Authenticator. The test verifies that:

- hybrid sections are visible;
- detected component sets are unchanged;
- selected versions are unchanged;
- CycloneDX component content is unchanged;
- vulnerability output is unchanged; and
- the default matcher remains unchanged.

This is a stronger guardrail than a prose statement because the two report paths are generated and compared. The test does not prove model accuracy on new APKs; it proves absence of authority leakage into protected outputs.

3.9.3 Authority Matrix

Table 3.8 records the production boundary.

Table 3.8: AI authority matrix in the implemented MVP.

System action	AI authority	Responsible mechanism
Report wording and risk triage	Allowed, opt-in	Random Forest and hybrid router annotations with visible report-only status
Investigation prioritization	Allowed	Report-only profiles and evidence summaries
Component presence decision	Forbidden	Deterministic multi-feature matcher and acceptance policy
Exact-version selection	Forbidden	Deterministic matcher, ambiguity rules, and source validation where available
CycloneDX component emission	Forbidden	SBOM generator receives accepted deterministic detections only
Vulnerability confirmation	Forbidden	Component/version evidence followed by OSV affected-range interpretation
Source-validation metric	Forbidden	Direct comparison between emitted SBOM rows and source dependency evidence

Figure 3.3 summarizes the same boundary as a control flow.

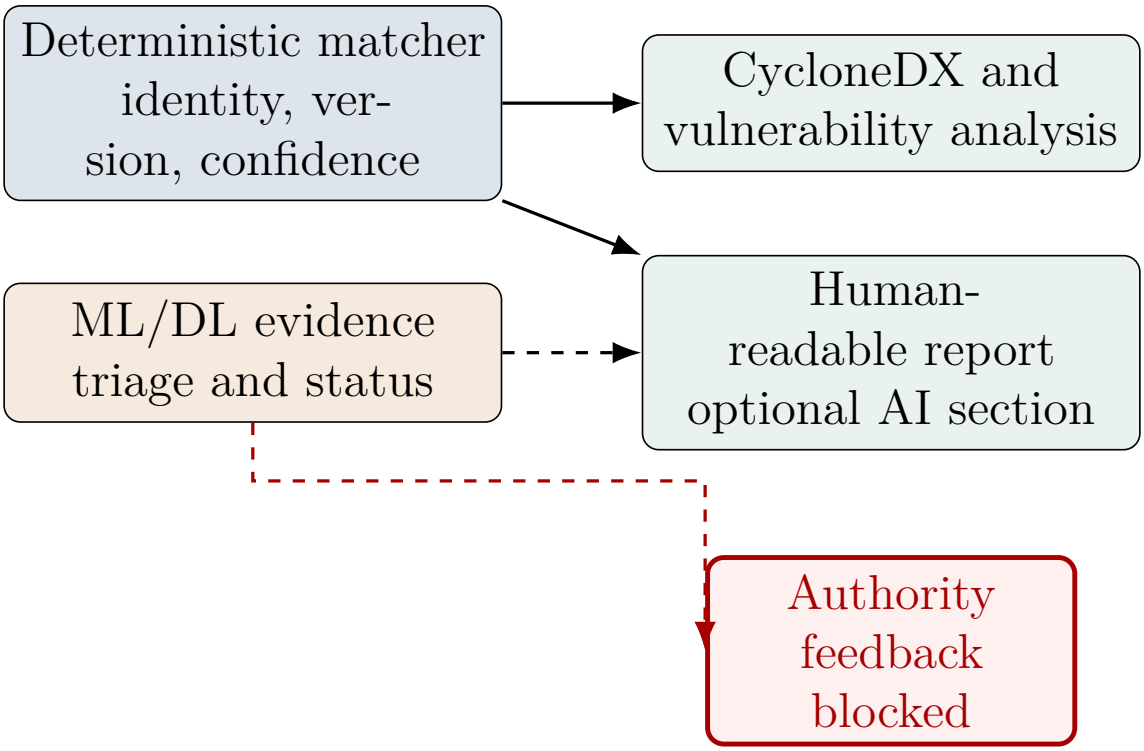


Figure 3.3: Learned evidence may enrich reports but is blocked from the authoritative matcher-to-SBOM path.

3.10 Threats to Validity and Lessons Learned

3.10.1 Dataset Validity

The structured dataset is large in row count but narrow in provenance. Many rows arise from the same project pipelines and decision rules. Row count does not equal independent APK diversity. The 31,589-row manifest includes project-held-out lanes but explicitly records that no external corpus was used.

The raw-opcode dataset has stronger case separation but only 14 APKs. Two APK cases per label are insufficient for broad claims about R8 behavior. Method-level rows from one APK also share build configuration and application context, so they cannot be treated as fully independent samples.

3.10.2 Label Validity

The structured labels represent actions and evidence states used by this project. Some labels are produced from fields later included in the full token profile, creating shortcut risk. The labels are valuable for automating project triage, but they are not a neutral ontology of all Android component identification errors.

Future work should include independently reviewed labels, explicit annotation guidelines, inter-rater agreement, and test applications collected outside the current evidence-generation process.

3.10.3 Metric Validity

Macro-F1 protects minority classes from being hidden by frequent classes, but it does not express calibration, operational cost, or the severity of a false confirmation. The Random Forest's perfect results on some policy-shaped classes coexist with weak risk/mismatch separation. Per-class metrics and the confusion matrix are therefore necessary.

The full structured Transformer's high score is not directly comparable to the raw-opcode result because the tasks, input information, classes, and datasets differ. The chapter compares them as project experiments, not as a single leaderboard.

3.10.4 Reproducibility

All models use fixed random state 42, generated JSON corpora, recorded parameters, serialized model files, and deterministic split functions. Artifact hashes are stored in the Chapter 3 evidence snapshot. These controls support reproduction of the current local evidence.

Reproducibility does not solve external validity. Re-running the same project-derived corpus can reproduce a score without proving performance on a new ecosystem or time period.

3.10.5 Engineering Lessons

The AI experiments produced five practical lessons:

1. grouping by APK case is a minimum requirement; random row splitting would be unsafe;
2. high structured-model performance must be audited for unavailable source and workflow context;
3. feature importance and ablation are useful for understanding dependence, but not causal proof;
4. a simple baseline can defeat a more complex neural model; and
5. report-only integration can provide visible value without granting unsupported component or vulnerability authority.

3.11 Conclusion

Artificial intelligence is a supporting contribution in this project, not the source of the final SBOM. A case-grouped Random Forest achieved macro-F1 0.708 and is useful for report-only triage, although its risk and mismatch classes remain weak. A full structured-token Transformer achieved macro-F1 0.977, but its source and diagnostic context create shortcut risk. Removing those fields reduced the score to 0.882 and produced a more credible, still report-only result.

The most direct bytecode-learning experiment was negative. A raw-opcode Transformer achieved macro-F1 0.110 on held-out APK cases, below the opcode-count logistic-regression

baseline of 0.128. This result blocks promotion and shows that limited method sequences without broader structural context are insufficient.

The implemented safety boundary is therefore deliberate: learned models may annotate reports and prioritize investigation, while the deterministic matcher remains responsible for component identity, exact version, CycloneDX emission, vulnerability interpretation, and source-validation metrics. Chapter 4 now places this bounded AI contribution within the broader evolution of Android third-party-library identification techniques.

Part III

State of the Art

Chapter 4

State of the Art

4.1 Introduction and Review Scope

Recovering an SBOM from a compiled Android APK intersects several research areas: third-party library identification, exact-version detection, obfuscation-resilient similarity, software composition analysis, and vulnerability mapping. This chapter reviews the approaches most directly related to those tasks and identifies the gap addressed by the proposed system.

The review corpus was assembled from the project paper collection and then audited against accessible primary PDFs, publisher records, arXiv records, and public artifacts. The comparison focuses on:

- library-level and version-level identification;
- the representation used to compare APK and reference code;
- resistance to renaming, shrinking, repackaging, and optimization;
- partial-library and multidex behavior;
- dataset realism and metric interpretation;
- runtime, availability, and reproducibility;
- standard SBOM output and vulnerability mapping.

Reported percentages are not treated as a universal ranking. The reviewed papers use different datasets, obfuscators, reference libraries, thresholds, and definitions of a correct match. Original-paper results are therefore presented as evidence about a method under its own evaluation conditions, while the later common-benchmark results are used to discuss realistic generalization.

4.2 Evolution of Android Third-Party Library Detection

Android third-party library detection evolved from fast package and frequency signatures toward structural correspondence, optimization-aware matching, and learned representa-

tions. Figure 4.1 summarizes this progression. The generations overlap: later tools often retain efficient signatures as a coarse stage and add deeper evidence only for shortlisted candidates.

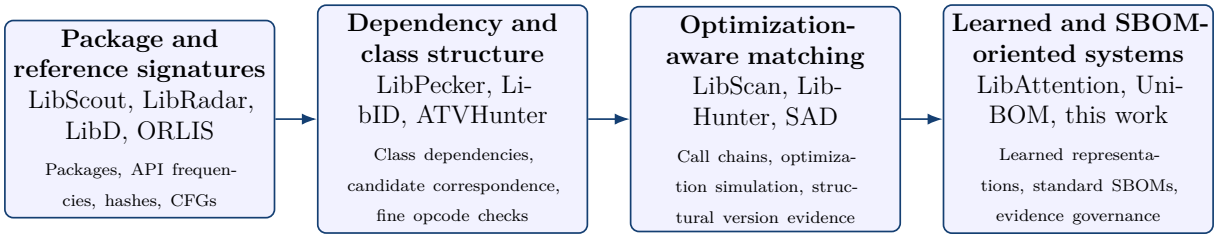


Figure 4.1: Evolution from fragile package signatures toward structural, optimization-aware, learned, and SBOM-oriented analysis.

4.2.1 Package, Frequency, and Reference Profiles

Early approaches sought compact signatures that could be generated and compared at scale. LibRadar, for example, used Android API frequencies and package clustering to discover reused code quickly. Its design favored speed, but clustering did not provide reliable exact-version labels and remained dependent on package organization. Later systematic evaluations found that package-structure-based candidate construction is error-prone and that multidex, partial imports, and library dependencies create additional blind spots [37, 9].

LibScout established a reference-profile model for identifying known libraries and versions. It represents packages, classes, and methods in a Merkle-style hierarchy and replaces application-specific types in method descriptors with placeholders. This improves resilience to identifier renaming and permits similarity-based partial matching. Its longitudinal study also demonstrated the security value of exact library-version attribution [6]. The main limitation is structural: package and class hierarchies can change under flattening, shrinking, and optimization.

LibD moved from names toward internal dependencies and opcode-derived hashes. Its homogeny graph uses inclusion, inheritance, and call relations to recover candidate components before feature hashing. The approach was evaluated at market scale and demonstrated that structural relations can be more robust than package labels, but it focused on discovering recurring libraries rather than resolving exact versions [38].

ORLIS used a two-stage design: similarity digests first reduced the candidate space, after which class-level CFG evidence and Jaccard similarity supported a finer comparison. The work is important because it combined a coarse filter with partial-library similarity and explicitly studied obfuscation [39]. Its principal trade-off was computational cost, showing that structural resilience must be balanced with scalable candidate reduction.

4.2.2 Class Dependencies and Version-Aware Detection

LibPecker computes strict class signatures from internal class dependencies, then introduces adaptive class thresholds and weighted similarity to tolerate customization and eliminated dead code. Its original evaluation reported 98.1% precision and 91% recall [40]. These results are methodologically useful, but later common-benchmark evidence shows why

original-paper performance should not be generalized without preserving the dataset and protocol.

LibID formulates library identification as a constrained correspondence problem and uses binary integer programming to select consistent matches. It also uses synthetic applications to tune thresholds and F-Droid applications as source-backed ground truth. The method identifies specific versions and is designed to survive code shrinking and package modification [41]. Its optimization formulation improves global consistency but increases implementation and runtime complexity.

ATVHUNTER addresses version and vulnerability identification through a coarse-to-fine pipeline. CFG evidence retrieves potential references, then opcodes inside basic blocks distinguish candidate versions. The work reports 90.55% precision and 88.79% recall and connects detected versions with a vulnerability database [7]. This is one of the closest pre-SBOM pipelines to the current project because it links compiled-code attribution, exact versions, and security consequences. However, its output is not a standard SBOM, and its evaluation does not fully represent modern R8 method inlining.

4.3 Optimization-Aware and Structural Matching

LibScan builds potential class correspondences and refines them through method opcode and call-chain opcode similarity. Early-stop rules and reuse of intermediate results reduce the cost of the deeper comparison. This design addresses repackaging, dead-code removal, and control-flow changes more directly than package-only methods [42].

LibHunter studies a narrower but important question: how compiler optimizations affect library and version detectors. It explicitly models inlining and call-site optimization and reports improvements over earlier tools on optimized, obfuscated, and shrunk applications [43]. Its contribution is methodological as well as algorithmic: optimization must be evaluated separately from identifier renaming because it can remove or merge the very units used by a matcher.

SAD targets version-level false positives through class structural correspondence. It first retrieves candidate app classes from class dependency graphs, compares class summaries, and finally evaluates the structural similarity of the matched subgraphs. On its obfuscated datasets, SAD reports 97.64% library-level F1, 84.82% version-level F1, and 94.12% class-level F1 [8]. These are strong reported results, but they belong to the paper's three evaluation datasets and should not be compared directly with a different real-APK denominator.

The public LibScope artifact represents another direction: replacing independent local matches with iterative global matching over compact Bloom filter features [44]. Global consistency is relevant when several classes have individually plausible correspondences. At the time of this review, the accessible artifact was anonymized, so it is included as an emerging matching direction rather than as a source of quantitative claims.

4.4 Learned Representations

LibAttention replaces hand-written correspondence rules with a feature language model. APK and library code are converted to Smali, transformed into an obfuscation-resistant

vocabulary, encoded by a Transformer, and compressed for downstream classification. The paper reports an F1 improvement exceeding 30 percentage points over rule-based methods on its Android R8 dataset [45].

The learned approach is valuable because R8 transformations evolve and may not preserve the exact features expected by a manually designed detector. Nevertheless, LibAttention performs library-presence classification rather than exact-version identification. It therefore cannot independently provide the version evidence required for reliable vulnerability mapping or a versioned SBOM. Learned representations are better understood here as assistance for hard identity or risk cases than as automatic version authority.

4.5 SBOM-Oriented Approaches

Most Android TPL detectors output library names, candidate versions, or class correspondences rather than a standard SBOM. That difference matters because an operational SBOM also requires package identifiers, evidence provenance, component relationships, uncertainty handling, and an interoperable format.

UniBOM is not Android-specific, but it demonstrates a broader SBOM-analysis architecture that combines multiple evidence sources and supports CycloneDX and SPDX representations [46]. Its relevance to this project is architectural: binary recovery should be connected to standard output, visualization, and vulnerability analysis rather than stopping at a detector-specific list.

The Android-specific gap remains significant. ATVHUNTER connects versions to vulnerabilities, and LibScout studies vulnerable library histories, but neither provides the evidence-qualified CycloneDX workflow required by this project. Standard SBOM generation must also avoid converting a top similarity candidate into an unsupported exact version.

4.6 Benchmark and Assessment Evidence

The 2020 empirical assessment and its extended systematic study compared publicly available tools under common criteria. They found that no single method dominated all dimensions: LibScout was strong in effectiveness, LibRadar in efficiency and usability, and LibPecker in the evaluated obfuscation settings. More importantly, the studies identified module decoupling, partial imports, multidex, AAR preprocessing, and tool complementarity as recurring problems [37, 9].

TPL-Benchmark revisits the field with 6,055 applications, 5,756 unique libraries, 15,274 versions, and 62,218 app-library mappings. Under its unified protocol, the highest library-level F1 among the ten evaluated tools is 60.15%, achieved by LibPecker, despite substantially higher values reported in several original papers [10]. This does not prove that the original experiments were incorrect. It demonstrates that performance is highly sensitive to dataset construction, ground truth, reference coverage, partial inclusion, and modern compiler transformations.

The benchmark also separates library-level from version-level results and shows that stronger identity recall does not automatically imply reliable version selection. This observation directly motivates separate identity and version confidence in the proposed

system.

4.7 Comparative Analysis

Table 4.1 compares representative tools using common capability dimensions. Quantitative results remain in the preceding text so that each number stays attached to its evaluation context.

Table 4.1: Capability-oriented comparison of representative Android TPL and SBOM approaches.

Approach	Primary goal	Main evidence	Version level	Optimization focus	Output
LibScout	Known library/version matching	Fuzzy method descriptors and hierarchical hashes	Yes	Renaming; limited structural change	Detector-specific library/version list
LibD	Market-scale library discovery	Dependency relations and opcode hashing	No	Name obfuscation	Discovered library clusters
ORLIS	Obfuscation-resilient matching	CFG structure, sdhash, ssdeep, Jaccard	Limited	Classical obfuscators	Class/library mappings
LibPecker	Precise partial-library matching	Class dependencies, adaptive thresholds, weighted similarity	Limited	Customization and dead-code removal	Detector-specific matches
LibID	Consistent known-library matching	Class/member features and integer programming	Yes	Shrinking and package modification	Library/version identifications
ATVHUNTER	Version and vulnerability identification	CFG shortlist and basic-block opcode evidence	Yes	Obfuscation and dead-code removal	Version and vulnerability records
LibScan	Fine class correspondence	Method and call-chain opcode similarity	Yes	Repackaging and advanced obfuscation	Detector-specific matches
LibHunter	Optimization-aware detection	Inlining and call-site optimization simulation	Yes	Explicit compiler optimization	Library/version matches
SAD	Version-level structural matching	CDG candidates, class summaries, matched subgraphs	Yes	Obfuscated applications	Version and class correspondences
LibAttention	Learned library presence	Smali feature language model and Transformer	No	Android R8	Binary library presence
UniBOM	Unified SBOM analysis	Binary, filesystem, and source evidence	Depends on input analyzers	Not Android-specific	CycloneDX and SPDX

The table exposes a recurring trade-off. Fast approaches reduce search cost but provide less reliable version evidence. Structural and optimization-aware approaches improve robustness but are more expensive and remain dependent on reference coverage. Learned methods adapt to transformations but do not automatically provide explainable version attribution. SBOM-oriented systems provide interoperable output but need trustworthy component evidence from upstream analyzers.

4.8 Limitations of Existing Approaches

4.8.1 Dataset and Metric Fragmentation

Original papers frequently evaluate curated libraries or generated obfuscation variants, while later benchmarks include broader real applications. Precision, recall, F1, version accuracy, and class correspondence may also use different units. A value reported for one dataset cannot be treated as a universal ceiling or directly compared with another paper's percentage.

4.8.2 Package and Reference Dependence

Package-based segmentation is fragile under renaming and flattening. Reference-based systems avoid clustering ambiguity but cannot detect a coordinate or version that is absent from the database. Adding many versions also increases candidate pressure and can create false near-ties. Reference coverage and matcher performance must therefore be measured separately.

4.8.3 Incomplete R8 and Optimization Coverage

Renaming is the easiest transformation because much of the bytecode structure survives. Shrinking removes library code, while optimization and inlining rewrite method boundaries and call relations. The progression from LibScout through LibScan, LibHunter, SAD, and LibAttention shows increasing attention to these transformations, but no reviewed approach establishes universal exact-version recovery under all modern R8 behaviors.

4.8.4 Weak Standard Output and Provenance

Most detectors stop at tool-specific matches. They do not emit a standard SBOM, distinguish application files from Maven packages, preserve ambiguous version sets, or attach the evidence required to audit a claim. Vulnerability mapping is particularly sensitive to this omission because the same coordinate at two adjacent versions may produce different advisory results.

4.8.5 Limited Uncertainty and Failure Reporting

A top candidate is not necessarily a confirmed component. Several reviewed approaches expose similarity scores, but few make identity uncertainty, version uncertainty, reference

absence, and unsupported native or hidden code first-class output categories. Realistic deployment requires abstention and bounded claims, not only improved ranking.

4.9 Impact on the Proposed MVP

The project did not copy one detector. It adopted, adapted, or rejected ideas according to implementation feasibility and target/control evidence. Table 4.2 summarizes the principal decisions.

Table 4.2: State-of-the-art ideas and their disposition in the proposed MVP.

Research idea	Decision	Project implementation or rationale
Known versioned reference profiles	Adopted	Maven JAR/AAR artifacts are converted to DEX and stored as coordinate/version feature profiles.
Package-only module boundaries	Rejected	Package evidence remains one signal, but structural, opcode, literal, and dependency evidence are required.
Coarse-to-fine candidate verification	Adapted	Candidate reduction and weighted evidence ranking follow the multi-stage pattern, with fail-fast controls for unsafe shortlists.
Class dependency and call-chain evidence	Adopted diagnostically and partially in production	Class dependencies contribute to the matcher; multi-scale features are extractable but remain inert or report-only when global scoring regresses controls.
Generic newest-version tie breaking	Rejected	Recency is not independent APK evidence and caused target/control conflicts.
Learned R8 assistance	Adapted as report-only	Random Forest and Transformer experiments support risk triage but cannot change selected versions, SBOM components, or vulnerability labels.
Standard SBOM and package identifiers	Adopted	Accepted detections are serialized as CycloneDX components with Package URLs and evidence properties.
Single aggregate benchmark score	Rejected	Controlled, clean, partial, source-known, experiment-only, and limitation lanes are reported separately.

This decision process is as important as feature selection. Opcode n-grams, API references, multi-scale R8 blends, broad priority databases, and source-known rerankers were implemented or evaluated, but they remained disabled, diagnostic, or experiment-only when controls did not justify promotion.

4.10 Research Gap and Positioning

The reviewed literature leaves five connected gaps:

1. exact-version evidence degrades more severely than library identity under shrinking and optimization;
2. reference coverage and matcher recall are often mixed into one score;
3. standard SBOM generation is rarely integrated with Android binary attribution;
4. vulnerability claims frequently inherit unexamined version uncertainty;
5. negative experiments, abstentions, and boundary cases are not always visible in the final result.

The proposed contribution is positioned as an end-to-end, evidence-gated APK-to-CycloneDX pipeline. It combines research-backed static features with versioned Maven references, separates identity from exact version confidence, validates emitted rows against source when available, and keeps native, hidden-code, AI, and experimental evidence outside unsupported production authority.

The objective is not to claim a universally superior detector from incompatible percentages. It is to provide a working and auditable system in which each SBOM and vulnerability claim remains connected to its evidence, denominator, and limitation.

4.11 Conclusion

Android TPL research progressed from fast package and hash signatures to class dependencies, global correspondence, optimization-aware simulation, and learned representations. These advances improve robustness, but benchmark evidence shows that realistic library and version recovery remains difficult, particularly when reference coverage, partial inclusion, R8 optimization, and adjacent-version similarity interact.

The literature also reveals a systems gap: accurate detection alone does not produce a trustworthy SBOM. Standard output, evidence provenance, ambiguity, source validation, and vulnerability-claim boundaries must be designed together. Chapter 5 answers this gap by presenting the architecture of the proposed evidence-gated APK-to-SBOM system.

Part IV

Contribution

Chapter 5

Proposed Solution and System Design

5.1 Introduction

The proposed solution is a static, evidence-gated pipeline that transforms a compiled Android APK into a CycloneDX SBOM and a confidence-aware vulnerability report. The design separates extraction, reference construction, matching, version selection, and downstream security interpretation so that each claim can be traced to its supporting evidence.

5.2 Functional and Non-Functional Requirements

The main functional requirements are APK inspection, component candidate generation, exact or ambiguous version estimation, CycloneDX generation, vulnerability lookup, and human-readable reporting. The main non-functional requirements are explainability, reproducibility, conservative claims, bounded experimental isolation, and traceable evidence.

5.3 General Architecture

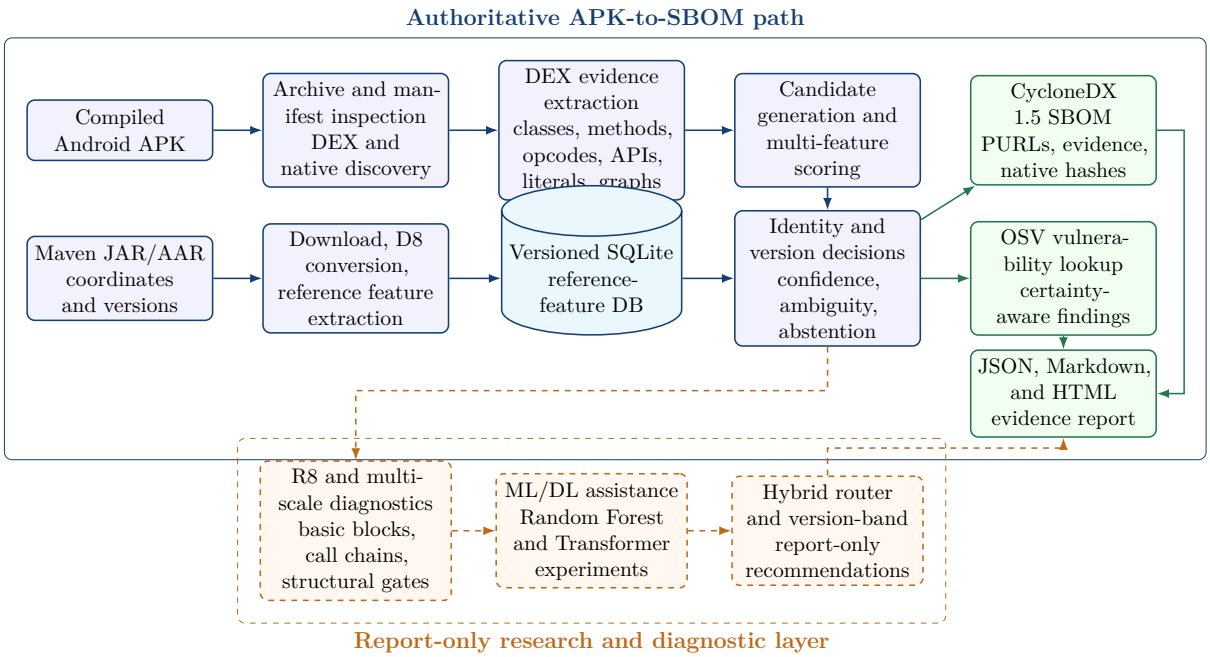


Figure 5.1: Detailed architecture of the implemented APK-to-SBOM system, including the authoritative analysis and reference-building paths and the non-authoritative ML/DL/R8 assistance layer.

The architecture has two authoritative input paths. The upper path extracts archive, manifest, DEX, native, and forensic evidence from the target APK. The middle path converts versioned Maven artifacts into the same DEX-oriented feature representation and stores them in SQLite. These paths meet at candidate scoring and the separate identity/version decision stage. Accepted decisions feed CycloneDX, OSV, and human-readable reports. The dashed orange layer contains R8 diagnostics, multi-scale structural features, Random Forest and Transformer experiments, and the hybrid router. It may annotate a report, but it cannot change selected versions, SBOM components, vulnerability labels, or source-validation metrics.

5.4 APK Extraction and DEX Analysis

The extraction layer discovers standard and nonstandard DEX files, parses application metadata, records resource and asset inventory, and inventories native files. The DEX analysis layer derives classes, methods, opcode statistics, API references, string literals, and dependency structures used during matching.

5.5 Reference Database Design

Reference features are built from versioned Maven JAR and AAR artifacts. Each artifact is downloaded, converted to a comparable DEX representation when necessary, analyzed,

and stored in SQLite. Shared calibrated databases are kept separate from evaluation-only and experiment-only databases.

5.6 Evidence Model

The matcher combines several evidence families:

Table 5.1: Evidence families used by the component-attribution pipeline.

Evidence family	Design role
Packages, classes, methods	Candidate generation and identity support
Opcode features	Bytecode similarity under partial identifier changes
Class dependency graph	Structural evidence across related classes
Literals and unique features	Discrimination between nearby versions
API/member references	Optional contextual signal
POM/dependency context	Conservative coherence support
Application context	Ambiguity explanation and risk reporting

5.7 Candidate Generation and Version Selection

The system first narrows the reference search space and then evaluates candidate coordinates and versions using multiple evidence families. Library identity and exact-version confidence are distinct decisions. When adjacent versions remain inseparable, the system records an ambiguous version set instead of silently converting a near-tie into an exact fact.

5.8 CycloneDX and Vulnerability Design

Selected components are serialized as CycloneDX components with Package URLs and evidence properties. Native libraries are represented as file components with hashes. Vulnerability findings are constrained by component and version evidence: an uncertain version may support a potential investigation hint, but not an unqualified confirmed-vulnerability claim.

5.9 Experimental Isolation and Fail-Fast Controls

Alternative reference databases, AI suggestions, R8 diagnostics, and scoring experiments are isolated from the shared calibrated path. A target improvement is insufficient for promotion if controlled, clean, partial, no-Maven, or boundary controls regress.

5.10 Use Cases and Behavioral Diagrams

The following views refine the high-level architecture into actor responsibilities, runtime behavior, reference construction, deployment boundaries, and persistent data. Solid blue elements belong to the authoritative MVP path. Dashed orange elements are optional research or experiment paths and cannot alter a production decision unless they pass the promotion gates described previously.

5.10.1 Actors and Use Cases

The principal user is an analyst or auditor who supplies an APK and consumes the generated evidence. A reference-database maintainer prepares reproducible Maven examples, while a researcher evaluates changes in isolated databases. OSV is an external service rather than an internal source of component identity.

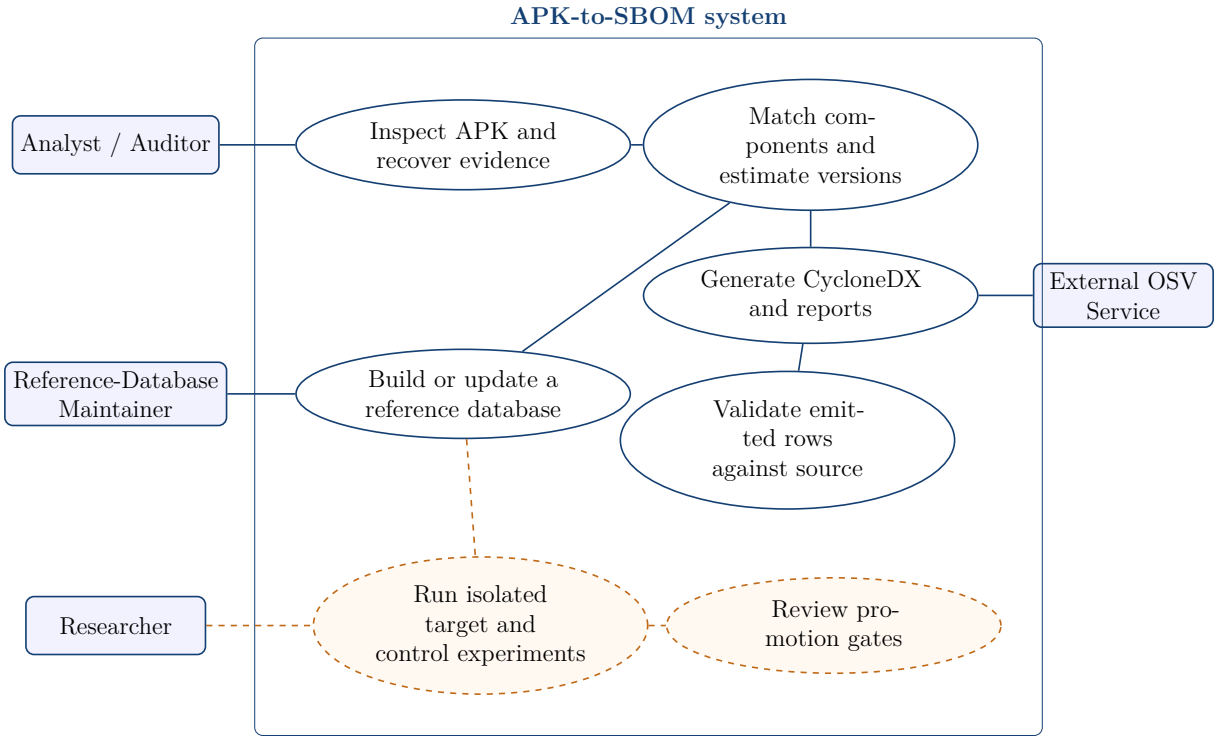


Figure 5.2: Actors and use cases, with experiment-only actions shown as dashed orange elements.

Figure 5.2 separates ordinary analysis from system maintenance and research. In particular, building an experimental database is not equivalent to promoting it: promotion requires an explicit review after target and control results have been checked.

5.10.2 APK-to-Report Activity

The runtime activity in Figure 5.3 shows two conservative branches. A weak or ambiguous match is retained as uncertainty rather than forced into an exact version, and vulnerability lookup is optional so that offline runs remain reproducible.

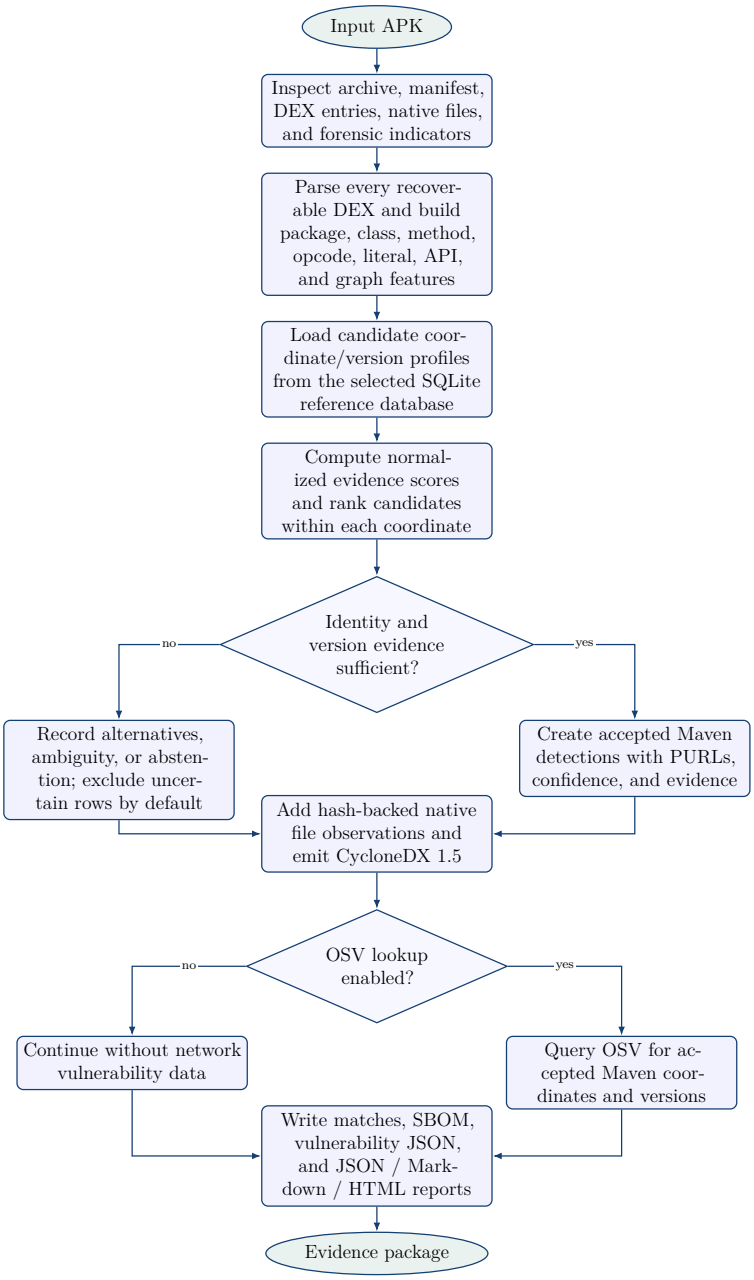


Figure 5.3: Activity flow from APK input to the final evidence package.

5.10.3 APK-to-Report Interaction Sequence

Figure 5.4 identifies the runtime ownership of each operation. The report orchestrator invokes archive inspection and matching; the matcher itself performs DEX analysis and reads the reference database. Only accepted decisions are passed to CycloneDX and OSV generation.

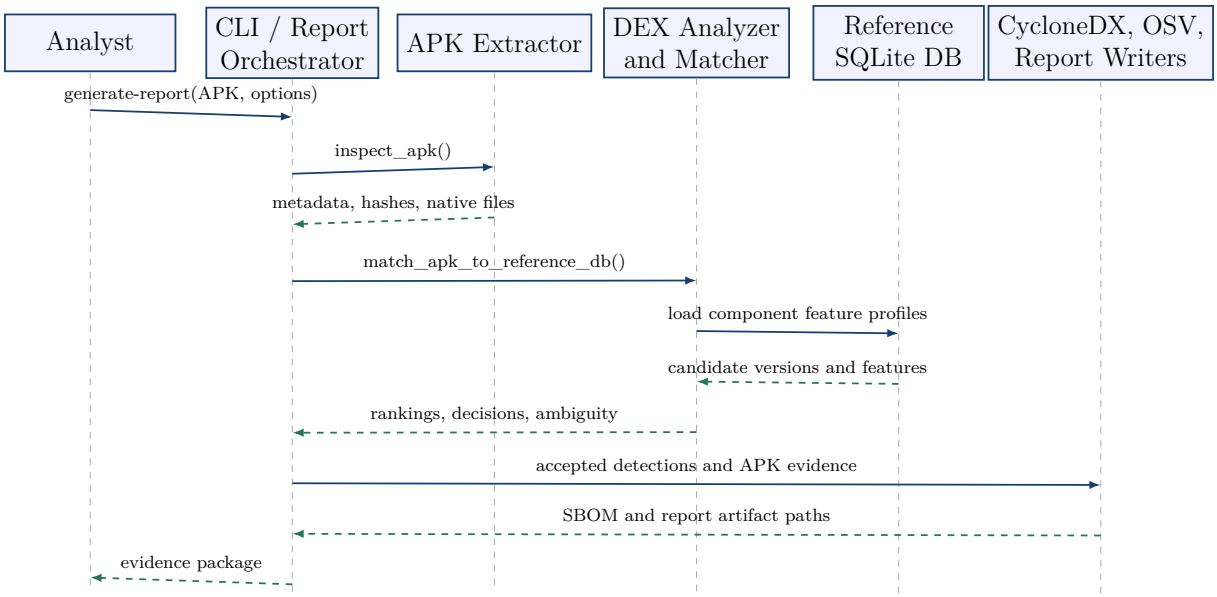


Figure 5.4: Sequence of interactions during an APK-to-report execution.

5.10.4 Reference-Database Construction Sequence

The second sequence, Figure 5.5, uses the same DEX feature representation as APK analysis. This avoids comparing source-level library descriptions directly with compiled APK bytecode. Conversion records also preserve skipped and failed artifacts instead of silently treating them as valid empty libraries.

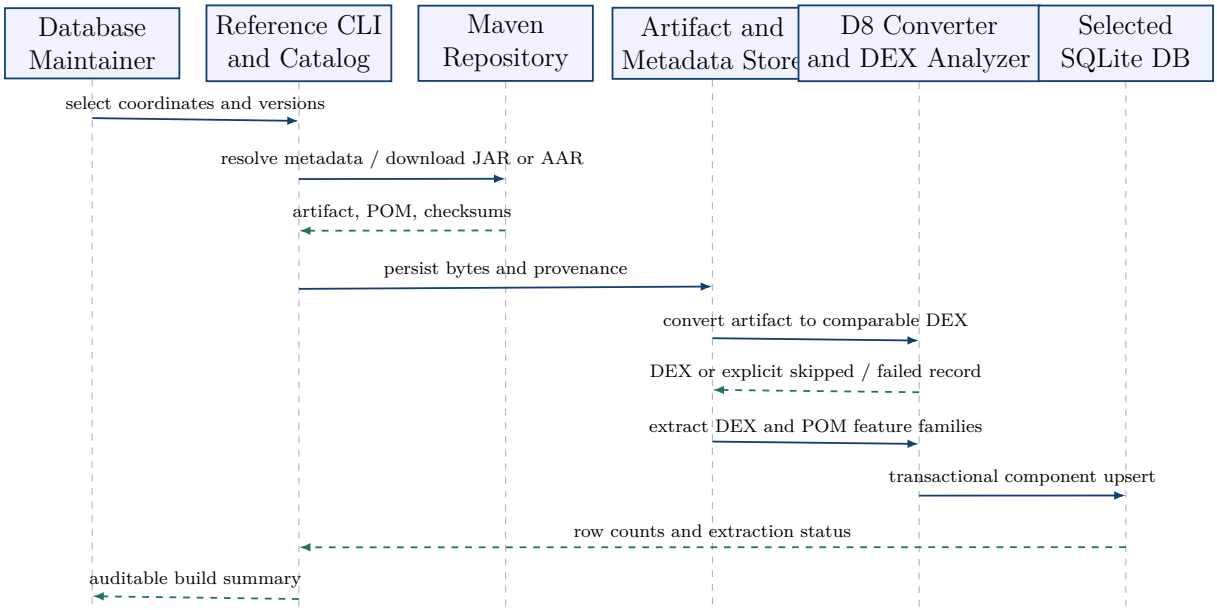


Figure 5.5: Sequence used to construct a versioned Maven reference database.

5.10.5 Component and Deployment View

The implementation can run through the command-line interface directly or in the project Docker environment. APKs, selected SQLite databases, and generated artifacts remain

filesystem resources. Network access is required for Maven acquisition and OSV lookup, but not for deterministic matching with an existing database. Figure 5.6 also makes the authority boundary explicit: report-only AI and R8 diagnostics may annotate a report but cannot feed the matcher, CycloneDX component set, or vulnerability labels.

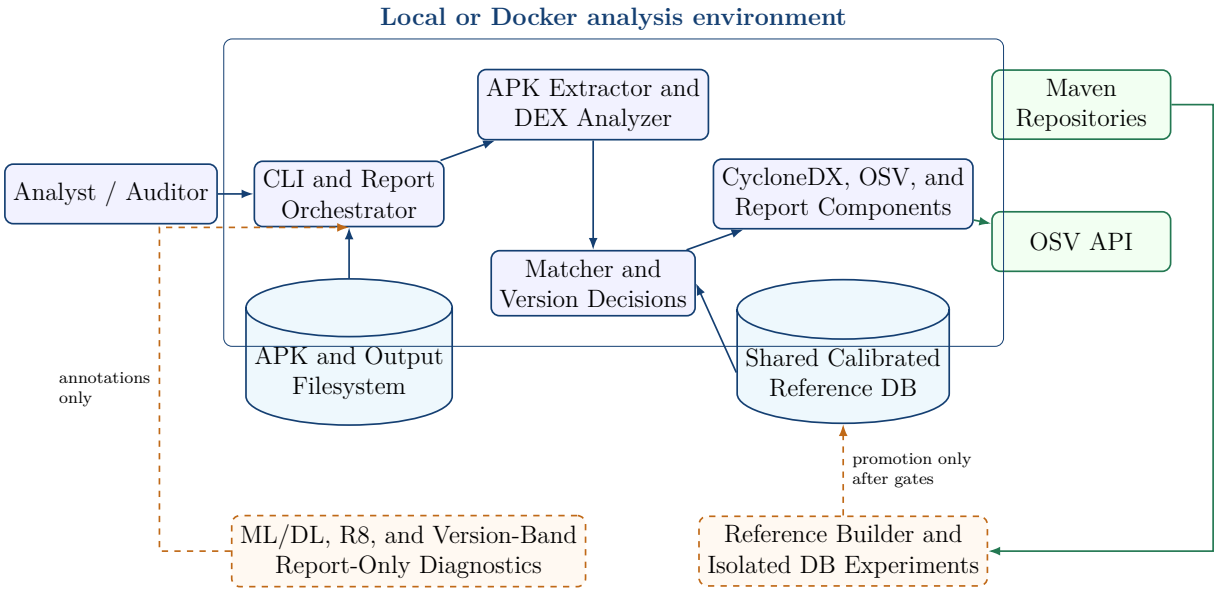


Figure 5.6: Component and deployment view with authoritative and report-only paths separated.

5.10.6 Reference-Database Data Model

The physical SQLite schema is centered on one row per Maven coordinate and version in the `components` table. Every feature table refers to that row through `component_id`. Figure 5.7 groups tables by role to keep the model readable; the labels are the actual table names used by the implementation.

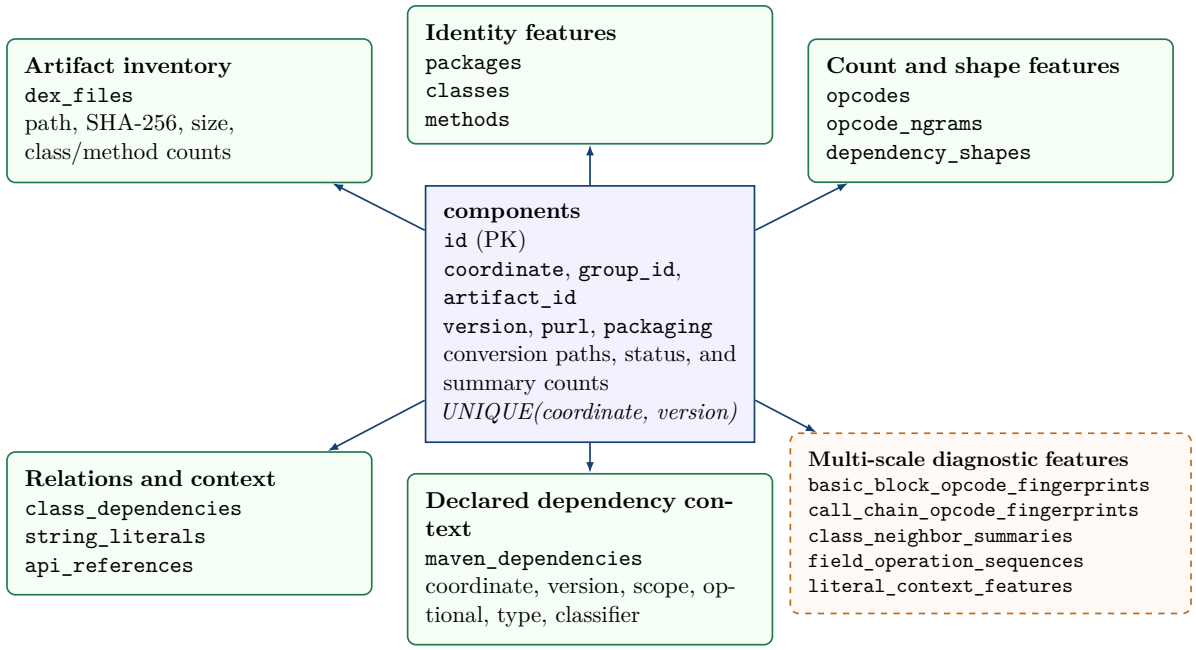


Figure 5.7: Simplified physical schema of the SQLite reference-feature database; each arrow represents a `component_id` foreign-key relation.

This one-to-many organization permits transactional replacement of a component's feature rows while preserving a stable coordinate/version identity. It also allows new diagnostic feature families to be stored without changing the meaning of the calibrated production weights.

5.11 Conclusion

The architecture turns APK observations into explicitly bounded SBOM claims. The following chapter explains how these components were implemented and how their behavior was evaluated.

Chapter 6

Implementation, Evaluation, and Discussion

6.1 Introduction

The architecture presented in Chapter 5 was implemented as a Python command-line system that accepts compiled Android APKs, extracts static evidence, compares it with versioned Maven reference profiles, emits a CycloneDX SBOM, and optionally queries vulnerabilities. This chapter has three objectives. First, it explains the principal implementation modules and their failure behavior. Second, it defines an evaluation protocol that keeps controlled, real, source-known, experimental, and limitation evidence separate. Third, it reports both successful and failed experiments so that the final conclusions reflect the system's actual evidence boundaries.

All numerical results in this chapter were frozen on 20 June 2026 from generated JSON artifacts and regenerated thesis tables. The frozen snapshot is maintained with the project evidence package and records the numerator, denominator, evidence class, permitted interpretation, and principal limitation of each result.

6.2 Development Environment

The implementation uses a small set of technologies selected for reproducibility rather than for visual demonstration. Table 6.1 summarizes their role.

Table 6.1: Development and execution environment of the implemented MVP.

Technology	Project version or role	Use
Python	3.12 or newer	CLI orchestration, evidence extraction, matching, validation, report generation, and evaluation scripts.
Androguard	4.1.3	Parsing DEX bytecode and recovering classes, methods, instructions, literals, and references.
NetworkX	3.4.2	Graph-oriented dependency representations and structural diagnostics.
SQLite	Embedded database	Persistent coordinate/version profiles and child feature tables.
Android build tools	D8, build-tools 36.0.0	Conversion of JAR/AAR bytecode into the same DEX representation analyzed in APKs.
Docker and Compose	Containerized toolchain	Android SDK, Java, Gradle compatibility versions, scanners, and isolated project execution.
CycloneDX	Specification 1.5	Standard JSON SBOM representation [4].
OSV	Maven package/version lookup	Ecosystem-aware vulnerability queries for accepted Maven detections.
Pytest	Development verification	Unit and integration-oriented regression tests.

The Docker image includes Java, Android command-line tools, D8, several Gradle versions, and optional external SBOM scanners. Multiple Gradle versions are necessary because the real-APK source-validation set contains both modern and legacy Android projects. The core matcher itself does not require network access after the APK and SQLite reference database are available. Network access is needed only for operations such as Maven acquisition and OSV queries.

During the final evidence freeze, the local project virtual environment passed all 275 tests in 34.64 seconds. A separate compilation check over `src/`, `scripts/`, and `tests/` also passed. These checks verify the current Python implementation; they do not replace APK-level target and control evaluations.

6.3 Implementation by Module

6.3.1 Module Map

Table 6.2 maps the implemented responsibilities to their principal source files. The implementation deliberately separates extraction, matching, SBOM generation, vulnerability interpretation, and source validation so that a failure in one stage does not silently become

evidence for another.

Table 6.2: Principal implementation modules and their observable outputs.

Module	Principal source	Responsibility and output
CLI	`src/apk_sbom/cli.py`	Parses commands and flags, selects stable or experimental paths, and writes structured command output.
APK extractor	`src/apk_sbom/apk_extractor.py`	Validates the APK archive; records hashes, manifest data, DEX entries, native files, and forensic indicators.
DEX analyzer	`src/apk_sbom/dex_analyzer.py`	Aggregates package, class, method, opcode, literal, API, and dependency evidence across all recoverable DEX files.
Reference acquisition	`src/apk_sbom/maven_downloader.py`	Downloads versioned JAR/AAR artifacts, POM metadata, declared dependencies, and checksums.
Reference conversion	`src/apk_sbom/reference_converter.py`	Extracts AAR bytecode when required and invokes D8 to obtain comparable DEX files.
Feature database	`src/apk_sbom/reference_features.py`	Transactionally stores one component row per coordinate/version and its child feature rows.
Matcher	`src/apk_sbom/matcher.py`	Scores candidate profiles, separates identity from version confidence, records alternatives, and applies guarded family logic.
SBOM generator	`src/apk_sbom/sbom.py`	Produces CycloneDX 1.5 components, Package URLs, evidence properties, and native file hashes.
Vulnerability layer	`src/apk_sbom/vulnerability.py`	Queries OSV, deduplicates aliases, extracts severity, and applies certainty labels.
Report generator	`src/apk_sbom/report.py`	Orchestrates the full pipeline and writes JSON, Markdown, and HTML reports.
Source validator	`src/apk_sbom/source_validation.py`	Compares emitted Maven components with resolved Gradle dependency evidence.
AI/R8 assistants	`src/apk_sbom/ai_r8_assistant.py` and related modules	Produce report-only risk and investigation suggestions without changing authoritative output.

6.3.2 Extraction and DEX Analysis

The extractor first checks that the input is a readable ZIP/APK archive and computes its SHA-256 hash. It discovers both standard `classes*.dex` entries and DEX files in unusual paths. The latter are retained as forensic indicators because hidden or dynamically loaded code cannot be treated as ordinary statically recovered application logic.

Manifest metadata includes the package name, version name, version code, minimum SDK, target SDK, and declared permissions. Native `.so` entries are recorded with their ABI, path, size, and hash. They are not converted into Maven identities because the current reference pipeline is designed for JVM/Android bytecode artifacts.

The DEX analyzer parses each discovered DEX independently and then aggregates the successfully recovered evidence. A parse error is retained in the output; it is not converted into an empty-feature result. The default score uses packages, class descriptors, method signatures, opcode histograms, class dependencies, dependency-shape summaries, literals, and version-unique class/method/literal evidence. API-reference and opcode n-gram features are extracted but have zero default weight. The guarded version-unique literal/context reranker and composite family shortlist are enabled by default, while broader basic-block, call-chain, class-neighbor, field-operation, and multi-scale features remain diagnostic or report-only.

6.3.3 Reference Construction

The reference pipeline begins with a catalog of Maven coordinates and selected versions. Each artifact is downloaded with provenance metadata and a SHA-256 hash. JAR artifacts are passed directly to D8. For AAR artifacts, the converter extracts `classes.jar` first. Resource-only, POM-only, and source-wrapper artifacts are recorded as no-bytecode outcomes instead of being represented as empty libraries.

Converted references are processed by the same DEX analyzer used for APKs. This shared representation reduces a major comparison mismatch: APK evidence is compared with compiled reference evidence rather than source-code descriptions. The SQLite database stores one unique component row for each coordinate/version pair and child rows for every feature family. Re-extraction clears and rewrites the selected component's child rows transactionally.

Exploratory growth never writes directly into the shared calibrated database. Focused, priority, large, and single-coordinate experiments use separate SQLite paths. A reference addition becomes eligible for shared promotion only after target improvements and current controls are evaluated.

6.3.4 Evidence Scoring and Version Decisions

For a candidate component c, the matcher combines the configured feature scores using a normalized weighted sum:

$$S(c) = \frac{\sum_{i \in F} w_i s_i(c)}{\sum_{i \in F} w_i}, \tag{6.1}$$

where F is the configured feature set, $s_i(c)$ is the corresponding similarity or coverage

score, and w_i is its configured weight. A missing or empty reference feature family contributes a score of zero; a disabled family has weight zero. The default weights are shown in Table 6.3. The resulting value is a ranking score, not a calibrated probability.

Table 6.3: Default matcher weights in the authoritative path.

Feature	Weight	Feature	Weight
Package coverage	0.15	Class coverage	0.45
Method coverage	0.25	Opcode similarity	0.15
Dependency similarity	0.10	Literal overlap	0.08
Version-unique evidence	0.12	API references	0.00
Opcode tri-grams	0.00		

Only features with nonzero configured weights contribute to the normalized denominator. The zero default values are intentional: API-reference and opcode tri-gram experiments did not justify global production weight. The class signal receives the largest weight because it is discriminative in clear cases, while structural, opcode, and literal evidence reduce dependence on preserved names.

Candidate versions are grouped by coordinate. The matcher records the best candidate, score gap, alternatives, ambiguity reason, and any versions close enough to form an ambiguous set. Identity confidence answers whether the library family is supported; version confidence answers whether one release is sufficiently separated from sibling versions. A strong identity can therefore coexist with an uncertain exact version.

6.3.5 SBOM, Vulnerability, and Reporting Layers

Accepted Maven decisions become CycloneDX components with group, artifact, version, PURL, confidence labels, match type, ambiguity properties, and evidence summaries. Native files become CycloneDX file components with hashes and metadata. Uncertain Maven decisions are excluded by default; an explicit flag is required to include them.

The vulnerability layer sends Maven coordinate/version pairs to the OSV batch endpoint. Returned OSV, GHSA, and CVE identifiers are deduplicated, and severity data is extracted when available. A source-validation artifact can then relabel a finding as source-confirmed, source-version-mismatch, source-missing, or report-only. This policy prevents a correct vulnerability lookup on an incorrect selected version from being presented as a confirmed application vulnerability.

The report generator writes:

- `matches.json`, containing ranked candidates and decisions;
- `sbom.cdx.json`, containing the standard SBOM;
- `osv-vulnerabilities.json`, when OSV is enabled;
- `report.json`, the combined machine-readable evidence;
- `report.md` and `report.html`, the human-readable reports.

Optional AI, R8, version-band, and hybrid sections are loaded as attachments. They can add explanatory text but cannot modify selected versions, SBOM components, vulnerability labels, or source-validation metrics.

6.4 Command-Line Workflow

6.4.1 Minimal APK-to-Report Run

The ordinary workflow requires an APK and an existing SQLite reference database:

Listing 6.1: Minimal deterministic end-to-end report command.

```
docker compose run --rm apk-sbom python -m apk_sbom.cli generate-report \
  path/to/application.apk \
  --db-path data/reference_db/reference_features_calibrated_combined.sqlite \
  --output-dir data/results/application-report
```

For an offline or deterministic matcher run, `--skip-osv` disables network queries without changing extraction, matching, SBOM generation, or report creation. The `--include-uncertain` flag changes the meaning of the SBOM because uncertain decisions become components; it should therefore be used only for diagnostic consumers.

6.4.2 Diagnostic Commands

The system exposes narrower commands for inspecting individual stages:

Table 6.4: Core CLI commands and their expected result.

Command	Expected result
`inspect-apkAPK`	Archive, manifest, DEX, native-file, hash, and forensic evidence; no component matching.
`analyze-dexAPK`	Aggregated bytecode feature counts and parse errors.
`match-apkAPK`	Ranked reference candidates and separate identity/version decisions.
`generate-sbomAPK`	CycloneDX JSON built from accepted matcher decisions.
`scan-sbom-osvSBOM`	OSV vulnerability JSON for Maven components.
`generate-reportAPK`	Complete evidence package in JSON, Markdown, and HTML.
`validate-source-deps`	Confirmed, version-mismatch, and missing classifications against a Gradle dependency tree.

The `--coordinategroup:artifact` flag is valuable for diagnosis, but it supplies the library identity externally. Results obtained with that flag are coordinate-constrained separability tests, not ordinary unconstrained APK detection.

6.4.3 Reference-Database Workflow

A reproducible reference build follows four commands:

Listing 6.2: Reference-database construction workflow.

```
apk-sbom expand-reference-config \
  --catalog configs/popular_android_maven_catalog.json \
  --output configs/libraries.reproduction.json --latest-count 1 --limit 5
apk-sbom download-maven --config configs/libraries.reproduction.json \
  --output-root data/reference_db/artifacts_reproduction
apk-sbom convert-reference-dex \
  --artifact-root data/reference_db/artifacts_reproduction \
  --output-root data/reference_db/dex_reproduction
apk-sbom extract-reference-features \
  --dex-root data/reference_db/dex_reproduction \
  --db-path data/reference_db/reference_features_reproduction.sqlite
```

The destructive `--reset` flag is used only when creating an explicitly rebuildable database. Later batches append or replace individual coordinate/version rows. Experimental paths must remain distinct from the shared calibrated database.

6.5 Evaluation Methodology

6.5.1 Research Questions

The evaluation operationalizes the four research questions introduced in Chapter 1:

RQ1: Can the pipeline recover exact Maven versions when ground truth and adequate reference evidence are available?

RQ2: How does emitted-version quality change between clean and difficult real APK families?

RQ3: How much of the full source dependency denominator is represented by the reference database and emitted by the matcher?

RQ4: Can SBOM, vulnerability, and AI assistance preserve uncertainty and authority boundaries?

6.5.2 Evidence Layers

The datasets are not merged into one accuracy number. Their roles are defined in Table 6.5.

Table 6.5: Evaluation layers and permitted claims.

Layer	Cases	Ground truth and permitted use
Controlled fixtures	72	Exact by construction; used for expected component and version confirmation across clear, R8, repackaging, multidex, and multi-library categories.
Clean source-validated real APKs	5	Resolved source dependency trees and no emitted mismatch; strongest real emitted-version evidence.
Partial source-validated real APKs	97	Resolved source evidence with explicit mismatches or nuanced rows; used to characterize difficult families.
Source-validated no-Maven-signal controls	4	Resolved runtime dependency trees, but zero emitted Maven components; excluded from the emitted-version rate and retained as matcher-recall and emission-threshold controls.
Source-known v2 lane	106	Exact source coordinate/version slices; used only as a recognition and reference-coverage upper bound.
Limitation cases	4	Native-heavy, hidden-code, or obfuscated samples; used to evaluate warnings and scope, not Maven-version accuracy.
Report-only and experiment-only lanes	varies	Used to evaluate potential improvements without changing authoritative output.

6.5.3 Metrics and Denominators

For controlled fixtures, exact expected-version confirmation is:

$$R_{\text{controlled}} = \frac{\text{expected coordinate/version pairs recovered exactly}}{\text{expected coordinate/version pairs}}. \tag{6.2}$$

For source-validated real APKs, emitted Maven version confirmation is:

$$C_{\text{emitted}} = \frac{\text{emitted Maven rows confirmed by source}}{\text{emitted Maven rows checked against source}}. \tag{6.3}$$

Equation 6.3 measures the quality of emitted rows. It is not full dependency recall. The full-denominator exact source recall is:

$$R_{\text{source}} = \frac{\text{source dependency rows emitted with exact version}}{\text{resolved source dependency coordinate/version rows}}. \tag{6.4}$$

The evaluation also records version mismatches and missing or nuanced rows. The latter category includes emitted coordinates absent from the supplied source tree and artifact-family situations requiring separate interpretation.

6.5.4 Regression and Promotion Protocol

Every scoring, reference, shortlist, or reranking proposal follows a fail-fast protocol:

1. create an isolated implementation or SQLite database;
2. run one or more intended target cases;
3. run controlled, clean, partial, no-Maven, and boundary controls appropriate to the proposal;
4. stop when a material regression already blocks safe promotion;
5. classify the outcome as promoted, report-only, experiment-only, rejected, or blocked.

A target improvement alone is insufficient. This rule is especially important for adjacent AndroidX and Kotlin versions, where tiny score differences may reverse between applications.

6.6 Controlled Evaluation

The controlled inventory contains 72 APK fixtures and 84 expected Maven coordinate/version pairs. It covers clear version bands, R8 rename, R8 shrink, R8 optimization, shrink plus optimization, package flattening, inline-like call-site stress, multidex packaging, and multi-library combinations.

Table 6.6: Controlled evaluation by transformation category.

Controlled category	Cases	Exact expected versions
Clear fixtures	25	25/25
R8 rename	8	8/8
R8 shrink	8	8/8
R8 optimize	6	6/6
R8 shrink and optimize	6	6/6
Package flattening/repackaging	6	6/6
Inline-like call-site stress	6	6/6
Multidex and multi-library	7	19/19
Total	**72**	**84/84**

The 84/84 result establishes that the implemented pipeline can recover the expected versions across the controlled inventory. It does not establish perfect real-world behavior. Controlled fixtures contain known artifacts, known transformations, and reference coverage selected by construction. Their principal role is to guard mechanics and detect regressions.

6.7 Source-Validated Real-APK Evaluation

6.7.1 Main Emitted-Version Results

Table 6.7 reports clean and partial APKs separately before presenting the aggregate.

Table 6.7: Source-validated emitted Maven version results.

Slice	Cases	Checked	Exact	Wrong	Missing/nuance	Rate
Clean real APKs	5	47	47	0	0	100.0%
Partial real APKs	97	1,790	880	636	274	49.2%
Source-validated APKs with emitted Maven rows	**102**	**1,837**	**927**	**636**	**274**	**50.5%**

The clean set contains F-Droid, OpenTracks, Tusky, Daily Dozen, and QuickDic. F-Droid and OpenTracks provide richer component sets. Tusky, Daily Dozen, and QuickDic are clean but sparse, with only one or two emitted Maven components. Consequently, the 47/47 result is strong evidence for those emitted rows but cannot represent every real APK family.

The partial set contains modern AndroidX/Kotlin applications, minified and R8 cases, email and security applications, native-heavy frameworks, older dependency eras, and large dependency trees. Confirming 880 of 1,790 emitted versions shows that the output remains useful, while the 636 wrong-version and 274 missing/nuanced rows expose the cost of nearby version similarity, artifact-family differences, and incomplete reference coverage.

Across the 102 source-validated APKs that emitted Maven components, 927 of 1,837 emitted versions are confirmed exactly. The resulting 50.5% is therefore called the *emitted Maven version confirmation rate*. It must not be described as full dependency recall.

Four additional source-validated cases—Privacy Browser, Mastodon, monocles browser, and Blue Square Speedometer—have resolved runtime dependency trees but emit zero Maven components. They are excluded from the 927/1,837 metric because they provide no emitted rows to score; they remain separate matcher-recall and emission-threshold controls and are included in the full 106-case source denominator.

6.7.2 Full Source Dependency Denominator

The resolved source trees contain 12,082 coordinate/version rows across 106 source-tree cases. The distinction between emission quality and coverage is shown in Table 6.8.

Table 6.8: Coverage-first view of the full source dependency denominator.

Metric	Rows	Rate
Source rows whose coordinate exists in the shared DB	2,220/12,082	18.4%
Source rows whose exact coordinate/version exists in the shared DB	1,231/12,082	10.2%
Source rows emitted with the exact source version	927/12,082	7.7%

This result changes the interpretation of the 50.5% metric. Among rows the system emitted, approximately half of the versions are source-confirmed. Against the full source dependency denominator, only 7.7% are emitted exactly. Some source rows are metadata-only, platform, alias, transformed, or removed before packaging, but the denominator still reveals that both reference coverage and matcher recall remain major limitations.

6.8 Source-Known Upper-Bound Evaluation

The final v2 evaluation addresses a narrower question: if the exact source-observed coordinate/version profiles are known and the search space is restricted to those profiles, can the system recognize and report them without emitting wrong versions?

The v2 database contains 10,517 component profiles. For each source-tree APK, the evaluator constructed a slice containing exact coordinate/version rows from its resolved source dependency tree. All 106 cases completed.

Table 6.9: Source-version constrained v2 upper-bound results.

Metric	Value
Completed source-tree cases	106/106
Emitted rows checked	6,772
Exact source confirmations	6,772
Wrong-version emitted rows	0
Missing emitted rows	0
Remaining source-gap rows	5,238

The 6,772/6,772 result is a source-version constrained recognition upper bound, not unconstrained APK-only production accuracy. Source knowledge changes the candidate space and removes the ordinary version-selection problem.

The remaining 5,238 source-gap rows were not hidden. Their principal actionable groups are 2,281 matcher-recall candidates, where an exact reference exists but the system did not emit it, and 442 missing-reference candidates. The other rows include metadata/source-only artifacts, target-artifact aliases, project modules, platform/common-source boundaries, relocated coordinates, and already-covered siblings. This taxonomy shows that adding versions alone cannot solve every missing source row.

6.9 Vulnerability Demonstration

Three evidence levels are used for vulnerability reporting:

1. an exact controlled component/version can support a confirmed demonstration;
2. a source-confirmed real emitted component/version can support a source-validated finding;
3. a version mismatch, ambiguous selection, or unvalidated row remains potential or report-only.

Table 6.10: Representative vulnerability demonstrations and their authority.

Case	Observed result	Interpretation
Calibrated F-Droid	23/23 emitted Maven versions source-confirmed; 0 OSV vulnerabilities	Clean negative demonstration.
Controlled Jackson Databind	Exact `jackson-databind@2.9.10.8`; three HIGH advisories: CVE-2020-36518, CVE-2022-42003, and CVE-2022-42004	Controlled positive demonstration.
Real-APK OSV triage	Eight advisories across six component rows: one source-confirmed, six source-version-mismatch, and one report-only	Only the source-confirmed row is source-validated vulnerability evidence.

The prototype also produces human-readable reports that can be inspected by an analyst. Figures 6.1 to 6.5 show screenshots rendered from the generated report artifacts. They are included as implementation evidence: the same runs produced `report.html`, `report.json`, `report.md`, `sbom.cdx.json`, and `osv-vulnerabilities.json`. The figures therefore demonstrate the working end-to-end output path rather than introducing a new evaluation denominator.



Figure 6.1: Generated report screenshot for the clean F-Droid lane: APK metadata, DEX/native extraction summary, 23 detected Maven components, and zero OSV vulnerability findings.



APK SBOM Recovery Report

page 2/5

Clean real APK lane: F-Droid, 23 Maven components, OSV 0

`org.fdroid.fdroid.permission.UPDATE_REPOS`

No extraction indicators were raised.

Obfuscation Risk

- Level: low
- Score: 0.20
- Reason: At least half of emitted components are low-confidence version candidates.

Detected Components

Component	Version	Candidate reasons	Identity confidence	Version confidence	Match type	Vulnerabilities
`androidx.gridlayout:gridlayout`	1.0.0	package, class, method, opcode, graph, literals, version-unique	confirmed/high (0.983)	exact/high (0.972)	hybrid-exact	0
`androidx.localbroadcastmanager:localbroadcastmanager`	1.0.0	package, class, method, opcode, graph, literals	confirmed/high (0.962)	candidate/low (0.871)	hybrid-candidate	0
`androidx.swiperefreshlayout:swiperefreshlayout`	1.0.0	package, class, method, opcode, graph, literals, version-unique	confirmed/high (0.919)	likely/medium (0.870)	hybrid-likely	0
`androidx.concurrent:concurrent-futures`	1.1.0	package, class, method, opcode, graph, literals	confirmed/high (0.937)	candidate/low (0.843)	hybrid-candidate	0
`androidx.recyclerview:recyclerview`	1.4.0	package, class, method, opcode, graph, literals, version-unique	likely/medium (0.835)	likely/medium (0.832)	hybrid-likely	0
`androidx.appcompat:appcompat`	1.7.0	package, class, method, opcode, graph, literals, version-unique	likely/medium (0.802)	likely/medium (0.815)	hybrid-likely	0
`androidx.lifecycle:lifecycle-viewmodel-android`	2.8.7	package, class, method, opcode, graph, literals, version-unique, artifact-family	likely/medium (0.822)	likely/medium (0.808)	hybrid-likely	0
`com.squareup.okhttp3:okhttp`	4.12.0	package, class, method, opcode, graph, literals, version-unique	likely/medium (0.802)	likely/medium (0.807)	hybrid-likely	0
`androidx.exifinterface:exifinterface`	1.3.6	package, class, method, opcode, graph, literals, version-unique	likely/medium (0.813)	likely/medium (0.792)	hybrid-likely	0
`io.ktor:ktor-client-okhttp-jvm`	2.3.12	package, class, method, opcode, graph, literals	likely/medium (0.877)	candidate/low (0.788)	hybrid-candidate	0
`androidx.work:work-runtime`	2.10.0	package, class, method, opcode, graph, literals, version-unique	likely/medium (0.757)	likely/medium (0.752)	hybrid-likely	0
`androidx.fragment:fragment`	1.6.1	package, class, method, opcode, graph, literals, version-unique	likely/medium (0.763)	candidate/low (0.749)	hybrid-candidate	0
`androidx.compose.ui:ui-android`	1.7.8	package, class, method, opcode, graph, literals, version-unique	likely/medium (0.744)	candidate/low (0.727)	hybrid-candidate	0
`org.jetbrains.kotlinx:kotlinx-coroutines-android`	1.7.3	package, class, method, opcode, graph, literals, version-unique				

Figure 6.2: Generated report screenshot for the clean F-Droid lane: component evidence rows and confidence-aware version decisions continue without confirmed vulnerability findings.

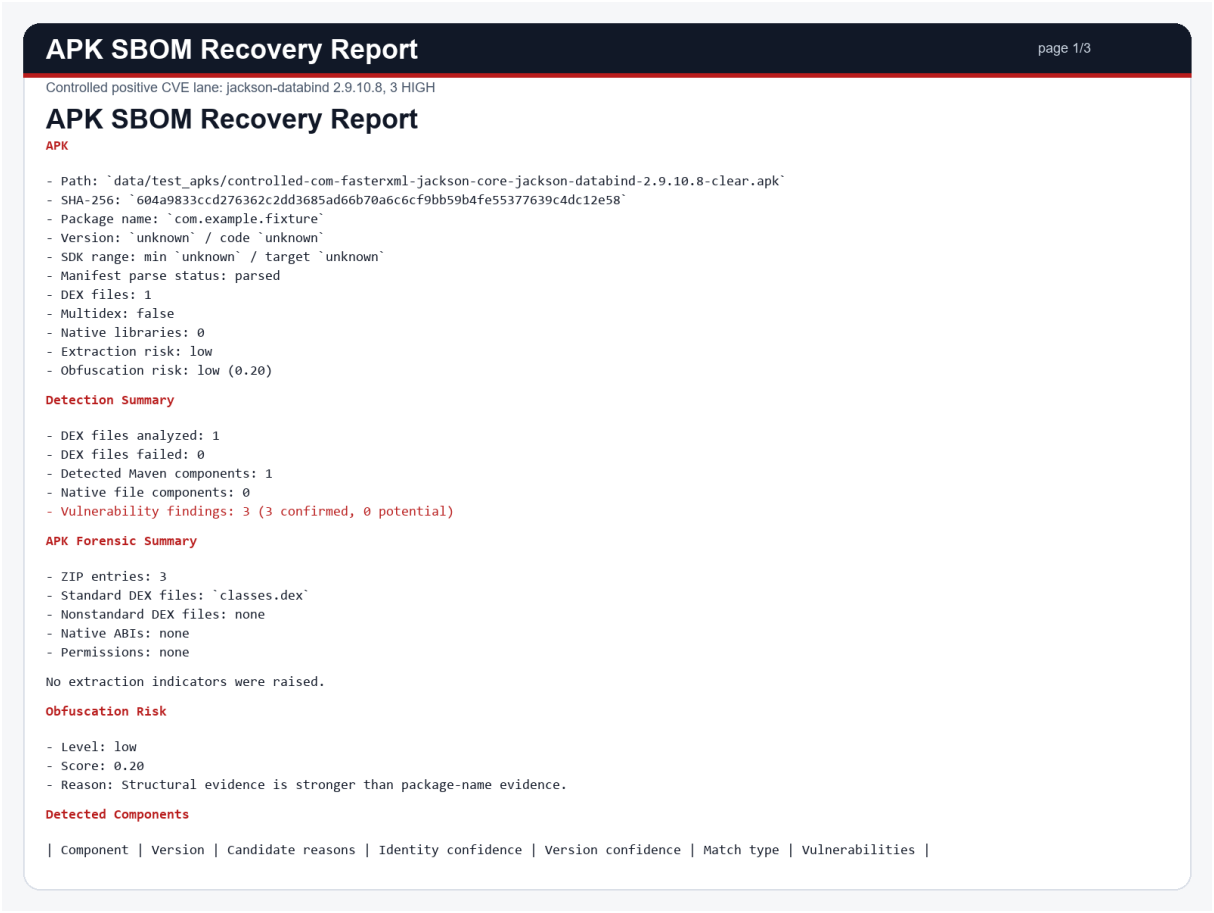


Figure 6.3: Generated report screenshot for the controlled vulnerable lane: the fixture APK is processed end to end and the detection summary records one Maven component with three vulnerability findings.



APK SBOM Recovery Report

page 2/3

Controlled positive CVE lane: jackson-databind 2.9.10.8, 3 HIGH

|---|---:|---|---|---|---|---|
| `com.fasterxml.jackson.core:jackson-databind` | 2.9.10.8 | package, class, method, opcode, graph, literals, version-unique | confirmed/high (1.000) | exact/high (1.000) | hybrid-exact | 3 confirmed |

Native Libraries

No native `.so` files were found in the APK.

Version Decisions

Component	Selected version	Identity	Version	Score	Gap	Alternatives
`com.fasterxml.jackson.core:jackson-databind`	2.9.10.8	confirmed/high (1.000)	exact/high (1.000)	1.000	0.183	2.12.7.1 (0.817), 2.13.5 (0.802), 2.15.2 (0.755)
`com.google.code.gson:gson`	2.8.5	uncertain/uncertain (0.146)	uncertain/uncertain (0.161)	0.161	0.015	2.8.6 (0.146), 2.8.9 (0.146)
`com.squareup.retrofit2:retrofit`	2.8.1	uncertain/uncertain (0.133)	uncertain/uncertain (0.137)	0.137	0.000	2.9.0 (0.137)
`com.squareup.okhttp3:okhttp`	4.8.0	uncertain/uncertain (0.125)	uncertain/uncertain (0.130)	0.130	0.000	4.9.0 (0.130), 4.9.1 (0.130), 4.12.0 (0.130)
`com.squareup.okio:okio-jvm`	3.7.0	uncertain/uncertain (0.124)	uncertain/uncertain (0.127)	0.127	0.000	3.6.0 (0.127), 3.0.0 (0.125)
`commons-codec:commons-codec`	1.15	uncertain/uncertain (0.119)	uncertain/uncertain (0.125)	0.125	0.001	1.16.1 (0.124), 1.16.0 (0.124)
`io.ktor:ktor-client-core-jvm`	2.3.12	uncertain/uncertain (0.119)	uncertain/uncertain (0.124)	0.124	0.124	none
`com.squareup.okio:okio`	2.9.0	uncertain/uncertain (0.119)	uncertain/uncertain (0.123)	0.123	0.000	2.8.0 (0.122)
`io.ktor:ktor-http-jvm`	2.3.12	uncertain/uncertain (0.115)	uncertain/uncertain (0.119)	0.119	0.119	none
`io.ktor:ktor-utils-jvm`	2.3.12	uncertain/uncertain (0.114)	uncertain/uncertain (0.119)	0.119	0.119	none
`io.ktor:ktor-io-jvm`	2.3.12	uncertain/uncertain (0.112)	uncertain/uncertain (0.117)	0.117	0.117	none
`io.ktor:ktor-client-okhttp-jvm`	2.3.12	uncertain/uncertain (0.098)	uncertain/uncertain (0.107)	0.107	0.107	none

Vulnerabilities

Component	Version	Finding confidence	Severity	Advisory	Aliases	Summary
`com.fasterxml.jackson.core:jackson-databind`	2.9.10.8	confirmed	HIGH	`GHSA-57j2-w4cx-62h2`	CVE-2020-36518	Deeply nested json in jackson-databind
`com.fasterxml.jackson.core:jackson-databind`	2.9.10.8	confirmed	HIGH	`GHSA-jjjh-jjxp-wpff`	CVE-2022-42003	Uncontrolled Resource Consumption in Jackson-databind
`com.fasterxml.jackson.core:jackson-databind`	2.9.10.8	confirmed	HIGH	`GHSA-rgv9-q543-rqg4`	CVE-2022-42004	Uncontrolled Resource Consumption in FasterXML jackson-databind

Figure 6.4: Generated report screenshot for the controlled vulnerable lane: Jackson Databind version 2.9.10.8 is selected with exact high-confidence evidence, then OSV returns CVE-2020-36518, CVE-2022-42003, and CVE-2022-42004 as confirmed HIGH findings for the demonstration.



Figure 6.5: Generated report screenshot for the controlled vulnerable lane: the report lists the persisted output artifacts, including match details, CycloneDX SBOM, JSON report, Markdown report, HTML report, and OSV vulnerability JSON.

The result demonstrates why vulnerability scanning cannot be separated from component attribution. OSV may correctly report advisories for the selected version while the selected version itself disagrees with source evidence. Applying source-validation labels therefore reduces false certainty without discarding potentially useful triage information.

6.10 AI and R8 Evaluation

The AI layer was evaluated after the deterministic pipeline and evidence exports were available. Its purpose is risk triage and investigation routing, not direct component authority.

Table 6.11: Machine-learning and Transformer results with authority decisions.

Model or gate	Dataset	Result	Decision
Random Forest assistant	20,523 project evidence rows, grouped by case	Macro-F1 0.708; accuracy 0.690	Integrated as report-only assistance.
Structured-token Transformer	20,523 rows	Macro-F1 0.977	Research-status result; shortcut/context risk prevents authority.
MVP-safe Transformer	20,523 rows with source/version shortcuts removed	Macro-F1 0.882; accuracy 0.907	Candidate for report-only risk triage; production decision wiring disabled.
Raw-opcode Transformer	1,680 method sequences from 14 APKs	Macro-F1 0.110 versus opcode-count baseline 0.128	Negative result; research-only and not promotable.
Strict APK-only promotion matrix	Five guarded profiles	0 eligible production candidates after controls	Matcher, SBOM, and CVE authority remain blocked.

The Random Forest split is grouped by case identifier to reduce direct leakage between rows from the same APK case. Macro-F1 is reported because the six action labels are imbalanced. Feature-importance and ablation results are retained to expose dependence on confidence, score, and context features.

The full structured-token Transformer performs strongly, but its input contains project-context signals that can simplify classification. The MVP-safe profile removes source versions, raw candidate versions, source rank, source-validation roles, and diagnostic shortcuts. Its lower but still useful result supports only report-level triage. The raw-opcode experiment is the most direct learned bytecode lane, and its failure to beat a simple baseline is preserved as negative evidence.

A report-only smoke test confirms the authority boundary: enabling AI sections leaves detected components, selected versions, semantically normalized SBOM content, and vulnerability labels unchanged.

6.11 Failed and Non-Promoted Experiments

Negative experiments are part of the contribution because they show which apparently promising changes were rejected by target/control evidence.

Table 6.12: Representative failed, blocked, or non-promoted experiments.

Experiment	Observed result	Decision and lesson
Global opcode tri-gram weight	Regressed established controls without a confirmed-version gain	Disabled by default; a useful signal is not automatically a safe global weight.
Global API-reference weight	Changed no guarded source-validation metric	Disabled by default because no measured benefit justified extra authority.
Global multi-scale R8 blend	Found four target improvements but also six controlled, 30 real source-validated, and eight clean-real regressions	Rejected globally; retained as diagnostic features and narrower report-only experiments.
Version-band reranking	565 wrong-version rows appear fixable under source-known constraints; APK-only safe fixes remained 0	Not promoted; source-known opportunity is not APK-only evidence.
Deep candidate expansion	Rescoring 134 Activity versions selected an older, incorrect alpha release	Not promoted; a larger candidate set requires stronger version-specific evidence.
Priority reference database	Exact source-version coverage reached 79.9%, but I2P output expanded from 18 rows to 111 in the broad preflight and introduced 14 source-missing rows	Fail-fast rejection for production-style use; database remains evaluation-only.
Fragment 1.6.2	Ten affected rows selected 1.6.2 and sampled controls stayed clean, but 1.6.1/1.6.2 retained very small candidate-level score gaps without matched version-unique APK evidence	Experiment-only; no shared DB promotion without version-unique APK evidence.
Raw-opcode Transformer	Macro-F1 0.110, below baseline 0.128	Honest negative result; larger and richer bytecode context is required.

These outcomes support the fail-fast policy. The project favors a stable, bounded default over a larger but poorly controlled result surface.

6.12 Comparison with Existing Work

Chapter 4 reviews Android library detectors whose objectives, datasets, and metrics differ substantially. Consequently, this thesis does not rank the MVP against reported percentages from incompatible benchmarks. Instead, Table 6.13 compares capabilities that can be stated consistently.

Table 6.13: Dimension-based positioning relative to the reviewed detector families.

Dimension	Common emphasis in prior work	Implemented MVP position
Primary objective	Library presence, similarity, or version detection	End-to-end evidence recovery, SBOM emission, validation, and vulnerability interpretation.
Reference representation	Class, package, hash, opcode, API, graph, or learned profiles	Hybrid DEX profile with name, opcode, literal, API, dependency, and guarded contextual signals.
Version uncertainty	Often one top candidate or threshold decision	Separate identity/version confidence, alternatives, and ambiguous version sets.
Obfuscation handling	Technique-specific robustness evaluations	Controlled R8 categories, real minified cases, and explicit report-only diagnostics.
Standard SBOM output	Frequently absent from detector prototypes	CycloneDX 1.5 with PURLs, evidence properties, and native file hashes.
Vulnerability mapping	Often a downstream security use case	OSV integration with source-aware confirmed, potential, and report-only labels.
Negative experiments	Usually secondary to the promoted method	Preserved in a decision ledger and used to block unsafe promotion.
Full denominator transparency	Often benchmark-specific	Emitted-version quality, full source dependency coverage, and source-known upper bound reported separately.

The main engineering difference is therefore integration and evidence governance, not a claim that one similarity function universally outperforms all prior detectors.

6.13 Discussion

6.13.1 Answers to the Research Questions

RQ1: Can exact versions be recovered? Yes, under bounded conditions. The system confirms 84/84 controlled expected versions and 47/47 emitted versions in the clean real slice. The partial and full-denominator results show that this capability does not generalize uniformly across all source dependencies.

RQ2: How does behavior change across APK families? Clean, well-covered APKs support exact emission. Modern AndroidX/Kotlin, older dependency eras, R8/minified, native-heavy, and large-tree applications produce more adjacent-version mismatches and missing/nuanced rows. The clean/partial split is therefore an essential result, not merely a reporting choice.

RQ3: What limits full recovery? Both reference coverage and matcher recall. The shared database covers the exact source version for 10.2% of the 12,082 source rows, while end-to-end exact recall is 7.7%. In the larger source-known lane, the remaining actionable gaps divide into missing-reference and matcher-recall candidates.

RQ4: Can downstream output preserve uncertainty? Yes. The SBOM stores confidence and ambiguity properties, vulnerability findings can be relabeled by source validation, native files remain hash-backed file components, and AI assistance is report-only.

6.13.2 Strengths

The system's principal strengths are:

- an operational path from compiled APK to standard SBOM and vulnerability report;
- a common DEX representation for APK and reference artifacts;
- separate library-identity and exact-version decisions;
- source-backed validation and a visible full dependency denominator;
- fail-fast target/control promotion rules;
- explicit native, hidden-code, and AI authority boundaries;
- reproducible machine-readable evidence and human-readable reports.

6.13.3 Error Categories

The current errors are not one homogeneous matcher failure. They include:

- adjacent versions with nearly equivalent compiled evidence;
- public, Android-target, JVM, KTX, and sibling artifact families;
- source versions absent from the shared reference database;
- exact profiles present but not surfaced by candidate generation;

- metadata-only or platform dependencies that do not provide ordinary DEX bytecode;
- dependencies removed or transformed during building and shrinking;
- false-positive extra emissions encouraged by broad package overlap;
- native, dynamic, hidden, or protected code outside the current JVM attribution model.

6.13.4 Practical Value of Partial Results

A partial SBOM can still identify high-confidence components, document native files, provide hashes, reveal version ambiguity, and guide vulnerability investigation. Its practical value depends on preserving uncertainty. A smaller evidence-qualified SBOM is safer than a larger list of precise-looking but unsupported coordinates and versions.

6.13.5 Runtime and Scalability

SQLite lookup and APK DEX analysis are practical for focused and calibrated databases. Broad evaluation databases introduce a different problem: candidate generation must remain precise enough to avoid both excessive runtime and false emissions. The priority-database preflight demonstrated that package-overlap shortlists can recover intended rows while severely inflating the output. A safer large-database deployment therefore requires a stronger APK-only shortlist or candidate-emission gate rather than only additional hardware or a larger database.

6.14 Threats to Validity

6.14.1 Internal Validity

Implementation defects, stale generated tables, or inconsistent configuration could affect the results. These risks are reduced through automated tests, generated thesis tables, explicit artifact paths, and a frozen evidence snapshot. However, not every long-running APK evaluation is rerun for each writing session.

6.14.2 Construct Validity

The emitted Maven version confirmation rate measures correctness among emitted rows, not complete dependency recovery. Presenting it as full recall would be a construct error. This chapter therefore reports the 12,082-row source denominator separately.

Source dependency trees are strong external evidence, but they represent resolved build dependencies rather than a perfect bytecode manifest. Gradle may include BOMs, metadata, compile-time modules, transformed artifacts, and dependencies removed by shrinking.

6.14.3 External Validity

The source-tree dataset contains 106 source-validated APKs with diverse categories: 102 emit Maven rows and four are no-Maven-signal controls. It is not a random sample of all Android applications. Open-source F-Droid and GitHub projects are overrepresented because their source and dependency evidence can be audited. Commercial protectors, games, Flutter, Unity, and additional dynamically loaded applications require broader future datasets.

6.14.4 Reference-Database Bias

The selected reference database directly constrains what can be recognized. A missing version can force a nearby candidate, while a broad database can increase both runtime and false-positive pressure. The source-known v2 lane measures a useful upper bound but cannot estimate unconstrained production accuracy.

6.14.5 AI Validity

The AI datasets are derived from project evidence and are not an independent external benchmark. The high full structured-token Transformer result may benefit from contextual shortcuts. Grouped splits and the MVP-safe profile reduce this risk, while the negative raw-opcode result demonstrates that a more direct representation is not yet competitive. AI conclusions are therefore limited to report-only assistance.

6.15 Conclusion

The implemented MVP provides a complete static APK-to-SBOM workflow: extraction, DEX analysis, Maven reference matching, confidence-aware version decisions, CycloneDX generation, OSV lookup, source validation, and multi-format reporting.

The evaluation establishes three bounded conclusions. First, the mechanics are reliable on the controlled inventory and on the clean real emitted rows. Second, difficult real APKs remain useful but expose substantial version and coverage limitations: 927 of 1,837 emitted Maven versions are confirmed across the 102 source-validated APKs that emit Maven rows, while four additional source-validated cases remain no-Maven-signal controls. Exact end-to-end recall against the full source denominator is 927 of 12,082. Third, the source-known v2 and AI lanes are valuable diagnostic upper bounds and assistants, but their authority remains separate from unconstrained APK-only SBOM decisions.

The final chapter synthesizes these contributions, limitations, and future directions without converting experimental evidence into stronger claims than the evaluation supports.

Part V

General Conclusion

Chapter 7

General Conclusion and Perspectives

7.1 General Conclusion

This thesis investigated whether a useful Software Bill of Materials can be recovered when the available Android artifact is a compiled APK rather than a source repository or reproducible build environment. The problem is difficult because dependency archives are merged into DEX, source-level coordinates and versions are not preserved as a complete manifest, and R8 may rename, remove, inline, merge, or reorganize the evidence needed for component attribution. Vulnerability analysis adds a further requirement: an advisory mapping is only as trustworthy as the component identity and exact-version evidence on which it depends.

The project addressed this problem through a static, evidence-gated APK-to-SBOM pipeline. The implemented MVP:

- inspects APK archives, manifests, DEX files, resource and asset inventories, native libraries, and extraction-risk indicators;
- extracts packages, classes, methods, opcodes, literals, API references, dependencies, and structural features from all successfully parsed DEX files;
- constructs versioned Maven reference profiles by acquiring JAR/AAR releases and converting their bytecode to a comparable DEX representation;
- ranks component and version candidates through explainable multi-feature evidence;
- separates component-identity confidence from exact-version confidence and retains ambiguous alternatives;
- emits accepted evidence as CycloneDX JSON with Package URLs;
- queries OSV while preserving confirmed, potential, and report-only vulnerability interpretations;
- validates emitted Maven rows against source dependency trees when auditable source evidence is available; and
- produces JSON, Markdown, and HTML reports that expose evidence, decisions, uncertainty, and limitations.

The contribution is therefore broader than a similarity score. It combines binary evidence extraction, reference construction, component attribution, standard serialization, vulnerability interpretation, source validation, and experiment governance in one reproducible workflow. The project also records why apparently promising changes were not promoted. Global opcode weighting, broad API weighting, recency-based version selection, global multi-scale fusion, broad priority-database shortlisting, and raw-opcode deep learning were rejected, disabled, or isolated when their controls did not justify production authority.

7.1.1 Evaluation Synthesis

The controlled evaluation confirms that the implemented mechanics work when the expected components and reference evidence are known: 84 of 84 expected Maven versions were recovered across 72 controlled fixtures. The clean source-validated real slice confirms 47 of 47 emitted Maven versions. These results demonstrate that static APK evidence can support exact component/version claims under favorable conditions.

The broader real-APK evaluation gives a more realistic boundary. Across the 102 source-validated APKs that emitted Maven components, 927 of 1,837 emitted versions were confirmed, corresponding to a 50.5% *emitted Maven version confirmation rate*. This metric describes correctness among emitted Maven rows; it is not full dependency recall. Four additional source-validated APKs emitted no Maven components and remain separate matcher-recall and emission-threshold controls.

Against the complete source dependency denominator, the system emitted 927 exact coordinate/version rows from 12,082 resolved source rows, or 7.7%. The shared reference database contained the exact source version for 1,231 of those rows, or 10.2%. The difference between controlled success, emitted-row confirmation, reference coverage, and full-denominator recall is one of the principal findings of the thesis. A matcher cannot identify a missing reference, and a large reference catalog is not sufficient if candidate generation introduces false emissions.

The source-known v2 lane confirmed 6,772 of 6,772 emitted rows when each APK's candidate space was constrained to exact source-known coordinate/version references. This result demonstrates recognition under a favorable upper-bound condition. It does not measure unconstrained APK-only accuracy and does not remove the remaining 5,238 source-gap rows.

The vulnerability demonstrations confirm the same authority rule. Exact and source-consistent package/version evidence can support a confirmed advisory mapping. A source-version mismatch must remain potential even when the external vulnerability service correctly returns an advisory for the selected version.

The ML/DL work supports report triage rather than component authority. The MVP-safe structured-token Transformer provides promising report-only triage evidence, while its production decision wiring remains disabled. The raw-opcode Transformer failed to beat its simpler logistic-regression baseline. The integration smoke tests show that AI sections can be visible without changing selected components, versions, CycloneDX content, or vulnerability output.

7.1.2 Practical Significance

The MVP is useful when interpreted as an evidence-recovery and audit tool, not as a replacement for a trusted build-generated SBOM. It can recover strong component claims, reveal version ambiguity, produce interoperable output, guide vulnerability investigation, and identify unsupported regions such as native-heavy or hidden-code applications.

Its practical value comes from making partial knowledge explicit. A report can distinguish:

- a source-confirmed exact Maven component;
- a strong component identity with an ambiguous version band;
- a report-only candidate requiring analyst review;
- a missing-reference or matcher-recall gap;
- a hash-backed native file observation; and
- a protected or dynamically loaded boundary outside static coverage.

This distinction is safer than emitting one precise-looking dependency list in which supported and unsupported claims are indistinguishable.

7.2 Answers to the Research Questions

Table 7.1 summarizes the answers established by the evaluation. The wording preserves the evidence classes and denominators defined in Chapter 6.

Table 7.1: Answers to the research questions and their evidence boundaries.

RQ	Answer	Established boundary
RQ1	Exact Maven versions can be recovered under bounded conditions.	Controlled fixtures recover 84/84 expected versions, and the clean real slice confirms 47/47 emitted versions. Partial and full-denominator evidence shows that this capability depends on reference availability, surviving bytecode, and version separability.
RQ2	Emitted-version quality varies substantially across APK families.	Clean, well-covered APKs support strong exact claims. Partial, minified, AndroidX/Kotlin-era, and large-tree cases expose more wrong-version, missing, or ambiguous rows. Native-heavy and hidden-code cases expose unsupported analysis boundaries and remain separate from Maven-version accuracy.
RQ3	Reference coverage and matcher recall both constrain complete recovery.	The shared database exactly covers 1,231/12,082 source rows, while the full pipeline emits 927/12,082 exact rows. The source-known v2 lane demonstrates that broader exact references enable recognition, but its constrained candidate space is an upper bound rather than production accuracy.
RQ4	Downstream outputs can preserve uncertainty and authority boundaries.	CycloneDX components carry confidence and ambiguity evidence; OSV findings inherit component/source-validation status; native files remain hash-backed observations; and AI assistance is report-only and cannot alter protected outputs.

The combined answer is conditional rather than absolute. Static APK analysis can produce a useful and auditable SBOM, but trustworthy exact-version recovery requires adequate references, separable compiled evidence, cautious candidate generation, and downstream systems that preserve uncertainty.

7.3 Limitations

7.3.1 Reference Coverage

The reference database defines the universe of versions the matcher can recognize. If the true coordinate or version is absent, the system may select a neighboring release or abstain; it cannot recover a version it has never modeled. The shared database was kept calibrated and deliberately smaller than the evaluation-only catalogs because uncontrolled expansion increased false-positive pressure.

The priority database demonstrated this trade-off clearly. It substantially increased exact source-version coverage, but package-overlap shortlisting caused many extra and mismatched emissions. Reference growth must therefore be coupled with stronger APK-only candidate gating.

7.3.2 Matcher Recall and Candidate Generation

Some exact references exist but are not surfaced by the current shortlist or do not receive enough evidence to cross the emission policy. Package and class overlap are efficient on clear applications, yet they become fragile after renaming, repackaging, shrinking, and partial inclusion.

The current matcher combines several evidence families, but its production weights remain conservative. Advanced API, opcode n-gram, POM coherence, and multi-scale features are available diagnostically; global versions of these signals either produced no measured benefit or regressed controls.

7.3.3 Adjacent-Version and Artifact-Family Ambiguity

Nearby releases often share most of their packages, classes, methods, literals, and dependency structure. R8 can remove the small differences that distinguish them. In these cases, selecting the newest release is not a valid inference.

Android target variants add another ambiguity layer. Public, JVM, Android, KTX, runtime, and multiplatform artifacts may share bytecode while source trees and published coordinates describe different abstraction levels. The MVP contains several scoped family rules, but no universal artifact-family resolver.

7.3.4 Native, Hidden, and Dynamic Code

The system inventories native `.so` files using paths, hashes, sizes, and ABI information. It does not infer general native package names and versions. Native provenance requires a separate reference and binary-analysis pipeline that can handle stripping, static linking, compiler variation, and architecture-specific builds.

Static analysis also cannot inventory code that is absent from the APK, downloaded at runtime, encrypted beyond the implemented process, or hidden inside a commercial protector. The extractor can report nonstandard payloads and elevated risk, but these indicators do not justify invented Maven components.

7.3.5 Runtime, Scalability, and Operational Coverage

Focused SQLite databases and ordinary APKs are practical for local analysis. Very broad databases increase both comparison cost and false candidate pressure. Scalability is therefore not only a hardware problem; it depends on candidate retrieval quality, indexing, family-aware filtering, and emission policy.

The source-tree dataset contains 106 source-validated APKs: 102 emit Maven components and four are no-Maven-signal controls. It is diverse but not a random sample of all Android applications. Open-source applications are overrepresented because their source and dependency trees can be audited. Commercial games, proprietary protectors, Flutter, Unity, additional native-heavy applications, and more dynamic-loading cases require separate evaluation.

7.3.6 Source Validation

Resolved Gradle dependency trees provide strong external evidence, but they are not perfect bytecode manifests. They may include platform, metadata, bill-of-materials, build-time, optional, transformed, or unused dependencies. Shrinking may remove declared code, while vendored code can appear in the binary without a direct source dependency row.

Source validation is also expensive for legacy projects. Obsolete Gradle versions, unavailable repositories, removed plug-ins, included builds, and network dependencies can block reproduction. Isolated validation copies were used where necessary, and unresolved cases were not promoted into clean accuracy.

7.3.7 Vulnerability Interpretation

The system maps accepted Maven package/version evidence to OSV advisories. It does not perform exploitability analysis, call-path reachability, runtime configuration assessment, or remediation verification. A confirmed package/version advisory means the known component version falls within an affected range; it does not mean that an exploit is reachable in the application.

7.3.8 Artificial-Intelligence Authority

The AI datasets are project-derived and not an independent external benchmark. The strongest full structured-token result contains source and diagnostic shortcuts. The safer source-free result remains suitable only for report triage, and the direct raw-opcode experiment underperformed its baseline.

AI may therefore annotate risk, wording, and investigation priority. It may not select a component, confirm a version, modify a CycloneDX row, or change a vulnerability label. A future authority change would require external data, strict APK-only inputs, calibration, and target/control evidence.

7.4 Future Perspectives

Future work should extend the evidence base in an order that reduces false claims before increasing output volume. Table 7.2 summarizes the recommended priorities and their promotion conditions.

Table 7.2: Prioritized future work and the evidence required for promotion.

Priority	Direction	Required evidence
P1	Safer large-catalog candidate retrieval	Improve identity recall and reference coverage without reproducing priority-database false emissions; pass clean, partial, controlled, and same-coordinate controls.
P1	Exact-version separability and calibrated abstention	Use version-distinctive evidence and calibrated margins; demonstrate fewer wrong versions without hiding unresolved cases.
P2	Broader source-observed reference coverage	Add bytecode-bearing source-used releases through reproducible acquisition and isolated promotion gates; keep metadata-only and unsupported artifacts explicit.
P2	Scoped multi-scale R8 evidence	Evaluate class, call-chain, basic-block, field-operation, and literal-context evidence per risk profile; prohibit global promotion when any protected control regresses.
P3	Native component provenance	Build a separate native reference corpus and evaluation protocol for ABI-specific shared libraries, stripping, and static linking.
P3	Dynamic and protected-code observation	Combine static risk indicators with sandboxed runtime collection, integrity checking, and explicit provenance for loaded payloads.
P4	External benchmarks and reproducibility	Evaluate on independently curated APKs, additional ecosystems and build eras, and compatible public benchmarks with published denominator definitions.
P4	Calibrated report-only learning	Train on external, independently reviewed labels and APK-only inputs; require calibration and strict output equivalence before considering any authority expansion.

7.4.1 Safer Candidate Retrieval

The highest-value near-term improvement is a stronger APK-only shortlist for large reference catalogs. Possible mechanisms include inverted indexes over rare classes and methods, coordinate-family retrieval, approximate nearest neighbor search over structural summaries, and multi-stage coarse-to-fine verification. The critical gate is not shortlist recall alone. A successful method must recover intended candidates while preventing the extra emissions observed in broad package-overlap experiments.

7.4.2 Calibrated Evidence and Version Abstention

Current scores rank candidates but are not formal probabilities. Future work can calibrate identity confidence, exact-version confidence, and abstention thresholds using held-out source-validated cases. Calibration should be coordinate- and family-aware because score distributions differ across libraries, build eras, and optimization profiles.

Version-set prediction may be more honest than single-version selection for feature-equivalent releases. A future calibrated set-valued method could produce candidate sets with explicitly evaluated coverage, provided its assumptions and validation population are stated.

7.4.3 Reference and Provenance Knowledge Graph

The current SQLite schema stores component features and declared Maven dependencies. A provenance-aware knowledge graph could connect:

- public coordinates, target variants, aliases, and relocated artifacts;
- release chronology and version families;
- Maven dependency relationships;
- source-observed usage;
- binary fingerprints and evidence occurrences;
- vulnerability affected ranges; and
- the decisions and controls that justified or rejected a claim.

Such a graph could improve artifact-family reasoning and explain why a candidate was considered. It must remain secondary to observed APK evidence; graph coherence should not become a new form of metadata-only certainty.

7.4.4 Native and Dynamic Components

Native recovery should be developed as a separate module with its own features, references, and metrics. Useful evidence may include exported and imported symbols, function fingerprints, strings, section structure, build identifiers, compiler signatures, and architecture-normalized code features.

Dynamic-code work should record when and where a payload was obtained, its hash, signature, loading mechanism, and execution context. Runtime collection must not be merged silently with the original APK inventory; provenance should distinguish packaged, unpacked, and downloaded components.

7.4.5 Larger and More Independent Evaluation

Future evaluation should increase application diversity by family rather than by convenience. Priority categories include:

- modern AndroidX and Kotlin multiplatform applications;

- legacy Maven and JCenter-era builds;
- commercial or protected applications for which analysis is legally authorized;
- games and cross-platform frameworks;
- native-heavy and dynamically extensible applications; and
- multiple releases of the same application to measure temporal drift.

An independent corpus should separate dataset construction, label review, and tool development. It should publish both emitted-row quality and full source or binary-ground-truth denominators so that coverage and matcher precision are not collapsed.

7.4.6 AI as a Guarded Research Direction

Learned representations remain promising for complex R8 evidence, but the next step should not be a larger model on the same shortcuts. A stronger AI study requires:

- more independent APK cases per class;
- labels reviewed independently of the features used for prediction;
- APK-only inputs available at deployment time;
- hierarchical aggregation from methods to classes, libraries, and applications;
- calibration and out-of-distribution detection;
- simple and strong baselines; and
- a proof that authoritative outputs remain unchanged until promotion gates pass.

The appropriate near-term role remains analyst assistance: identify unusual evidence patterns, prioritize review, and recommend which deterministic diagnostic should run next.

7.5 Closing Remarks

This project demonstrates that APK-only SBOM recovery is possible, but not uniformly complete. Strong exact-version claims can be recovered when reference coverage and surviving evidence are favorable. Difficult applications expose missing references, shortlist failures, adjacent-version ties, artifact-family ambiguity, native code, hidden payloads, and optimization-induced evidence loss.

The central engineering lesson is that trustworthy recovery depends as much on evidence governance as on feature design. A system must know not only how to select a candidate, but also when to abstain, how to preserve alternatives, how to propagate uncertainty to vulnerabilities, and how to reject an improvement that harms controls.

The resulting MVP does not claim complete reconstruction of an Android build. It provides a reproducible path from compiled evidence to an auditable CycloneDX inventory, with explicit denominators, provenance, ambiguity, and scope boundaries. In software

supply-chain analysis, transparent partial knowledge is more useful than unsupported certainty.

Appendix A

Reproducibility and Evidence Index

This appendix records the environment, command sequences, evidence classes, and generated artifacts needed to reproduce or audit the principal results of this thesis. It complements the conceptual and implementation descriptions in Chapters 5 and 6. The commands are written for the repository layout used during the final evidence freeze of 20 June 2026. The corresponding Git base revision was `2983d54`; thesis sources and generated documentation created after that revision were present as an uncommitted working-tree layer.

A.1 Build and Runtime Environment

A.1.1 Recommended Docker Environment

Docker Compose is the preferred execution path because it standardizes the Python, Android, Java, and Gradle toolchains used by the project. The main environment is summarized in Table A.1. Exact bit-for-bit rebuilds still depend on mutable package repositories and unpinned upstream scanner installers, so the dated generated artifacts remain the authoritative evidence snapshot.

Table A.1: Reproduction environment used by the project.

Element	Configuration
Python	Python 3.12 or newer; the frozen project environment used Python 3.12.0.
Core libraries	Androguard 4.1.3, NetworkX 3.4.2, Requests 2.32.3, Packaging 24.2, Pydantic 2.10.6, and Rich 13.9.4.
Android conversion	Android command-line tools and D8 from Build Tools 36.0.0; legacy Build Tools 29.0.3 are also installed.
Build compatibility	Default JDK plus JDK 11, Gradle 9.5.0, and compatibility installations for Gradle 8.14.3, 8.13, 6.1.1, 5.1.1, and 4.6.
Feature storage	SQLite reference databases containing versioned Maven profiles and extracted evidence.
Output standards	CycloneDX JSON with Package URLs; OSV is used for online vulnerability lookup.
Optional AI	Scikit-learn 1.5–1.x and PyTorch 2.4–2.x; these dependencies support report-only experiments and are not required for deterministic matching.

From the repository root, the container and test suite can be verified with:

Listing A.1: Docker build and verification sequence.

```
docker compose build
docker compose run --rm apk-sbom python scripts/check_toolchain.py
docker compose run --rm apk-sbom python -m apk_sbom.cli --help
docker compose run --rm apk-sbom pytest -q
```

The final local regression run reported 275 passing tests. This count is a dated project-state observation, not a guarantee that a future checkout will retain the same number of tests.

A.1.2 Host-Python Alternative

The deterministic Python commands can also run in a Python 3.12 virtual environment:

Listing A.2: Host-Python setup and test sequence.

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -e ".[dev]"
python -m apk_sbom.cli --help
pytest -q
```

Reference conversion on the host additionally requires a working D8/Android SDK installation. The optional extras `.[ml]` and `.[dl]` are needed only to reproduce the machine-learning experiments discussed in Chapter 3.

A.2 Core APK-to-SBOM Command Sequence

A.2.1 Stage-Level Inspection

The following sequence separates archive inspection, DEX analysis, component matching, and complete report generation. PowerShell variables are used only to keep paths readable.

Listing A.3: Deterministic APK analysis and report workflow.

```
$apk = "data/test_apks/real/F-Droid.apk"
$db = "data/reference_db/reference_features_calibrated_combined.sqlite"
$out = "data/results/reproduction-fdroid"

docker compose run --rm apk-sbom python -m apk_sbom.cli inspect-apk $apk
docker compose run --rm apk-sbom python -m apk_sbom.cli analyze-dex $apk
docker compose run --rm apk-sbom python -m apk_sbom.cli match-apk $apk `
  --db-path $db --top 10 --min-score 0.05
docker compose run --rm apk-sbom python -m apk_sbom.cli generate-report $apk `
  --output-dir $out --db-path $db --top 30 --skip-osv
```

The stable shared database used by the ordinary lane is `data/reference_db/reference_features_calibrated_combined.sqlite`. The `--skip-osv` flag makes the run independent of the availability of the OSV service. It does not disable extraction, matching, SBOM generation, or the local report.

A.2.2 Expected Output Package

A successful `generate-report` run normally writes the files listed in Table A.2.

Table A.2: Principal generated report artifacts.

Artifact	Purpose
`matches.json`	Ranked component candidates, evidence scores, confidence, selected versions, and ambiguity information.
`sbom.cdx.json`	CycloneDX SBOM containing accepted Maven components, Package URLs, evidence properties, and hash-based native file components.
`osv-vulnerabilities.json`	OSV responses and normalized vulnerability findings when online scanning is enabled.
`report.json`	Combined machine-readable report used by evaluation and dashboard scripts.
`report.md`	Human-readable Markdown summary.
`report.html`	Browser-oriented report and the principal demonstration surface.

The narrower CLI commands are useful when a stage fails: `inspect-apk` checks archive and manifest evidence; `analyze-dex` checks parsed bytecode features; `match-apk` checks

candidate ranking; `generate-sbom` writes the SBOM without the full report; and `scan-sbom-osv` performs vulnerability enrichment separately.

A.3 Reference-Database Construction

A.3.1 Isolated Build Sequence

Reference artifacts must be built in isolated paths unless a later promotion gate explicitly authorizes their inclusion in the shared calibrated database. A minimal isolated build is:

Listing A.4: Isolated reference-database construction.

```
$config = "configs/libraries.json"
$artifacts = "data/reference_db/artifacts_reproduction"
$dex = "data/reference_db/dex_reproduction"
$db = "data/reference_db/reference_features_reproduction.sqlite"

docker compose run --rm apk-sbom python -m apk_sbom.cli download-maven `
  --config $config --output-root $artifacts --dry-run
docker compose run --rm apk-sbom python -m apk_sbom.cli download-maven `
  --config $config --output-root $artifacts --quiet
docker compose run --rm apk-sbom python -m apk_sbom.cli convert-reference-dex `
  --artifact-root $artifacts --output-root $dex --quiet
docker compose run --rm apk-sbom python -m apk_sbom.cli `
  extract-reference-features --dex-root $dex --db-path $db --reset --quiet
```

The dry run exposes the planned Maven requests before network or disk work. JAR files are converted directly; AAR files are unpacked and their bytecode is converted to a common DEX representation. Feature extraction then writes package, class, method, opcode, API-reference, graph, literal, and contextual evidence to SQLite.

A.3.2 Reset and Promotion Safety

The `--reset` flag is destructive for the database path supplied to the command. It is therefore valid only for a new or explicitly rebuildable experiment database. Incremental batches omit `--reset`. An experiment is not promoted merely because target rows improve: at least two affected targets and clean, partial, no-Maven-signal, and boundary controls are checked first. Any regression fails the promotion gate. This policy is why several useful version and R8 experiments remain report-only or experiment-only.

A.4 Source Validation and Vulnerability Enrichment

OSV scanning can be run after an offline report:

Listing A.5: OSV enrichment and source validation.

```
docker compose run --rm apk-sbom python -m apk_sbom.cli scan-sbom-osv `
  "$out/sbom.cdx.json" --output "$out/osv-vulnerabilities.json"
```

```
docker compose run --rm apk-sbom python -m apk_sbom.cli validate-source-deps `
  --sbom "$out/sbom.cdx.json" `
  --dependency-tree "path/to/runtimeClasspath.txt" `
  --output-dir "$out/source-validation"
```

Source validation is an evaluation operation. It compares emitted Maven coordinates and versions with a resolved Gradle dependency tree when source is available. The dependency tree is not an input to normal APK matching. Similarly, OSV can determine whether a supplied package version appears in an affected range, but it cannot repair an uncertain or incorrect component version. The report therefore preserves confirmed, potential, mismatch, and report-only vulnerability classes.

A.5 Dataset and Evidence Inventory

Table A.3: Evidence classes and their permitted interpretation.

Class	Role	Permitted claim
Controlled fixtures	Generated APKs with known embedded versions	Exact-version recovery under the represented clear, R8, multidex, and multi-library transformations.
Clean source-validated	Real APKs whose emitted Maven versions all match the resolved source evidence	Clean emitted-version confirmation, reported separately from harder applications.
Partial source-validated	Real APKs containing both confirmed and unresolved or wrong-version rows	Useful partial recovery; not a total success or total failure.
Source-validated no-Maven signal	Real APKs with resolved runtime dependency trees but zero emitted Maven components	Matcher-recall and emission-threshold controls; excluded from the emitted-version rate but included in the full source denominator.
Full source denominator	All resolved source coordinate–version rows	End-to-end recall, including dependencies that the matcher never emits.
Source-known upper bound	Candidate selection constrained by source versions	Recognition capacity under exact source-version constraints; not APK-only accuracy.
Report-only/experimental	AI, R8, reranking, and scoped database lanes	Diagnostic assistance or promotion evidence only; no change to official components, versions, SBOMs, CVEs, or source-validation metrics.
Boundary/limitation	Native-heavy, hidden-code, heavily transformed, or no-Maven-signal cases	Demonstrated system boundary; excluded from Maven exact-version accuracy when no valid denominator exists.

The authoritative row-level and aggregate index is generated at `docs/thesis/THESIS_ARTIFACT_INDEX.md`. The principal frozen evidence snapshots used for the final chapters are `docs/thesis/PFE_CHAPTER6_EVIDENCE_SNAPSHOT_2026-06-20.md` and `docs/thesis/PFE_CHAPTER3_AI_EVIDENCE_SNAPSHOT_2026-06-20.md`.

A.6 Claim-to-Artifact Traceability

Table A.4 provides a compact audit map. The Markdown and JSON artifacts remain the authoritative detailed sources; the table does not replace their denominators, exclusions, and row-level data.

Table A.4: Principal thesis claims and reproducibility artifacts.

Claim or result	Frozen value	Primary artifact
Controlled exact-version recovery	84/84 expected versions across 72 fixtures	`data/results/evaluation/thesis-evaluation-tables.json`; `docs/evaluation/THESIS_EVALUATION_TABLES.md`
Clean real-APK confirmation	47/47 emitted Maven versions across five APKs	`docs/evaluation/THESIS_EVALUATION_TABLES.md`
Partial real-APK confirmation	880/1,790 emitted Maven versions across 97 APKs	`docs/evaluation/THESIS_EVALUATION_TABLES.md`
Source-validated APKs with emitted Maven rows	927/1,837 emitted Maven versions across 102 APKs, or 50.5%	`data/results/evaluation/thesis-evaluation-tables.json`
Source-validated no-Maven-signal controls	Four APKs with resolved runtime dependency trees and zero emitted Maven rows	`docs/evaluation/NO_MAVEN_SIGNAL_AUDIT.md`
Full source dependency denominator	927/12,082 exact rows emitted, or 7.7%; shared-DB exact-version coverage 1,231/12,082, or 10.2%	`data/results/evaluation/reference-db-coverage-first-dashboard.json`; `docs/evaluation/REFERENCE_DB_COVERAGE_FIRST_DASHBOARD.md`
Source-known upper bound	6,772/6,772 emitted rows under exact source-version constraints	`docs/evaluation/DEMO_FINAL_LARGE_V2_SOURCE_VERSION_EVALUATION.md`; `docs/evaluation/DEMO_FINAL_LARGE_V2_SOURCE_GAP_TRIAGE.md`

Claim or result	Frozen value	Primary artifact
AI authority decision	MVP-safe structured Transformer macro-F1 0.882; raw-opcode Transformer 0.110 below the 0.128 baseline; report-only	`data/results/evaluation/ai-r8-final-evaluation.json`; `data/results/evaluation/ai-r8-mvp-integration-gate.json`
Accepted and rejected experiments	Promotion status, controls, failure reason, and next evidence requirement	`docs/thesis/EXPERIMENT_DECISION_LEDGER.md`
Defense/demo package	Rehearsed artifact order and safe demonstration commands	`docs/evaluation/FINAL_DEMO_RUNBOOK_V2.md`; `docs/evaluation/FINAL_DEMO_CHECKLIST.md`

A.7 Reproduction Constraints

Some operations depend on mutable external infrastructure. Maven downloads can fail because an artifact moved, a legacy repository disappeared, or Gradle metadata no longer resolves under a modern toolchain. OSV lookup requires network access and can change as advisories are updated. Large reference databases also make unconstrained matching materially slower than a small demonstration database. For these reasons, generated JSON, source dependency trees, proof bundles, decision ledgers, and dated evidence snapshots are retained alongside commands.

Reproduction must also preserve the distinction between rerunning an existing frozen artifact and recomputing it with current remote services. A different OSV response, Maven repository state, or dependency-resolution result does not retroactively change the thesis result; it creates a new dated observation that must be reported separately. Finally, successful command execution does not by itself authorize promotion. Shared-database, matcher, AI, and vulnerability decisions remain subject to the target/control and evidence boundaries defined in this thesis.

Bibliography

[1] Jon Boyens et al. *Cybersecurity Supply Chain Risk Management Practices for Systems and Organizations*. Tech. rep. NIST Special Publication 800-161 Revision 1. National Institute of Standards and Technology, 2022. DOI: 10.6028/NIST.SP.800-161r1.

[2] National Telecommunications and Information Administration. *Software Bill of Materials*. URL: https://www.ntia.gov/page/software-bill-materials (visited on 06/20/2026).

[3] SPDX Workgroup. *SPDX Specifications*. URL: https://spdx.dev/use/specifications/ (visited on 06/19/2026).

[4] OWASP CycloneDX Project. *CycloneDX Specification*. URL: https://cyclonedx.org/specification/overview/ (visited on 06/19/2026).

[5] Android Developers. *Shrink, Obfuscate, and Optimize Your App*. URL: https://developer.android.com/build/shrink-code (visited on 06/19/2026).

[6] Michael Backes, Sven Bugiel, and Erik Derr. "Reliable Third-Party Library Detection in Android and Its Security Applications." In: *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. 2016, pp. 356–367. DOI: 10.1145/2976749.2978333.

[7] Xian Zhan et al. "ATVHUNTER: Reliable Version Detection of Third-Party Libraries for Vulnerability Identification in Android Applications." In: *Proceedings of the 43rd IEEE/ACM International Conference on Software Engineering*. 2021, pp. 1695–1707. DOI: 10.1109/ICSE43902.2021.00150.

[8] Bolin Zhou et al. "Version-Level Third-Party Library Detection in Android Applications via Class Structural Similarity." In: *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering*. 2025, pp. 604–615. DOI: 10.1145/3756681.3756933.

[9] Xian Zhan et al. "A Systematic Assessment on Android Third-Party Library Detection Tools." In: *arXiv preprint arXiv:2108.01964* (2021). DOI: 10.48550/arXiv.2108.01964.

[10] Jintao Gu et al. "Revisiting Third-Party Library Detection: A Ground Truth Dataset and Its Implications Across Security Tasks." In: *arXiv preprint arXiv:2509.04091* (2025). DOI: 10.48550/arXiv.2509.04091.

[11] Package URL Project. *Package URL Specification*. URL: https://github.com/package-url/purl-spec (visited on 06/19/2026).

[12] CVE Program. *CVE Program Overview*. URL: https://www.cve.org/About/Overview (visited on 06/20/2026).

[13] MITRE. *About CWE*. URL: https://cwe.mitre.org/about/index.html (visited on 06/20/2026).

[14] National Institute of Standards and Technology. *National Vulnerability Database: General Information*. URL: https://nvd.nist.gov/general (visited on 06/20/2026).

[15] Forum of Incident Response and Security Teams. *Common Vulnerability Scoring System*. URL: https://www.first.org/cvss/ (visited on 06/20/2026).

[16] Forum of Incident Response and Security Teams. *Exploit Prediction Scoring System*. URL: https://www.first.org/epss/ (visited on 06/20/2026).

[17] Cybersecurity and Infrastructure Security Agency. *Known Exploited Vulnerabilities Catalog*. URL: https://www.cisa.gov/known-exploited-vulnerabilities-catalog (visited on 06/20/2026).

[18] Open Source Vulnerabilities Project. *OSV Schema*. URL: https://ossf.github.io/osv-schema/ (visited on 06/20/2026).

[19] Open Source Vulnerabilities Project. *OSV API Documentation*. URL: https://google.github.io/osv.dev/api/ (visited on 06/20/2026).

[20] Android Developers. *Application Fundamentals*. URL: https://developer.android.com/guide/components/fundamentals (visited on 06/19/2026).

[21] Android Developers. *App Manifest Overview*. URL: https://developer.android.com/guide/topics/manifest/manifest-intro (visited on 06/20/2026).

[22] Android Developers. *Configure Your Build*. URL: https://developer.android.com/build (visited on 06/20/2026).

[23] Android Developers. *Create an Android Library*. URL: https://developer.android.com/studio/projects/android-library (visited on 06/20/2026).

[24] Gradle. *Variant-Aware Dependency Resolution*. URL: https://docs.gradle.org/current/userguide/variant_aware_resolution.html (visited on 06/20/2026).

[25] Android Developers. *Analyze Your Build with the APK Analyzer*. URL: https://developer.android.com/studio/debug/apk-analyzer (visited on 06/20/2026).

[26] Android Developers. *Enable Multidex for Apps with over 64K Methods*. URL: https://developer.android.com/build/multidex (visited on 06/20/2026).

[27] Android Developers. *Sign Your App*. URL: https://developer.android.com/studio/publish/app-signing (visited on 06/20/2026).

[28] Android Developers. *Android ABIs*. URL: https://developer.android.com/ndk/guides/abis (visited on 06/20/2026).

[29] Android Developers. *Dynamic Code Loading*. URL: https://developer.android.com/privacy-and-security/risks/dynamic-code-loading (visited on 06/20/2026).

[30] Android Developers. *d8*. URL: https://developer.android.com/tools/d8 (visited on 06/20/2026).

[31] OWASP Foundation. *Static Analysis*. URL: https://mas.owasp.org/MASTG/techniques/android/MASTG-TECH-0014/ (visited on 06/20/2026).

[32] OWASP Foundation. *Reverse Engineering Android Apps*. URL: https://mas.owasp.org/MASTG/techniques/android/MASTG-TECH-0013/ (visited on 06/20/2026).

[33] Leo Breiman. "Random Forests." In: *Machine Learning* 45.1 (2001), pp. 5–32. DOI: 10.1023/A:1010933404324.

[34] Ashish Vaswani et al. "Attention Is All You Need." In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017. URL: https://papers.nips.cc/paper/7181-attention-is-all-you-need (visited on 06/20/2026).

[35] Robert Geirhos et al. "Shortcut Learning in Deep Neural Networks." In: *Nature Machine Intelligence* 2 (2020), pp. 665–673. DOI: 10.1038/s42256-020-00257-z.

[36] Marina Sokolova and Guy Lapalme. "A Systematic Analysis of Performance Measures for Classification Tasks." In: *Information Processing and Management* 45.4 (2009), pp. 427–437. DOI: 10.1016/j.ipm.2009.03.002.

[37] Xian Zhan et al. "Automated Third-Party Library Detection for Android Applications: Are We There Yet?" In: *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*. 2020, pp. 919–930. DOI: 10.1145/3324884.3416582.

[38] Menghao Li et al. "LibD: Scalable and Precise Third-Party Library Detection in Android Markets." In: *Proceedings of the 39th International Conference on Software Engineering*. 2017, pp. 335–346. DOI: 10.1109/ICSE.2017.38.

[39] Yan Wang et al. "ORLIS: Obfuscation-Resilient Library Detection for Android." In: *Proceedings of the 5th IEEE/ACM International Conference on Mobile Software Engineering and Systems*. 2018. DOI: 10.1145/3197231.3197248.

[40] Yuan Zhang et al. "Detecting Third-Party Libraries in Android Applications with High Precision and Recall." In: *Proceedings of the 25th IEEE International Conference on Software Analysis, Evolution and Reengineering*. 2018, pp. 141–152. DOI: 10.1109/SANER.2018.8330204.

[41] Jiexin Zhang, Alastair R. Beresford, and Stephan A. Kollmann. "LibID: Reliable Identification of Obfuscated Third-Party Android Libraries." In: *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 2019. DOI: 10.1145/3293882.3330563.

[42] Yafei Wu et al. "LibScan: Towards More Precise Third-Party Library Identification for Android Applications." In: *32nd USENIX Security Symposium*. 2023. URL: https://www.usenix.org/conference/usenixsecurity23/presentation/wu-yafei (visited on 06/20/2026).

[43] Zifan Xie et al. "How Does Code Optimization Impact Third-Party Library Detection for Android Applications?" In: *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 2024. DOI: 10.1145/3691620.3695554.

[44] Anonymous authors. *Towards Global Matches for Third-Party Library Detection in Android: LibScope Artifact*. Public source-code and dataset artifact; author metadata was anonymized at the time of access. 2025. DOI: 10.5281/zenodo.15066884.

[45] Dahan Pan et al. "Pay Your Attention on Lib! Android Third-Party Library Detection via Feature Language Model." In: *Proceedings of the IEEE International Conference on Software Analysis, Evolution and Reengineering*. 2025, pp. 217–228. DOI: 10.1109/SANER64311.2025.00028.

[46] Vadim Safronov et al. “UniBOM: A Unified SBOM Analysis and Visualisation Tool for IoT Systems and Beyond.” In: *Proceedings of the 15th International Conference on the Internet of Things*. 2025, pp. 86–94. DOI: 10.1145/3770501.3770512.